

DITADO ESTRELADO: CÓDIGO-FONTE

Programa de computador para registro no INPI (Edital PRPGI/IFMA nº 14/2026).

Autor: Marcus Vinicius Souza Batista Strabello, Mestrado em Computação Aplicada (PPGCA), IFMA, Campus Monte Castelo.

Versão: commit 6f692af0d94a0637a39889d9dd1436996d0bfacc do repositório

github.com/MarcusViniciussb/DitadoEstrelado.

Linguagens: C# (Unity 2022.3), Python 3, Java (Android).

Arquivos: 26, com 7244 linhas de código de autoria própria.

Não fazem parte deste código (componentes de terceiros, apenas utilizados): motor Unity e TextMesh Pro; Google MediaPipe e modelo hand_landmarker.task (Apache 2.0); OpenCV; modelos 3D de Quaternius e Kay Lousberg (licenças livres); imagens do alfabeto de libras.com.br; scripts de exemplo do Unity (TutorialInfo) e do pacote Low Poly Fruits.

Índice

1. Assets/AlfabetoData.cs
2. Assets/ChuvaDeConfetes.cs
3. Assets/Cintilar.cs
4. Assets/ControladorCamera.cs
5. Assets/Editor/ConfigurarIcones.cs
6. Assets/Editor/ConstrutorAndroid.cs
7. Assets/Editor/ConstrutorWindows.cs
8. Assets/Editor/PrepararRecursos.cs
9. Assets/Flutuar.cs
10. Assets/GerenciadorDeAudio.cs
11. Assets/GerenciadorDeJogo.cs
12. Assets/HoverButton.cs
13. Assets/MenuPrincipal.cs
14. Assets/ModoEstudo.cs
15. Assets/Plugins/Android/RastreadorMediaPipe.java
16. Assets/RastreadorExterno.cs
17. Assets/RastreadorMediaPipeAndroid.cs
18. Assets/ReconhecedorLibras.cs
19. Assets/ReproduzirAnimacao.cs
20. Assets/RespirarSuave.cs
21. Assets/UIControle.cs
22. Assets/UIFabrica.cs
23. Assets/VisualizadorMaoUI.cs
24. RastreadorPython/gerar_figuras.py
25. RastreadorPython/importar_dataset.py
26. RastreadorPython/rastreador_maos.py

Assets/AlfabetoData.cs (31 linhas)

```
using UnityEngine;
using System.Collections.Generic;

[CreateAssetMenu(fileName = "NovoAlfabeto", menuName = "IA/Banco de LIBRAS")]
public class AlfabetoData : ScriptableObject
{
    // — Letras estáticas: uma "foto" dos 21 pontos —————
    [System.Serializable]
    public struct LetraPadrao
    {
        public string nome;
        public Vector3[] pontosNormalizados;
    }
    public List<LetraPadrao> letrasGravadas = new List<LetraPadrao>();

    // — Letras dinâmicas (H, J, K, X, Z...): um "filme" do movimento —————
    [System.Serializable]
    public struct QuadroDeMao
    {
        public Vector3[] pontos; // 21 pontos relativos ao pulso, num instante
    }

    [System.Serializable]
    public struct SinalDinamico
    {
        public string nome;
        public List<QuadroDeMao> quadros; // sequência de ~1,3s do movimento
    }
    public List<SinalDinamico> sinaisDinamicos = new List<SinalDinamico>();
}
```

Assets/ChuvaDeConfetes.cs (88 linhas)

```
using UnityEngine;
using UnityEngine.UI;
using System.Collections.Generic;

// ChuvaDeConfetes: cria dezenas de papeizinhos coloridos que caem girando
// pela tela (comemoração de vitória). Se destrói sozinho ao terminar.
//
// Uso: ChuvaDeConfetes.Lancar(canvas.transform);
public class ChuvaDeConfetes : MonoBehaviour
{
    class Confete
    {
        public RectTransform rt;
        public float velocidadeQueda;
        public float velocidadeGiro;
        public float balanço; // deriva horizontal
        public float fase;
    }

    private readonly List<Confete> confetes = new List<Confete>();
    private float alturaTela;
    private float vida = 6f; // segundos até se limpar

    private static readonly Color[] CORES =
    {
        new Color(1f, 0.85f, 0.25f), // amarelo
        new Color(0.30f, 0.75f, 1f), // azul
        new Color(0.35f, 0.85f, 0.4f), // verde
        new Color(1f, 0.45f, 0.55f), // rosa
        new Color(0.75f, 0.55f, 1f), // lilás
        Color.white,
    };

    public static void Lancar(Transform canvas)
    {
        var go = new GameObject("ChuvaDeConfetes", typeof(RectTransform));
        go.layer = 5;
        var rt = go.GetComponent<RectTransform>();
        rt.SetParent(canvas, false);
        rt.anchorMin = Vector2.zero;
        rt.anchorMax = Vector2.one;
        rt.sizeDelta = Vector2.zero;
        rt.SetAsLastSibling();
        go.AddComponent<ChuvaDeConfetes>();
    }

    void Start()
    {
        var meuRect = (RectTransform)transform;
        float largura = meuRect.rect.width;
        alturaTela = meuRect.rect.height;

        for (int i = 0; i < 60; i++)
        {
            var img = UIFabrica.CriarImagem(transform, "Confete",
                CORES[Random.Range(0, CORES.Length)],
                new Vector2(Random.Range(-largura / 2f, largura / 2f),
                    alturaTela / 2f + Random.Range(20f, 500f)),
                new Vector2(Random.Range(14f, 26f), Random.Range(8f, 14f)));
            img.raycastTarget = false;

            confetes.Add(new Confete
            {
                rt = img.rectTransform,
                velocidadeQueda = Random.Range(250f, 550f),
                velocidadeGiro = Random.Range(-360f, 360f),
                balanço = Random.Range(30f, 90f),
                fase = Random.Range(0f, Mathf.PI * 2f),
            });
        }
    }

    void Update()
    {
        foreach (var c in confetes)
        {
            Vector2 pos = c.rt.anchoredPosition;
            pos.y -= c.velocidadeQueda * Time.deltaTime;
            pos.x += Mathf.Sin(Time.time * 2.5f + c.fase) * c.balanço * Time.deltaTime;
            c.rt.anchoredPosition = pos;
            c.rt.Rotate(0, 0, c.velocidadeGiro * Time.deltaTime);
        }

        vida -= Time.deltaTime;
        if (vida <= 0f) Destroy(gameObject);
    }
}
```

Assets/Cintilar.cs (39 linhas)

```
using UnityEngine;
using UnityEngine.UI;

// Cintilar: faz uma imagem de UI pulsar como uma estrela no céu -
// o brilho (alfa) e o tamanho sobem e descem suavemente, cada estrela
// no seu próprio ritmo (velocidade e fase sorteadas).
public class Cintilar : MonoBehaviour
{
    [Range(0.2f, 6f)] public float velocidade = 2f;
    [Range(0f, 1f)] public float alfaMinimo = 0.15f;
    [Range(0f, 1f)] public float alfaMaximo = 1f;
    [Range(0f, 1f)] public float escalaExtra = 0.35f; // quanto "incha" no pico

    private Image imagem;
    private float fase;
    private Vector3 escalaBase;

    void Awake()
    {
        imagem = GetComponent<Image>();
        escalaBase = transform.localScale;
        fase = Random.Range(0f, Mathf.PI * 2f);
        velocidade *= Random.Range(0.5f, 1.6f); // cada estrela num ritmo próprio
    }

    void Update()
    {
        if (imagem == null) return;

        float s = (Mathf.Sin(Time.unscaledTime * velocidade + fase) + 1f) * 0.5f;

        var cor = imagem.color;
        cor.a = Mathf.Lerp(alfaMinimo, alfaMaximo, s);
        imagem.color = cor;

        transform.localScale = escalaBase * (1f + escalaExtra * s);
    }
}
```

Assets/ControladorCamera.cs (1234 linhas)

```
using UnityEngine;
using UnityEngine.UI;
using MediaPipe.HandLandmark;
using System.Collections;
using System.Collections.Generic;
// Apelidos para não confundir System.Diagnostics.Debug com UnityEngine.Debug
using Processo = System.Diagnostics.Process;
using InfoDeProcesso = System.Diagnostics.ProcessStartInfo;

// ControladorCamera: conecta a webcam ao detector de mão (HandLandmarkDetector)
// e passa os 21 pontos da mão para o ReconhecedorLibras.
// Este script fica num GameObject que também tem a RawImage da câmera.
public class ControladorCamera : MonoBehaviour
{
    [Header("Referências de Cena")]
    public RawImage fundoDoEcra; // RawImage que exibe o feed da câmera
    public ResourceSet ficheirosDaIA; // ScriptableObject do plugin HandLandmark
    // Detector de palma: primeiro estagio do MediaPipe, usado no celular para
    // localizar a mão e a sua inclinacao antes de chamar o modelo de pontos
    public MediaPipe.BlazePalm.ResourceSet ficheirosDaPalma;
    public Transform bolinhaDoDedo; // Objeto 3D que segue a ponta do indicador
    public ReconhecedorLibras reconhecedor;

    [Header("Calibração Visual (posição da bolinha)")]
    public float escalaX = -15f;
    public float escalaY = 10f;
    public float translaX = 0f;
    public float translaY = 0f;

    [Header("Controle do Jogo")]
    public bool MODO_TREINAMENTO = true;
    public GerenciadorDeJogo gerenciador; // Arraste o GameObject que tem GerenciadorDeJogo aqui

    [Header("Rastreador externo (Python/MediaPipe) - opcional")]
    // Se o script RastreadorPython estiver rodando, usa o MediaPipe completo
    // (muito melhor). Senão, cai automaticamente no rastreador interno.
    public RastreadorExterno rastreadorExterno;
    public bool iniciarRastreadorAutomaticamente = true; // Unity abre o Python sozinho!
    // Tempo máximo aguardando o Python carregar (a 1ª vez pode ser lenta:
    // antivírus + carga do MediaPipe). Enquanto o processo estiver vivo, espera.
    // (campo renomeado de "esperaRastreadorExterno" para o Unity esquecer o
    // valor antigo de 3s que ficou memorizado na cena)
    public float esperaMaximaRastreador = 45f;
    private bool usandoExterno = false;
    private Processo processoRastreador; // guardamos para fechar junto com o jogo

    [Header("Reconhecimento")]
    public float cooldownReconhecimento = 0.9f; // segundos mínimos entre duas letras reconhecidas
    public float tempoEstabilidade = 0.35f; // segundos segurando o MESMO sinal para valer
    private float tempoUltimoReconhecimento = -99f;
    private string letraCandidata = "";
    private float tempoLetraCandidata = 0f;

    // Lidos pela interface para mostrar "estou quase reconhecendo a letra X"
    public string LetraCandidata { get; private set; } = "";
    public float ProgressoCandidata { get; private set; } = 0f;
    public bool EsperandoMovimento { get; private set; } = false; // letra atual é dinâmica

    [Header("Letras dinamicas (com movimento)")]
    public float duracaoGravacaoMovimento = 1.3f; // segundos gravados no treinamento
    public float duracaoJanelaMovimento = 1.4f; // "memória" de movimento no jogo
    public float intervaloChecagemMovimento = 0.25f; // de quanto em quanto compara

    // Janela deslizante: os últimos ~1,4s de mão (usada pelo DTW)
    private List<Vector3[]> janelaMovimento = new List<Vector3[]>();

    // Leitura da janela para outras telas (usada pela pratica do modo estudo)
    public List<Vector3[]> JanelaDeMovimento { get { return janelaMovimento; } }
    private List<float> temposJanela = new List<float>();
    private float tempoUltimaAmostraJanela = 0f;
    private float tempoUltimaChecagemMovimento = 0f;

    [Header("Exibicao da camera")]
    public bool espelharImagem = true; // modo "selfie" (RawImage com escala X = -1)
    public bool inverterVertical = false; // ligue se o esqueleto aparecer de cabeça para baixo

    // Margem de segurança: a tela mostra só o miolo do sensor. Assim a mão no
    // CANTO DA TELA ainda está DENTRO do campo de visão da câmera, com folga
    // para o rastreador. 1.2 = tela mostra ~83% do quadro da câmera.
    [Range(1f, 1.5f)] public float zoomDaTela = 1.2f;
    private float zoomAnterior = 0f;

    [Header("Zoom inteligente (2 estagios, como o MediaPipe original)")]
    // Em vez de mandar o frame INTEIRO espremido para a IA, recorta um quadrado
    // ao redor da última posição da mão - a mão chega grande e nítida ao modelo.
    // Se algo ficar estranho, desligue aqui para voltar ao comportamento antigo.
    public bool zoomInteligente = true;
    [Range(1.5f, 4f)] public float margemDoZoom = 2.4f; // recorte = tamanho da mão × margem
```

```

// — Suporte ao celular —————
// Tudo daqui so entra em acao na compilacao para Android. No computador
// estes campos ficam inertes e o caminho percorrido e o de sempre.
private Material materialDeGiro;
private RenderTexture texturaExibicao;
private float larguraDoQuadro = 0f, alturaDoQuadro = 0f;
private float anguloDaMao = 0f;
private bool alinharMao = false;

// MediaPipe nativo: no celular ele assume todo o rastreamento. No
// computador esta referencia fica nula e o caminho de sempre continua
// valendo, porque quem rastreia por la e o MediaPipe do Python.
private RastreadorMediaPipeAndroid mediapipe;
private bool MediaPipeAtivo { get { return mediapipe != null && mediapipe.Pronto; } }
#if UNITY_ANDROID && !UNITY_EDITOR
private MediaPipe.BlazePalm.PalmDetector detectorDePalma;
private RenderTexture texturaDaPalma;
private float tempoDaProximaBuscaDePalma = 0f;
private float ladoDoQuadradoDaPalma = 0f;
#endif

public int GiroDaCamera { get; private set; }
public bool InverterVertical { get; private set; }
public bool InverterHorizontal { get; private set; }

public void DefinirAjusteDaCamera(int giro, bool vertical, bool horizontal)
{
    GiroDaCamera = ((giro % 360) + 360) % 360;
    InverterVertical = vertical;
    InverterHorizontal = horizontal;
}
#if UNITY_ANDROID && !UNITY_EDITOR
AtualizarTamanhoDoQuadro();
#endif

// Informacoes do painel de diagnostico
public string FonteDeRastreamento =>
    MediaPipeAtivo ? "MediaPipe (celular)"
    : (usandoExterno ? "externa (MediaPipe)"
    : (detetivePronto ? "interna" : "iniciando"));

public float ConfiancaAtual =>
    usandoExterno ? (rastreadorExterno != null ? rastreadorExterno.Score : 0f)
    : (detetivePronto ? detetive.Score : 0f);

public string ResolucaoDaCamera =>
    (minhaCamera == null) ? "-" : minhaCamera.width + "x" + minhaCamera.height;

private RenderTexture texturaRecorte; // quadrado enviado à IA
private Vector2 centroCaixa = new Vector2(0.5f, 0.5f); // centro do recorte (normalizado)
private float ladoCaixaPx = 0f; // lado do recorte em px (0 = busca ampla)
private Vector2 escalaBlit = Vector2.one; // último recorte aplicado
private Vector2 offsetBlit = Vector2.zero; // (para mapear pontos de volta)
private int framesSemMao = 0;

private WebCamTexture minhaCamera;
private HandLandmarkDetector detetive;
private bool cameraPronta = false;
private bool detetivePronto = false;

// Recorte central aplicado ao feed para NÃO esticar a imagem na tela
private Rect uvRecorte = new Rect(0, 0, 1, 1);
private int larguraTelaAnterior = 0;
private int alturaTelaAnterior = 0;

// Pontos da mão atuais - lidos pelo VisualizadorMaoUI para desenhar o esqueleto
public Vector3[] PontosDaMaoAtuais { get; private set; }
public bool MaoDetectada { get; private set; }

// Converte um keypoint (0..1 no frame INTEIRO da câmera) para posição na
// tela em pixels, já descontando o recorte e o espelho.
// O detector usa a convenção do Unity: Y cresce para CIMA.
// Todo script que precisa saber onde a mão está na tela deve usar isto.
public Vector2 PontoParaTela(Vector3 kp)
{
    float u = (kp.x - uvRecorte.x) / uvRecorte.width;
    float v = (kp.y - uvRecorte.y) / uvRecorte.height;
    if (espelharImagem) u = 1f - u;
    if (inverterVertical) v = 1f - v;
    return new Vector2(u * Screen.width, v * Screen.height);
}

void Start()
{
}
#if UNITY_ANDROID && !UNITY_EDITOR
// No celular nao existe o rastreador em Python
StartCoroutine(PrepararCameraDoCelular());
return;
#endif
#pragma warning disable 0162
#endif
if (rastreadorExterno != null && rastreadorExterno.enabled)
    StartCoroutine(EscolherFonteDeRastreamento());
else

```

```

        IniciarCameraInterna());
#if UNITY_ANDROID && !UNITY_EDITOR
#pragma warning restore 0162
#endif
    }

    // Fluxo de escolha da fonte de rastreamento:
    // 1) Já tem um rastreador Python rodando (aberto na mão)? Usa ele.
    // 2) Senão, o próprio Unity abre o Python invisível em segundo plano.
    // 3) Se nada funcionar, liga o rastreador interno antigo (fallback).
    IEnumerator EscolherFonteDeRastreamento()
    {
        Debug.Log("Procurando rastreador externo (Python/MediaPipe)...");

        // Alguém já abriu o rastreador manualmente?
        float fimVerificacao = Time.time + 0.7f;
        while (Time.time < fimVerificacao)
        {
            if (rastreadorExterno.Ativo)
            {
                usandoExterno = true;
                Debug.Log("Rastreador EXTERNO ja estava rodando - conectado!");
                yield break;
            }
            yield return null;
        }

        // Abre o Python sozinho (invisível); ele é fechado junto com o jogo
        if (iniciarRastreadorAutomaticamente)
            IniciarProcessoDoRastreador();

        // Nenhum processo foi aberto (Python ausente)? Não perde tempo esperando
        if (processoRastreador == null)
        {
            Debug.Log("Usando o rastreador interno.");
            IniciarCameraInterna();
            yield break;
        }

        float inicioEspera = Time.time;
        float proximoRelato = Time.time + 4f;
        bool morreu = false;
        Debug.Log("Aguardando o rastreador por ate " + esperaMaximaRastreador + "s...");

        while (Time.time - inicioEspera < esperaMaximaRastreador)
        {
            if (rastreadorExterno.Ativo)
            {
                usandoExterno = true;
                Debug.Log("Rastreador EXTERNO conectado! (MediaPipe completo, com profundidade Z)");
                yield break;
            }

            // O Python abriu e morreu? Mostra o erro DELE no Console e desiste já
            try { morreu = processoRastreador.HasExited; } catch { morreu = true; }
            if (morreu)
            {
                string logsErro;
                lock (logsDoRastreador) logsErro = logsDoRastreador.ToString();
                Debug.LogWarning("O rastreador Python fechou sozinho. Mensagens dele:\n" + logsErro);
                break;
            }

            // Mostra o progresso do carregamento (1ª vez pode demorar bastante)
            if (Time.time >= proximoRelato)
            {
                proximoRelato = Time.time + 4f;
                string ultimaLinha = "";
                lock (logsDoRastreador)
                {
                    string tudo = logsDoRastreador.ToString().TrimEnd();
                    int quebra = tudo.LastIndexOf('\n');
                    ultimaLinha = (quebra >= 0) ? tudo.Substring(quebra + 1) : tudo;
                }
                Debug.Log("Aguardando o MediaPipe carregar... (" +
                    Mathf.RoundToInt(Time.time - inicioEspera) + "s) " +
                    (ultimaLinha.Length > 0 ? "| Python: " + ultimaLinha : ""));
            }

            yield return null;
        }

        if (!morreu)
            Debug.Log("Rastreador externo nao respondeu a tempo - usando o rastreador interno.");
        EncerrarProcessoDoRastreador(); // não deixa processo órfão segurando a câmera
        IniciarCameraInterna();
    }

    // Guarda as últimas linhas impressas pelo Python (para mostrar se ele falhar)
    private System.Text.StringBuilder logsDoRastreador = new System.Text.StringBuilder();

    // Executa RastreadorPython/rastreador_maos.py sem abrir janela nenhuma

```

```

void IniciarProcessoDoRastreador()
{
    // Pasta RastreadorPython: fica na raiz do projeto (no Editor) ou ao
    // lado do executável (num build) - os dois casos são o pai de dataPath
    string raiz = System.IO.Directory.GetParent(Application.dataPath).FullName;
    string pasta = System.IO.Path.Combine(raiz, "RastreadorPython");
    string script = System.IO.Path.Combine(pasta, "rastreador_maos.py");

    if (!System.IO.File.Exists(script))
    {
        Debug.LogWarning("Rastreador: script nao encontrado em " + script);
        return;
    }

    foreach (string executavel in CandidatosAPython())
    {
        try
        {
            var info = new InfoDeProcesso
            {
                FileName = executavel,
                Arguments = "-u rastreador_maos.py",
                WorkingDirectory = pasta,
                UseShellExecute = false,
                CreateNoWindow = true, // invisível!
                RedirectStandardOutput = true, // capturamos o que ele imprime
                RedirectStandardError = true,
            };
            processoRastreador = Processo.Start(info);
            processoRastreador.OutputDataReceived += (s, e) => GuardarLogDoRastreador(e.Data);
            processoRastreador.ErrorDataReceived += (s, e) => GuardarLogDoRastreador(e.Data);
            processoRastreador.BeginOutputReadLine();
            processoRastreador.BeginErrorReadLine();
            Debug.Log("Rastreador Python iniciado em segundo plano: " + executavel);
            return;
        }
        catch { /* esse executavel nao funcionou - tenta o proximo */ }
    }
    Debug.LogWarning("Rastreador: Python nao encontrado no PC. " +
        "Abra EXECUTAR_RASTREADOR.bat manualmente ou instale o Python.");
}

// Lista de possíveis executáveis do Python, do mais confiável ao menos.
// IMPORTANTE: os atalhos "py"/"python" do PATH podem ser aliases especiais
// do Windows (WindowsApps) que FALHAM quando abertos pelo Unity - por isso
// procuramos primeiro o python.exe REAL nas pastas de instalação.
IEnumerable<string> CandidatosAPython()
{
    string local = System.Environment.GetFolderPath(
        System.Environment.SpecialFolder.LocalApplicationData);

    // 1) Gerenciador novo do Python (ex: AppData\Local\Python\pythoncore-3.14-64)
    string pastaNova = System.IO.Path.Combine(local, "Python");
    if (System.IO.Directory.Exists(pastaNova))
        foreach (string dir in System.IO.Directory.GetDirectories(pastaNova, "pythoncore-*"))
        {
            string exe = System.IO.Path.Combine(dir, "python.exe");
            if (System.IO.File.Exists(exe)) yield return exe;
        }

    // 2) Instalador clássico do python.org (AppData\Local\Programs\Python\Python3xx)
    string pastaClassica = System.IO.Path.Combine(local, "Programs", "Python");
    if (System.IO.Directory.Exists(pastaClassica))
        foreach (string dir in System.IO.Directory.GetDirectories(pastaClassica, "Python3*"))
        {
            string exe = System.IO.Path.Combine(dir, "python.exe");
            if (System.IO.File.Exists(exe)) yield return exe;
        }

    // 3) Launcher clássico em C:\Windows
    string pyGlobal = @"C:\Windows\py.exe";
    if (System.IO.File.Exists(pyGlobal)) yield return pyGlobal;

    // 4) Por último, os nomes do PATH (podem ser os aliases problemáticos)
    yield return "py";
    yield return "python";
}

void GuardarLogDoRastreador(string linha)
{
    if (string.IsNullOrEmpty(linha)) return;
    lock (logsDoRastreador)
    {
        logsDoRastreador.AppendLine(linha);
        // guarda só as últimas ~2000 letras
        if (logsDoRastreador.Length > 2000)
            logsDoRastreador.Remove(0, logsDoRastreador.Length - 2000);
    }
}

void EncerrarProcessoDoRastreador()
{

```



```

        if (processoRastreador == null) return;
        try { if (!processoRastreador.HasExited) processoRastreador.Kill(); } catch { }
        processoRastreador = null;
    }

    void IniciarCameraInterna()
    {
        if (ficheirosDaIA == null)
        {
            Debug.LogError("FATAL: 'ficheirosDaIA' (ResourceSet) nao esta vinculado no Inspector!");
            enabled = false;
            return;
        }

        // 1280x720: imagem mais nítida na tela e mais detalhe para a IA
        // (se a webcam não suportar, o Unity escolhe a resolução mais próxima)
    #if UNITY_ANDROID && !UNITY_EDITOR
        string camera = EscolherCamera();
        minhaCamera = (camera == null) ? new WebCamTexture(1280, 720)
                                         : new WebCamTexture(camera, 1280, 720, 30);
    #else
        minhaCamera = new WebCamTexture(1280, 720);
    #endif
        minhaCamera.wrapMode = TextureWrapMode.Clamp; // fora do quadro = repete a borda
        fundoDoEcra.texture = minhaCamera;
        minhaCamera.Play();

        StartCoroutine(InicializarDetetiveAposCamera());
    }

    #if UNITY_ANDROID && !UNITY_EDITOR
    // — Medicao no aparelho —
    private TMPro.TextMeshProUGUI painelDeMedicao;
    private int palmasEncontradas = 0;
    private float tempoDoProximoTexto = 0f;

    // Painei tecnico usado para investigar o rastreamento. Fica desligado:
    // o jogador nao tem porque ver confianca, recorte e quadros por segundo.
    const bool MOSTRAR_MEDICAO = false;

    void CriarPainelDeMedicao()
    {
        if (!MOSTRAR_MEDICAO) return;

        var canvas = FindObjectOfType<Canvas>();
        if (canvas == null) return;

        // Fundo escuro atras do texto: o painel precisa ser legivel mesmo
        // numa foto ou video comprimido da tela
        var fundo = UIFabrica.CriarImagem(canvas.transform, "FundoMedicao",
            new Color(0f, 0f, 0f, 0.75f), new Vector2(0, -250), new Vector2(1000, 300),
            UIFabrica.Arredondado(), true);
        UIFabrica.Ancorar(fundo, new Vector2(0.5f, 1f), new Vector2(0.5f, 1f));
        fundo.raycastTarget = false;

        painelDeMedicao = UIFabrica.CriarTexto(fundo.transform, "Medicao", "",
            42f, new Color(0.55f, 1f, 0.55f, 1f), Vector2.zero,
            new Vector2(960, 280), false);
        painelDeMedicao.alignment = TMPro.TextAlignmentOptions.Center;
        fundo.transform.SetAsLastSibling();
    }

    void AtualizarPainelDeMedicao()
    {
        if (painelDeMedicao == null || Time.unscaledTime < tempoDoProximoTexto) return;
        tempoDoProximoTexto = Time.unscaledTime + 0.2f;

        string letraEsperada = (gerenciador != null) ? gerenciador.LetraEsperada : "-";
        string quebra = "\n";
        painelDeMedicao.text =
            "confianca " + ConfiancaAtual.ToString("F2") +
            " mao " + (MaoDetectada ? "SIM" : "nao") + quebra +
            "palmas " + palmasEncontradas +
            " recorte " + Mathf.RoundToInt(ladoCaixaPx) +
            "px giro " + Mathf.RoundToInt(anguloDaMao) + quebra +
            "esperada " + (string.IsNullOrEmpty(letraEsperada) ? "-" : letraEsperada) +
            " vista " + (reconhecedor != null ? reconhecedor.UltimaLetra : "-") + quebra +
            "distancia " + (reconhecedor != null && reconhecedor.UltimaDistancia < 9999f
                ? reconhecedor.UltimaDistancia.ToString("F2") : "-") +
            " limite " + (reconhecedor != null ? reconhecedor.limiteDeSeguranca.ToString("F1") : "-") + quebra
        +
            "quadros/s " + Mathf.RoundToInt(1f / Mathf.Max(0.0001f, Time.smoothDeltaTime));
    }

    // — Camera do celular —
    IEnumerator PrepararCameraDoCelular()
    {
        if (!UnityEngine.Android.Permission.HasUserAuthorizedPermission(
            UnityEngine.Android.Permission.Camera))
        {
            UnityEngine.Android.Permission.RequestUserPermission(

```

```

        UnityEngine.Android.Permission.Camera);

        float limite = Time.realtimeSinceStartup + 30f;
        while (!UnityEngine.Android.Permission.HasUserAuthorizedPermission(
            UnityEngine.Android.Permission.Camera) &&
            Time.realtimeSinceStartup < limite)
            yield return null;
    }

    Application.targetFrameRate = 60;
    CriarPainelDeMedicao();
    alinharMao = false; // recorte alinhado aos eixos, sem giro extra
    zoomDaTela = 1.5f; // a tela mostra menos, sobrando margem nas bordas
    IniciarCameraInterna();

    // Sobe o MediaPipe do proprio Android. Se por algum motivo ele nao
    // subir, o rastreador interno continua respondendo como antes.
    mediapipe = new RastreadorMediaPipeAndroid();
    yield return mediapipe.Iniciar();
    if (!mediapipe.Pronto)
        Debug.LogWarning("MediaPipe do celular nao iniciou: " + mediapipe.Erro);
}

string EscolherCamera()
{
    var lista = WebCamTexture.devices;
    if (lista == null || lista.Length == 0) return null;
    foreach (var d in lista)
        if (d.isFrontFacing) return d.name;
    return lista[0].name;
}

// Descobre o giro do sensor e prepara a textura ja endireitada
void PrepararCorrecaoDeGiro()
{
    GiroDaCamera = 90; // valor que corresponde ao sensor na pratica
    InverterVertical = false;
    InverterHorizontal = false;

    if (PlayerPrefs.HasKey("camGiro"))
    {
        GiroDaCamera = PlayerPrefs.GetInt("camGiro", 90);
        InverterVertical = PlayerPrefs.GetInt("camInvV", 0) == 1;
        InverterHorizontal = PlayerPrefs.GetInt("camInvH", 0) == 1;
    }

    AtualizarTamanhoDoQuadro();

    var shader = Resources.Load<Shader>("RotacaoCamera");
    if (shader == null)
    {
        Debug.LogWarning("Shader de giro nao encontrado.");
        return;
    }

    materialDeGiro = new Material(shader);
    texturaExibicao = new RenderTexture((int)larguraDoQuadro, (int)alturaDoQuadro, 0);
    texturaExibicao.wrapMode = TextureWrapMode.Clamp;
    fundoDoEcra.texture = texturaExibicao;
}

void AtualizarTamanhoDoQuadro()
{
    if (minhaCamera == null || minhaCamera.width < 100) return;

    bool deitada = (GiroDaCamera == 90 || GiroDaCamera == 270);
    larguraDoQuadro = deitada ? minhaCamera.height : minhaCamera.width;
    alturaDoQuadro = deitada ? minhaCamera.width : minhaCamera.height;

    if (texturaExibicao != null &&
        (texturaExibicao.width != (int)larguraDoQuadro ||
         texturaExibicao.height != (int)alturaDoQuadro))
    {
        texturaExibicao.Release();
        texturaExibicao = new RenderTexture((int)larguraDoQuadro, (int)alturaDoQuadro, 0);
        texturaExibicao.wrapMode = TextureWrapMode.Clamp;
        fundoDoEcra.texture = texturaExibicao;
    }
    larguraTelaAnterior = 0; // obriga o recorte de tela a recalculer
}

// Converte um ponto do quadro endireitado para a imagem original
Vector2 ParaFonte(Vector2 uvEndireitado)
{
    float rad = GiroDaCamera * Mathf.Deg2Rad;
    float cos = Mathf.Cos(rad), sin = Mathf.Sin(rad);

    Vector2 p = new Vector2((uvEndireitado.x - 0.5f) * larguraDoQuadro,
        (uvEndireitado.y - 0.5f) * alturaDoQuadro);
    Vector2 g = new Vector2(p.x * cos + p.y * sin, -p.x * sin + p.y * cos);
    if (InverterHorizontal) g.x = -g.x;
    if (InverterVertical) g.y = -g.y;
}

```

```

        return new Vector2(0.5f + g.x / minhaCamera.width,
                           0.5f + g.y / minhaCamera.height);
    }

    // Primeiro estagio do reconhecimento: procura a mao no quadro inteiro e
    // define onde recortar e quanto girar. E o que o MediaPipe do computador
    // faz e faltava aqui; sem isso o modelo de pontos so acerta quando a mao
    // ja esta bem posicionada.
    void ProcurarPalma()
    {
        if (detectorDePalma == null || materialDeGiro == null) return;

        // O detector espera uma imagem quadrada. Entregar o quadro em pe
        // esticaria a mao e derrubaria as deteccoes, entao montamos um
        // quadrado com o quadro inteiro dentro.
        if (texturaDaPalma == null)
            texturaDaPalma = new RenderTexture(256, 256, 0);

        ladoDoQuadradoDaPalma = Mathf.Max(larguraDoQuadro, alturaDoQuadro);
        CopiarComGiro(texturaDaPalma, ParaFonte(new Vector2(0.5f, 0.5f)),
                      new Vector2(ladoDoQuadradoDaPalma, ladoDoQuadradoDaPalma));

        detectorDePalma.ProcessImage(texturaDaPalma, 0.5f);
        var deteccoes = detectorDePalma.Detections;
        palmasEncontradas = deteccoes.Length;
        if (deteccoes.Length == 0) return;

        // Fica com a deteccao mais confiavel
        int melhor = 0;
        for (int i = 1; i < deteccoes.Length; i++)
            if (deteccoes[i].score > deteccoes[melhor].score) melhor = i;
        var d = deteccoes[melhor];

        // As coordenadas vem do quadrado; aqui voltam para o quadro real
        float L = ladoDoQuadradoDaPalma;
        Vector2 centroNoQuadro = new Vector2(
            0.5f + (d.center.x - 0.5f) * L / larguraDoQuadro,
            0.5f + (d.center.y - 0.5f) * L / alturaDoQuadro);

        // Mesmo criterio do rastreo: o eixo pulso -> base do dedo medio
        Vector2 pulsoQ = new Vector2(d.wrist.x * L, d.wrist.y * L);
        Vector2 medioQ = new Vector2(d.middle.x * L, d.middle.y * L);
        float palmaNoQuadrado = Vector2.Distance(pulsoQ, medioQ);

        if (palmaNoQuadrado > 1f)
        {
            Vector2 centroPx = pulsoQ + (medioQ - pulsoQ) * 0.9f;
            centroNoQuadro = new Vector2(
                0.5f + (centroPx.x / L - 0.5f) * L / larguraDoQuadro,
                0.5f + (centroPx.y / L - 0.5f) * L / alturaDoQuadro);
        }

        centroCaixa = centroNoQuadro;
        float ladoEmPixels = (palmaNoQuadrado > 1f)
            ? palmaNoQuadrado * 3.4f
            : Mathf.Max(d.extent.x * L, d.extent.y * L) * 2.6f;
        ladoCaixaPx = Mathf.Clamp(ladoEmPixels, 180f,
                                  Mathf.Min(larguraDoQuadro, alturaDoQuadro));

        // Quanto girar: a direcao do pulso ate a base do dedo medio diz a
        // inclinacao da mao, e o modelo espera receber a mao em pe
        Vector2 eixo = new Vector2((d.middle.x - d.wrist.x) * L,
                                   (d.middle.y - d.wrist.y) * L);
        if (eixo.sqrMagnitude > 0.0001f)
            anguloDaMao = 90f - Mathf.Atan2(eixo.y, eixo.x) * Mathf.Rad2Deg;
    }

    void CopiarComGiro(RenderTexture destino, Vector2 centro, Vector2 tamanhoEmPixels,
                      float grausExtra = 0f)
    {
        float rad = (GiroDaCamera + grausExtra) * Mathf.Deg2Rad;
        materialDeGiro.SetVector("_Giro", new Vector4(Mathf.Cos(rad), Mathf.Sin(rad), 0, 0));
        materialDeGiro.SetVector("_Centro", new Vector4(centro.x, centro.y, 0, 0));
        materialDeGiro.SetVector("_Tamanhos", new Vector4(
            tamanhoEmPixels.x, tamanhoEmPixels.y, minhaCamera.width, minhaCamera.height));
        materialDeGiro.SetVector("_Espelho", new Vector4(
            InverterHorizontal ? -1f : 1f, InverterVertical ? -1f : 1f, 0, 0));
        Graphics.Blit(minhaCamera, destino, materialDeGiro);
    }
}

#endif

// Aguarda câmera ter dimensões reais antes de criar o detector.
// WebCamTexture começa com 16x16 de placeholder - criar o detector nesse momento causaria crash.
IEnumerator InicializarDetetiveAposCamera()
{
    Debug.Log("Aguardando camera inicializar...");

    while (!minhaCamera.isPlaying || minhaCamera.width < 100)
        yield return null;

    yield return new WaitForEndOfFrame();
}

```

```

#if UNITY_ANDROID && !UNITY_EDITOR
    PrepararCorrecaoDeGiro();

    // Primeiro estagio: o detector de palma encontra a mao no quadro
    // inteiro e informa onde ela esta e para que lado esta virada
    if (ficheirosDaPalma != null)
    {
        try
        {
            detectorDePalma = new MediaPipe.BlazePalm.PalmDetector(ficheirosDaPalma);
            Debug.Log("Detector de palma pronto (pipeline de dois estagios).");
        }
        catch (System.Exception e)
        {
            Debug.LogWarning("Detector de palma indisponivel: " + e.Message);
        }
    }
#endif

    try
    {
        detetive = new HandLandmarkDetector(ficheirosDaIA);
        detetivePronto = true;
        Debug.Log("HandLandmarkDetector criado! Resolucao: " + minhaCamera.width + "x" + minhaCamera.height);
    }
    catch (System.Exception e)
    {
        Debug.LogError("FATAL: Falha ao criar HandLandmarkDetector.\n" + e.Message);
        enabled = false;
    }
}

void Update()
{
    // Guarda: nenhuma fonte de rastreamento pronta ainda
    if (!usandoExterno && !detetivePronto) return;

    // Input SEMPRE antes de qualquer return.
    // GetKeyDown só existe por 1 frame - se ficar atrás de um return, some para sempre.
    if (MODULO_TREINAMENTO)
    {
        bool shift = Input.GetKey(KeyCode.LeftShift) || Input.GetKey(KeyCode.RightShift);

        // Shift+Tecla = APAGA todas as amostras daquela letra (para regravar do zero)
        // Tecla sozinha = GRAVA nova amostra (pode pressionar várias vezes para acumular)
        // O laço cobre TODAS as letras de A a Z (KeyCode.A até KeyCode.Z são sequenciais)
        for (KeyCode tecla = KeyCode.A; tecla <= KeyCode.Z; tecla++)
        {
            if (!Input.GetKeyDown(tecla)) continue;

            string letra = tecla.ToString(); // "A", "B", ... "Z"
            if (shift)
                reconhecedor.ApagarLetra(letra);
            else if (reconhecedor.EhLetraDinamica(letra))
                StartCoroutine(GravarMovimentoComAtraso(letra)); // grava o FILME
            else
                StartCoroutine(GravarComAtraso(letra)); // grava a FOTO
        }

        // Ç não existe no enum de teclas do Unity - no teclado brasileiro
        // (ABNT) a tecla Ç chega como "Semicolon" (posição do ; americano)
        if (Input.GetKeyDown(KeyCode.Semicolon))
        {
            if (shift)
                reconhecedor.ApagarLetra("Ç");
            else if (reconhecedor.EhLetraDinamica("Ç"))
                StartCoroutine(GravarMovimentoComAtraso("Ç"));
            else
                StartCoroutine(GravarComAtraso("Ç"));
        }
    }

    // Space = pula a PALAVRA (custa pontos)
    // Tab = pula a LETRA atual (custa pontos)
    // Backspace = volta à letra anterior
    if (Input.GetKeyDown(KeyCode.Space) && gerenciador != null) gerenciador.PularPalavra();
    if (Input.GetKeyDown(KeyCode.Tab) && gerenciador != null) gerenciador.PularLetra();
    if (Input.GetKeyDown(KeyCode.Backspace) && gerenciador != null) gerenciador.VoltarLetra();

    // Supercontrole do apresentador: setas trocam de palavra SEM custo
    // (só existe no teclado, então não afeta o jogador de celular)
    if (Input.GetKeyDown(KeyCode.RightArrow) && gerenciador != null) gerenciador.AvancarPalavraGratis();
    if (Input.GetKeyDown(KeyCode.LeftArrow) && gerenciador != null) gerenciador.VoltarPalavraGratis();

    // Mantém a imagem sem distorção (recorte central estilo "cover")
    AjustarRecorteDaCamera();
}

#if UNITY_ANDROID && !UNITY_EDITOR
    AtualizarPainelDeMedicao();
#endif

if (usandoExterno)

```

```

{
    // Video vem do Python via UDP (o Python é o dono da webcam)
    if (rastreadorExterno.Video != null && fundoDoEcra.texture != rastreadorExterno.Video)
        fundoDoEcra.texture = rastreadorExterno.Video;
    cameraPronta = rastreadorExterno.Video != null;
}
else if (minhaCamera.isPlaying && minhaCamera.didUpdateThisFrame)
{
    // Processa novo frame da câmera (apenas quando houver frame novo - performance)
    ProcessarFrame();
    cameraPronta = true;
}

if (!cameraPronta) return;

// Detecção com histerese: precisa de confiança ALTA (0.6) para começar,
// mas só solta quando cair BEM (0.45). Isso elimina o pisca-pisca do
// esqueleto quando a confiança fica oscilando perto do limite.
float confianca = usandoExterno ? rastreadorExterno.Score : detetive.Score;
#if UNITY_ANDROID && !UNITY_EDITOR
    if (MediaPipeAtivo)
    {
        mediapipe.Atualizar();
        confianca = mediapipe.Score;
    }
    // Mais tolerancia para MANTER a mao ja encontrada: sinais de punho
    // fechado abaixam a confianca e o rastreo se perdia neles
    MaoDetectada = MaoDetectada ? (confianca >= 0.4f) : (confianca >= 0.6f);
#else
    MaoDetectada = MaoDetectada ? (confianca >= 0.45f) : (confianca >= 0.6f);
#endif

    if (!MaoDetectada)
    {
        // Mão sumiu: limpa a memória de movimento (evita "emendar"
        // dois pedaços de gesto que não têm nada a ver)
        janelaMovimento.Clear();
        temposJanela.Clear();

        // Perdeu a mão (rastreador interno): depois de ~15 frames volta à
        // "busca ampla" (recorte grande no centro) para reencontrá-la
        if (!usandoExterno)
        {
            framesSemMao++;
        }
        #if UNITY_ANDROID && !UNITY_EDITOR
            // Com o detector de palma trabalhando, vale insistir na ultima
            // posicao conhecida antes de varrer o quadro inteiro de novo
            int limiteSemMao = (detectorDePalma != null) ? 90 : 15;
        #else
            const int limiteSemMao = 15;
        #endif

        if (framesSemMao > limiteSemMao)
        {
            centroCaixa = new Vector2(0.5f, 0.5f);
            ladoCaixaPx = 0f;
            anguloDaMao = 0f;
        }
        return;
    }
    framesSemMao = 0;

    Vector3[] pontosDaMao =
        MediaPipeAtivo ? (Vector3[])mediapipe.Pontos.Clone()
        : usandoExterno ? (Vector3[])rastreadorExterno.Pontos.Clone()
        : ColetarPontosDaMao();
    PontosDaMaoAtuais = pontosDaMao;

    #if UNITY_ANDROID && !UNITY_EDITOR
        // Informa ao reconhecedor em que formato os pontos chegam
        if (reconhecedor != null)
        {
            reconhecedor.aspectoDaCamera = (alturaDoQuadro > 0f)
                ? larguraDoQuadro / alturaDoQuadro
                : (float)minhaCamera.width / minhaCamera.height;
            reconhecedor.temProfundidade = true; // a profundidade agora é lida do modelo
        }
    #endif

    // Alimenta a memória de movimento (~15 quadros por segundo)
    AlimentarJanelaDeMovimento(pontosDaMao);

    // Reposiciona o recorte para o próximo frame seguir a mão (só no interno;
    // o MediaPipe externo já faz o próprio recorte por dentro)
    if (zoomInteligente && !usandoExterno && !MediaPipeAtivo)
        AtualizarCaixaDaMao(pontosDaMao, confianca);

    // Índice 8 = Index4 = ponta do dedo indicador
    Vector3 posicaoDaIA = pontosDaMao[8];
    float xNoEcra = ((posicaoDaIA.x - 0.5f) * escalaX) + translaX;
    float yNoEcra = ((posicaoDaIA.y - 0.5f) * escalaY) + translaY;

```

```

if (bolinhaDoDedo != null)
    bolinhaDoDedo.localPosition = new Vector3(xNoEcra, yNoEcra, 4f);

// Modo Jogo: reconhece letra e envia para o GerenciadorDeJogo
// So classifica com uma partida em andamento: no menu e na tela de
// estudo isso seria trabalho jogado fora a cada quadro
if (!MODULO_TREINAMENTO && reconhecedor != null &&
    gerenciador != null && gerenciador.JogoIniciado)
{
    // Cooldown: ignora reconhecimentos muito rápidos (evita registrar a mesma letra várias vezes)
    if (Time.time - tempoUltimoReconhecimento < cooldownReconhecimento)
    {
        LimparCandidata();
        return;
    }

    // O sistema sabe qual letra o jogador precisa fazer AGORA.
    // Se ela é DINÂMICA (H J K W X Z Ç), só o comparador de MOVIMENTO
    // trabalha; se é estática, só o de "foto" - economiza processamento
    // e um modo nunca atrapalha o outro.
    string letraEsperada = gerenciador.LetraEsperada;
    EsperandoMovimento = letraEsperada.Length > 0 &&
        reconhecedor.EhLetraDinamica(letraEsperada);

    if (EsperandoMovimento)
    {
        letraCandidata = "";
        LetraCandidata = "";
        ProgressoCandidata = 0f;

        if (Time.time - tempoUltimaChecagemMovimento >= intervaloChecagemMovimento)
        {
            tempoUltimaChecagemMovimento = Time.time;
            string letraMovimento = reconhecedor.ClassificarSinalDinamico(janelaMovimento);
            if (letraMovimento == letraEsperada && gerenciador.TentarLetra(letraMovimento))
            {
                // (sem aprendizado automático aqui: letra de MOVIMENTO não
                // deve virar amostra estática - poluiria o banco de fotos)
                tempoUltimoReconhecimento = Time.time;
                janelaMovimento.Clear();
                temposJanela.Clear();
            }
        }
        return;
    }

    string letraFeita = reconhecedor.ClassificarLetra(pontosDaMao);

    if (letraFeita == "Desconhecido" || letraFeita == "Nenhuma")
    {
        LimparCandidata();
        return;
    }

    // Filtro de estabilidade: a MESMA letra precisa ser vista por
    // alguns instantes seguidos antes de valer. Elimina falsos
    // positivos enquanto a mão troca de um sinal para outro.
    if (letraFeita != letraCandidata)
    {
        letraCandidata = letraFeita;
        tempoLetraCandidata = Time.time;
    }

    // Sinal MUITO parecido com o gravado? Aceita na metade do tempo.
    float tempoNecessario = (reconhecedor.UltimaDistancia < reconhecedor.distanciaDeCerteza)
        ? tempoEstabilidade * 0.5f
        : tempoEstabilidade;

    // A interface lê isto para mostrar a barrinha de progresso do sinal
    LetraCandidata = letraCandidata;
    ProgressoCandidata = Mathf.Clamp01((Time.time - tempoLetraCandidata) / tempoNecessario);

    if (ProgressoCandidata < 1f) return;

    bool acertou = gerenciador.TentarLetra(letraFeita);
    if (acertou)
    {
        tempoUltimoReconhecimento = Time.time;
        LimparCandidata();
    }
}
else
{
    LimparCandidata();
}
}

void LimparCandidata()
{
    letraCandidata = "";
    LetraCandidata = "";
    ProgressoCandidata = 0f;
}

```

```

    EsperandoMovimento = false;
}

// Ajusta o pedaço visível do feed (uvRect) para preencher a tela SEM esticar:
// recorta o centro da imagem, como os apps de câmera de celular fazem.
void AjustarRecorteDaCamera()
{
    // Dimensões da fonte de vídeo atual (webcam interna OU vídeo do Python)
    float larguraFonte, alturaFonte;
    if (usandoExterno)
    {
        if (rastreadorExterno.Video == null) return;
        larguraFonte = rastreadorExterno.Video.width;
        alturaFonte = rastreadorExterno.Video.height;
    }
    else
    {
        if (minhaCamera == null || minhaCamera.width < 100) return;
#if UNITY_ANDROID && !UNITY_EDITOR
        larguraFonte = (larguraDoQuadro > 0f) ? larguraDoQuadro : minhaCamera.width;
        alturaFonte = (alturaDoQuadro > 0f) ? alturaDoQuadro : minhaCamera.height;
#else
        larguraFonte = minhaCamera.width;
        alturaFonte = minhaCamera.height;
#endif
    }

    if (Screen.width == larguraTelaAnterior && Screen.height == alturaTelaAnterior &&
        zoomDaTela == zoomAnterior) return;
    larguraTelaAnterior = Screen.width;
    alturaTelaAnterior = Screen.height;
    zoomAnterior = zoomDaTela;

    float aspectoTela = (float)Screen.width / Screen.height;
    float aspectoCamera = larguraFonte / alturaFonte;

    // Fração do quadro visível em cada eixo (recorte "cover", sem esticar)
    float w, h;
    if (aspectoCamera > aspectoTela)
    {
        w = aspectoTela / aspectoCamera; // corta as laterais
        h = 1f;
    }
    else
    {
        w = 1f;
        h = aspectoCamera / aspectoTela; // corta topo e base
    }

    // Aplica a margem de segurança (mostra menos -> sobra folga nas bordas)
    w /= zoomDaTela;
    h /= zoomDaTela;

    uvRecorte = new Rect((1f - w) * 0.5f, (1f - h) * 0.5f, w, h);
    fundoDoEcra.uvRect = uvRecorte;
}

// Prepara a imagem e envia para a IA.
// Com zoom inteligente: recorta um quadrado ao redor da mão (Graphics.Blit)
// e envia só ele - a mão ocupa a imagem inteira e o modelo enxerga MUITO melhor.
void ProcessarFrame()
{
    if (!zoomInteligente)
    {
        escalaBlit = Vector2.one;
        offsetBlit = Vector2.zero;
        detetive.ProcessImage(minhaCamera);
        return;
    }

    if (texturaRecorte == null)
        texturaRecorte = new RenderTexture(320, 320, 0);

#if UNITY_ANDROID && !UNITY_EDITOR
    if (materialDeGiro != null && texturaExibicao != null)
        CopiarComGiro(texturaExibicao, ParaFonte(new Vector2(0.5f, 0.5f)),
            new Vector2(larguraDoQuadro, alturaDoQuadro));

    // Sem mão rastreada, procura a palma a cada quadro; com a mão em mãos,
    // uma conferida por segundo basta e o custo fica baixo
    // Com o MediaPipe nativo o trabalho acaba aqui: ele recebe a imagem
    // inteira e cuida sozinho de achar a palma, recortar e endireitar.
    if (MediaPipeAtivo)
    {
        mediapipe.Envia(texturaExibicao);
        escalaBlit = Vector2.one;
        offsetBlit = Vector2.zero;
        return;
    }

    if (detetiveDePalma != null &&
        (!MaoDetectada || Time.time >= tempoDaProximaBuscaDePalma))

```

```

{
    ProcurarPalma();
    tempoDaProximaBuscaDePalma = Time.time + 1f;
}
float w = larguraDoQuadro > 0f ? larguraDoQuadro : minhaCamera.width;
float h = alturaDoQuadro > 0f ? alturaDoQuadro : minhaCamera.height;
#else
float w = minhaCamera.width;
float h = minhaCamera.height;
#endif

// Sem mão rastreada: quadrado grande no centro (área visível da tela).
// Com mão rastreada: quadrado do tamanho da mão × margem.
float lado = (ladoCaixaPx > 0f) ? ladoCaixaPx : Mathf.Min(w, h);
lado = Mathf.Clamp(lado, 160f, Mathf.Min(w, h));

// Converte centro+lado do recorte em escala/deslocamento normalizados.
// O recorte pode "vazar" até 30% para fora do quadro: a borda é repetida
// (wrap Clamp), mas a mão continua no CENTRO do que a IA analisa -
// muito melhor do que empurrar o recorte e descentralizar a mão.
const float folga = 0.3f;
Vector2 escala = new Vector2(lado / w, lado / h);
Vector2 offset = new Vector2(centroCaixa.x - escala.x * 0.5f,
                             centroCaixa.y - escala.y * 0.5f);
offset.x = Mathf.Clamp(offset.x, -escala.x * folga, 1f - escala.x * (1f - folga));
offset.y = Mathf.Clamp(offset.y, -escala.y * folga, 1f - escala.y * (1f - folga));

#if UNITY_ANDROID && !UNITY_EDITOR
    if (materialDeGiro != null)
        CopiarComGiro(texturaRecorte,
                       ParaFonte(new Vector2(offset.x + escala.x * 0.5f,
                                               offset.y + escala.y * 0.5f)),
                       new Vector2(lado, lado), alinharMao ? anguloDaMao : 0f);
    else
        Graphics.Blit(minhaCamera, texturaRecorte, escala, offset);
#else
Graphics.Blit(minhaCamera, texturaRecorte, escala, offset);
#endif
detetive.ProcessImage(texturaRecorte);

escalaBlit = escala;
offsetBlit = offset;
}

// Recalcula a caixa de recorte a partir da mão detectada (com suavização,
// para o recorte deslizar atrás da mão em vez de pular)
void AtualizarCaixaDaMao(Vector3[] pontos, float confianca)
{
    float minX = 1f, maxX = 0f, minY = 1f, maxY = 0f;
    for (int i = 0; i < 21; i++)
    {
        if (pontos[i].x < minX) minX = pontos[i].x;
        if (pontos[i].x > maxX) maxX = pontos[i].x;
        if (pontos[i].y < minY) minY = pontos[i].y;
        if (pontos[i].y > maxY) maxY = pontos[i].y;
    }
}

#if UNITY_ANDROID && !UNITY_EDITOR
    if (alinharMao)
    {
        // A palma vai do pulso ate a base do dedo medio. Ela mantem o
        // mesmo tamanho com a mao aberta ou fechada, por isso guia tanto
        // o giro quanto o enquadramento.
        Vector2 pulsoPx = new Vector2(pontos[0].x * larguraDoQuadro,
                                       pontos[0].y * alturaDoQuadro);
        Vector2 baseDoMedioPx = new Vector2(pontos[9].x * larguraDoQuadro,
                                             pontos[9].y * alturaDoQuadro);
        Vector2 direcao = baseDoMedioPx - pulsoPx;
        float palmaPx = direcao.magnitude;

        if (palmaPx > 1f)
        {
            float desejado = 90f - Mathf.Atan2(direcao.y, direcao.x) * Mathf.Rad2Deg;
            anguloDaMao = Mathf.LerpAngle(anguloDaMao, desejado, 0.35f);

            // Enquadra pela palma, um pouco a frente dela para os dedos
            // caberem, exatamente como o MediaPipe faz
            Vector2 centroPx = pulsoPx + direcao * 0.9f;
            Vector2 centroDaPalma = new Vector2(centroPx.x / larguraDoQuadro,
                                                  centroPx.y / alturaDoQuadro);
            float ladoDesejado = Mathf.Clamp(palmaPx * 3.4f, 180f,
                                              Mathf.Min(larguraDoQuadro, alturaDoQuadro));

            centroCaixa = Vector2.Lerp(centroCaixa, centroDaPalma, 0.5f);
            ladoCaixaPx = (ladoCaixaPx <= 0f) ? ladoDesejado
                : Mathf.Lerp(ladoCaixaPx, ladoDesejado, 0.3f);
            return; // enquadramento ja definido pela palma
        }
    }
}

#endif

Vector2 centro = new Vector2((minX + maxX) * 0.5f, (minY + maxY) * 0.5f);

```



```

float larguraPx = (maxX - minX) * minhaCamera.width;
float alturaPx = (maxY - minY) * minhaCamera.height;

// Confiança caindo (mão rápida ou perto da borda)? Abre o recorte
// para dar mais contexto ao modelo ANTES de perder o rastreo.
float margem = margemDoZoom * ((confianca < 0.6f) ? 1.5f : 1f);
float alvoLado = Mathf.Max(larguraPx, alturaPx) * margem;

centroCaixa = Vector2.Lerp(centroCaixa, centro, 0.35f);
ladoCaixaPx = (ladoCaixaPx <= 0f) ? alvoLado : Mathf.Lerp(ladoCaixaPx, alvoLado, 0.25f);
}

// Converte os 21 landmarks do enum para Vector3[].
// HandLandmarkDetector.GetKeyPoint() retorna Vector2 (x,y normalizados 0-1)
// DENTRO DO RECORTE - aqui mapeamos de volta para o frame inteiro, assim
// todo o resto do jogo (esqueleto, hover, reconhecimento) continua igual.
// Definimos z=0 pois o reconhecimento de LIBRAS usa apenas posição 2D.
Vector3[] ColetarPontosDaMao()
{
    Vector3[] pontos = new Vector3[21];
    var valores =
    (HandLandmarkDetector.KeyPoint[])System.Enum.GetValues(typeof(HandLandmarkDetector.KeyPoint));
    for (int i = 0; i < valores.Length; i++)
    {
        Vector2 v2 = detetive.GetKeyPoint(valores[i]);
    #if UNITY_ANDROID && !UNITY_EDITOR
        // Profundidade: o modelo entrega quatro componentes por ponto e o
        // atalho de leitura devolvia apenas duas, descartando o eixo Z.
        // Lido aqui, ele fica na mesma escala do recorte.
        float z = detetive.VertexArray[i].z * escalaBlit.x;

        // O recorte pode ter girado para alinhar a mao: desfaz aqui
        float cx = v2.x - 0.5f, cy = v2.y - 0.5f;
        if (alinharMao && Mathf.Abs(anguloDaMao) > 0.01f)
        {
            float r = anguloDaMao * Mathf.Deg2Rad;
            float co = Mathf.Cos(r), se = Mathf.Sin(r);
            float rx = cx * co + cy * se;
            float ry = -cx * se + cy * co;
            cx = rx; cy = ry;
        }
        pontos[i] = new Vector3(offsetBlit.x + escalaBlit.x * (0.5f + cx),
                                offsetBlit.y + escalaBlit.y * (0.5f + cy), z);
    #else
        pontos[i] = new Vector3(offsetBlit.x + v2.x * escalaBlit.x,
                                offsetBlit.y + v2.y * escalaBlit.y, 0f);
    #endif
    }
    return pontos;
}

// Adiciona o quadro atual à "memória de movimento" (a ~15 quadros/s,
// independente do FPS do jogo) e descarta o que passou da janela
void AlimentarJanelaDeMovimento(Vector3[] pontos)
{
    if (Time.time - tempoUltimaAmostraJanela < 1f / 15f) return;
    tempoUltimaAmostraJanela = Time.time;

    janelaMovimento.Add((Vector3[])pontos.Clone());
    temposJanela.Add(Time.time);

    while (temposJanela.Count > 0 && temposJanela[0] < Time.time - duracaoJanelaMovimento)
    {
        temposJanela.RemoveAt(0);
        janelaMovimento.RemoveAt(0);
    }
}

// Treinamento de letra DINÂMICA: conta 3s e grava um "filme" de ~1,3s
IEnumerator GravarMovimentoComAtraso(string letra)
{
    Debug.Log("Letra com MOVIMENTO [" + letra + "]! Prepare-se... gravando em 3 segundos!");
    yield return new WaitForSeconds(3f);

    Debug.Log(">>> FACA O MOVIMENTO DE [" + letra + "] AGORA! <<<");
    var quadros = new List<Vector3[]>();
    float fim = Time.time + duracaoGravacaoMovimento;
    float proximaAmostra = 0f;

    while (Time.time < fim)
    {
        if (MaoDetectada && PontosDaMaoAtuais != null && Time.time >= proximaAmostra)
        {
            quadros.Add((Vector3[])PontosDaMaoAtuais.Clone());
            proximaAmostra = Time.time + 1f / 15f; // ~15 quadros/s
        }
        yield return null;
    }

    if (quadros.Count < 8)
    {
        Debug.LogError("Poucos quadros com a mao visivel (" + quadros.Count +

```

```

        "). Mantenha a mao no quadro e tente de novo.");
        yield break;
    }
    reconhecedor.GravarSinalDinamico(letra, quadros);
}

IEnumerator GravarComAtraso(string letra)
{
    Debug.Log("Prepare a letra [" + letra + "]! Gravando em 3 segundos...");
    yield return new WaitForSeconds(3f);

    float confianca = usandoExterno
        ? rastreadorExterno.Score
        : (detetivePronto ? detetive.Score : 0f);

    if (confianca < 0.5f)
    {
        Debug.LogError("Falhou! A IA nao detectou sua mao. Tente novamente.");
        yield break;
    }

    Vector3[] pontosAgora = usandoExterno
        ? (Vector3[])rastreadorExterno.Pontos.Clone()
        : ColetarPontosDaMao();
    reconhecedor.GravarLetra(letra, pontosAgora);
}

void OnDestroy()
{
    if (mediapipe != null) mediapipe.Encerrar();

    EncerrarProcessoDoRastreador(); // fecha o Python junto com o jogo
    detetive?.Dispose();
#ifdef UNITY_ANDROID && !UNITY_EDITOR
    detectorDePalma?.Dispose();
    if (texturaDaPalma != null) texturaDaPalma.Release();
#endif
    if (texturaRecorte != null) texturaRecorte.Release();
    if (texturaExibicao != null) texturaExibicao.Release();
    if (minhaCamera != null && minhaCamera.isPlaying)
        minhaCamera.Stop();
}
}

```

Assets/Editor/ConfigurarIcones.cs (80 linhas)

```
using UnityEditor;
using UnityEditor.Build;
using UnityEngine;

// ConfigurarIcones: aplica a marca do Ditado Estrelado como icone do aplicativo,
// tanto no Android quanto no executavel de computador.
//
// Pelo editor: menu Ditado Estrelado > Aplicar icones
// Tambem roda sozinho antes de cada APK, para o icone nunca ficar para tras.
//
// O aparelho recorta o icone no formato que usa - circulo, quadrado
// arredondado, gota. Por isso a marca fica bem dentro do quadrado, com folga
// nas bordas: o que estiver na beirada pode ser cortado.
public static class ConfigurarIcones
{
    const string PASTA = "Assets/Icones/";
    const string COMUM = PASTA + "icone.png";

    [MenuItem("Ditado Estrelado/Aplicar icones")]
    public static void Aplicar()
    {
        Texture2D comum = Preparar(COMUM);

        if (comum == null)
        {
            Debug.LogWarning("Icone nao encontrado em " + COMUM + " - nada a aplicar.");
            return;
        }

        // Android. As camadas do icone "adaptativo" so existem numa biblioteca
        // separada, que scripts de editor comuns nao enxergam. O icone classico
        // resolve: os aparelhos modernos recortam sozinhos a forma que usam
        // (circulo, quadrado arredondado), e por isso a marca foi desenhada
        // bem dentro do quadrado, longe das bordas que podem ser cortadas.
        Definir(NamedBuildTarget.Android, comum, IconKind.Application);

        // Executavel de computador (Windows, Mac, Linux)
        Definir(NamedBuildTarget.Standalone, comum, IconKind.Any);

        AssetDatabase.SaveAssets();
        Debug.Log("Icones aplicados ao Android e ao executavel de computador.");
    }

    // O Unity pede uma imagem para CADA tamanho que o sistema usa. A mesma
    // arte serve para todos: ele reduz cada uma na hora de compilar.
    static void Definir(NamedBuildTarget alvo, Texture2D arte, IconKind tipo)
    {
        int[] tamanhos = PlayerSettings.GetIconSizes(alvo, tipo);
        if (tamanhos == null || tamanhos.Length == 0) return;

        var lista = new Texture2D[tamanhos.Length];
        for (int i = 0; i < lista.Length; i++) lista[i] = arte;
        PlayerSettings.SetIcons(alvo, lista, tipo);
    }

    // Garante que a imagem esteja legivel e sem compressao: icone borrado
    // costuma ser so a compressao de textura aparecendo no lugar errado.
    static Texture2D Preparar(string caminho)
    {
        var importador = AssetImporter.GetAtPath(caminho) as TextureImporter;
        if (importador == null) return null;

        bool mudou = false;
        if (importador.textureType != TextureImporterType.Default)
        { importador.textureType = TextureImporterType.Default; mudou = true; }
        if (importador.textureCompression != TextureImporterCompression.Uncompressed)
        { importador.textureCompression = TextureImporterCompression.Uncompressed; mudou = true; }
        if (!importador.isReadable) { importador.isReadable = true; mudou = true; }
        if (importador.mipmapEnabled) { importador.mipmapEnabled = false; mudou = true; }
        if (importador.maxTextureSize < 1024) { importador.maxTextureSize = 1024; mudou = true; }

        if (mudou)
        {
            importador.SaveAndReimport();
            AssetDatabase.ImportAsset(caminho, ImportAssetOptions.ForceUpdate);
        }
        return AssetDatabase.LoadAssetAtPath<Texture2D>(caminho);
    }
}
```

Assets/Editor/ConstrutorAndroid.cs (52 linhas)

```
using UnityEditor;
using UnityEditor.Build.Reporting;
using UnityEngine;

// ConstrutorAndroid: gera o APK sem precisar abrir as janelas do editor.
//
// Pelo editor: menu Ditado Estrelado > Gerar APK para Android
// Pela linha de comando:
//   Unity.exe -quit -batchmode -projectPath <pasta> -executeMethod ConstrutorAndroid.Construir
//
// O arquivo sai em Builds/DitadoEstrelado.apk
public static class ConstrutorAndroid
{
    const string DESTINO = "Builds/DitadoEstrelado.apk";

    [MenuItem("Ditado Estrelado/Gerar APK para Android")]
    public static void Construir()
    {
        // O icone e aplicado aqui para nunca sair um APK com a marca antiga
        ConfigurarIcones.Aplicar();
        PrepararRecursos.Configurar();

        // Ajustes que precisam valer no momento da compilacao
        PlayerSettings.SetScriptingBackend(BuildTargetGroup.Android, ScriptingImplementation.IL2CPP);
        PlayerSettings.Android.targetArchitectures = AndroidArchitecture.ARM64;
        PlayerSettings.defaultInterfaceOrientation = UIOrientation.Portrait;
        PlayerSettings.allowedAutorotateToPortrait = true;
        PlayerSettings.allowedAutorotateToPortraitUpsideDown = false;
        PlayerSettings.allowedAutorotateToLandscapeLeft = true;
        PlayerSettings.allowedAutorotateToLandscapeRight = true;

        var opcoes = new BuildPlayerOptions
        {
            scenes = new[] { "Assets/Scenes/SampleScene.unity" },
            locationPathName = DESTINO,
            target = BuildTarget.Android,
            targetGroup = BuildTargetGroup.Android,
            options = BuildOptions.None
        };

        BuildReport relatorio = BuildPipeline.BuildPlayer(opcoes);
        var resumo = relatorio.summary;

        if (resumo.result == BuildResult.Succeeded)
            Debug.Log("APK gerado com sucesso: " + DESTINO +
                " (" + (resumo.totalSize / (1024 * 1024)) + " MB)");
        else
            Debug.LogError("A geracao do APK falhou: " + resumo.result +
                " erros: " + resumo.totalErrors);
    }
}
```

Assets/Editor/ConstrutorWindows.cs (86 linhas)

```
using System.IO;
using UnityEditor;
using UnityEditor.Build;
using UnityEditor.Build.Reporting;
using UnityEngine;

// ConstrutorWindows: gera o executavel para computador.
//
// Pelo editor: menu Ditado Estrelado > Gerar executavel para Windows
// Pela linha de comando:
//   Unity.exe -quit -batchmode -projectPath <pasta> -executeMethod ConstrutorWindows.Construir
//
// O resultado sai em Builds/Windows/, com o executavel e a pasta do rastreador
// em Python ao lado dele. O jogo procura essa pasta ao lado do proprio
// executavel, entao ela precisa viajar junto na hora de distribuir.
public static class ConstrutorWindows
{
    const string PASTA_SAIDA = "Builds/Windows";
    const string EXECUTAVEL = PASTA_SAIDA + "/DitadoEstrelado.exe";

    [MenuItem("Ditado Estrelado/Gerar executavel para Windows")]
    public static void Construir()
    {
        ConfigurarIcones.Aplicar();
        PrepararRecursos.Configurar();

        PlayerSettings.SetScriptingBackend(NamedBuildTarget.Standalone,
                                           ScriptingImplementation.Mono2x);
        PlayerSettings.fullScreenMode = FullScreenMode.Windowed;
        PlayerSettings.defaultScreenWidth = 1280;
        PlayerSettings.defaultScreenHeight = 720;
        PlayerSettings.resizableWindow = true;

        var opcoes = new BuildPlayerOptions
        {
            scenes = new[] { "Assets/Scenes/SampleScene.unity" },
            locationPathName = EXECUTAVEL,
            target = BuildTarget.StandaloneWindows64,
            targetGroup = BuildTargetGroup.Standalone,
            options = BuildOptions.None
        };

        BuildReport relatorio = BuildPipeline.BuildPlayer(opcoes);
        var resumo = relatorio.summary;

        if (resumo.result != BuildResult.Succeeded)
        {
            Debug.LogError("Falhou a geracao do executavel: " + resumo.result +
                           " (" + resumo.totalErrors + " erros)");
            EditorApplication.Exit(1);
            return;
        }

        CopiarRastreador();

        Debug.Log("Executavel gerado com sucesso: " + EXECUTAVEL +
                  " (" + (resumo.totalSize / (1024 * 1024)) + " MB)");
    }

    // O rastreador em Python precisa ficar ao lado do executavel: e ali que o
    // jogo o procura. Sem ele o jogo ainda abre, com o rastreador reserva
    // embarcado, porem com menos precisao.
    static void CopiarRastreador()
    {
        string origem = "RastreadorPython";
        string destino = Path.Combine(PASTA_SAIDA, "RastreadorPython");

        if (!Directory.Exists(origem))
        {
            Debug.LogWarning("Pasta " + origem + " nao encontrada - o executavel " +
                             "vai depender do rastreador reserva.");
            return;
        }

        Directory.CreateDirectory(destino);
        foreach (string caminho in Directory.GetFiles(origem))
        {
            string nome = Path.GetFileName(caminho);
            // As figuras da pasta servem a documentacao, nao ao jogo
            if (nome.StartsWith("fig_")) continue;
            File.Copy(caminho, Path.Combine(destino, nome), true);
        }
        Debug.Log("Rastreador em Python copiado para " + destino);
    }
}
```

Assets/Editor/PrepararRecursos.cs (32 linhas)

```
using UnityEditor;
using UnityEngine;

// PrepararRecursos: garante que as imagens usadas pelo menu estejam importadas
// como Sprite, para poderem ser carregadas por Resources.Load<Sprite> em tempo
// de execução. Roda automaticamente antes de cada build.
public static class PrepararRecursos
{
    const string SIMBOLO_LIBRAS = "Assets/Resources/simbolo_libras.png";

    public static void Configurar()
    {
        GarantirSprite(SIMBOLO_LIBRAS);
    }

    static void GarantirSprite(string caminho)
    {
        var imp = AssetImporter.GetAtPath(caminho) as TextureImporter;
        if (imp == null) { Debug.LogWarning("Recurso nao encontrado: " + caminho); return; }

        bool mudou = false;
        if (imp.textureType != TextureImporterType.Sprite)
        { imp.textureType = TextureImporterType.Sprite; mudou = true; }
        if (imp.spriteImportMode != SpriteImportMode.Single)
        { imp.spriteImportMode = SpriteImportMode.Single; mudou = true; }
        if (imp.alphaIsTransparency != true)
        { imp.alphaIsTransparency = true; mudou = true; }

        if (mudou) { imp.SaveAndReimport(); Debug.Log("Sprite configurado: " + caminho); }
    }
}
```

Assets/Flutuar.cs (48 linhas)

```
using UnityEngine;

// Flutuar: balanço suave, como se o objeto boiasse na água.
// Ele oscila para cima/baixo e para os lados, mas sempre em volta
// do ponto de origem - nunca "vai embora".
// O GerenciadorDeJogo adiciona este componente em toda fruta criada.
public class Flutuar : MonoBehaviour
{
    [Header("Tamanho do balanço (em unidades do mundo)")]
    public float amplitude = 0.25f;

    [Header("Velocidade do balanço")]
    public float velocidade = 1.1f;

    [Header("Gingado de rotacao (graus)")]
    public float anguloBalanço = 4f;

    private Vector3 origem; // ponto fixo em volta do qual flutua
    private Quaternion rotacaoBase; // rotação original da fruta
    private float fase; // cada fruta balança num ritmo próprio

    void Start()
    {
        origem = transform.position;
        rotacaoBase = transform.rotation;
        fase = Random.Range(0f, Mathf.PI * 2f);
    }

    void Update()
    {
        float t = Time.time * velocidade + fase;

        // Senos com frequências diferentes: o caminho vira um "oito" orgânico
        Vector3 deslocamento = new Vector3(
            Mathf.Sin(t * 0.63f) * amplitude,
            Mathf.Sin(t) * amplitude,
            0f);
        transform.position = origem + deslocamento;

        // Leve gingado de rotação, em volta da rotação original
        Quaternion gingado = Quaternion.Euler(
            Mathf.Sin(t * 0.9f) * anguloBalanço,
            Mathf.Sin(t * 0.7f) * anguloBalanço,
            0f);
        transform.rotation = rotacaoBase * gingado;
    }
}
```

Assets/GerenciadorDeAudio.cs (269 linhas)

```
using UnityEngine;

// GerenciadorDeAudio: todos os sons do jogo são GERADOS POR CÓDIGO (síntese de
// ondas senoidais) - não precisa de nenhum arquivo de áudio no projeto.
// Também não precisa colocar nada na cena: ele se cria sozinho na primeira
// vez que algum script chama GerenciadorDeAudio.TocarClique() etc.
public class GerenciadorDeAudio : MonoBehaviour
{
    static GerenciadorDeAudio instancia;
    const int TAXA = 44100; // amostras de áudio por segundo (qualidade CD)

    AudioSource fonteSfx; // efeitos sonoros
    AudioSource fonteMusica; // música de fundo

    AudioClip somClique;
    AudioClip somAcerto;
    AudioClip somVitoria;
    AudioClip somErro;
    AudioClip somVidaExtra;

    // Várias músicas tocando em rodízio (uma termina, entra a próxima)
    readonly System.Collections.Generic.List<AudioClip> musicas =
        new System.Collections.Generic.List<AudioClip>();
    AudioClip musicaVitoria;
    int musicaAtual = 0;
    int faixaFixa = -1; // >= 0: repete sempre a mesma faixa (música da fase)
    bool musicaligada = true;

    [Range(0f, 1f)] public float volumeSfx = 0.9f;
    [Range(0f, 1f)] public float volumeMusica = 0.3f;

    // — Acesso global (cria o objeto na primeira chamada) —————
    static GerenciadorDeAudio Obter()
    {
        if (instancia == null)
        {
            var go = new GameObject("GerenciadorDeAudio");
            instancia = go.AddComponent<GerenciadorDeAudio>();
        }
        return instancia;
    }

    void Awake()
    {
        instancia = this;

        fonteSfx = gameObject.AddComponent<AudioSource>();
        fonteSfx.playOnAwake = false;

        fonteMusica = gameObject.AddComponent<AudioSource>();
        fonteMusica.playOnAwake = false;
        fonteMusica.loop = false; // sem loop: quando acaba, o Update troca de faixa

        // Frequências em Hz: C5=523.25 E5=659.25 G5=783.99 C6=1046.5
        somClique = ClipDeNotas(new[] { 880f }, 0.07f, 0.05f);
        somAcerto = ClipDeNotas(new[] { 523.25f, 783.99f }, 0.09f, 0.18f);
        somVitoria = ClipDeNotas(new[] { 523.25f, 659.25f, 783.99f, 1046.5f }, 0.12f, 0.4f);
        somErro = ClipDeNotas(new[] { 196f, 130.81f }, 0.16f, 0.2f); // descendo = "ops!"
        somVidaExtra = ClipDeNotas(new[] { 659.25f, 783.99f, 1046.5f }, 0.08f, 0.3f); // subida rápida =
"ganhou!"

        GerarMusicas();
        fonteMusica.volume = volumeMusica;

        // Lembra a escolha do jogador entre sessões (1 = ligada)
        musicaligada = PlayerPrefs.GetInt("musicaligada", 1) == 1;
    }

    // Quando uma faixa termina: repete a faixa da fase (se fixada) ou rodízio
    void Update()
    {
        if (musicaligada && !fonteMusica.isPlaying && musicas.Count > 0)
        {
            musicaAtual = (faixaFixa >= 0 && faixaFixa < musicas.Count)
                ? faixaFixa
                : (musicaAtual + 1) % musicas.Count;
            fonteMusica.clip = musicas[musicaAtual];
            fonteMusica.Play();
        }
    }

    // — Métodos públicos (chame de qualquer script) —————
    public static void TocarClique() { var g = Obter(); g.fonteSfx.PlayOneShot(g.somClique, g.volumeSfx); }
    public static void TocarAcerto() { var g = Obter(); g.fonteSfx.PlayOneShot(g.somAcerto, g.volumeSfx); }
    public static void TocarVitoria() { var g = Obter(); g.fonteSfx.PlayOneShot(g.somVitoria, g.volumeSfx); }
    public static void TocarErro() { var g = Obter(); g.fonteSfx.PlayOneShot(g.somErro, g.volumeSfx); }
    public static void TocarVidaExtra() { var g = Obter(); g.fonteSfx.PlayOneShot(g.somVidaExtra, g.volumeSfx); }
```



```

public static bool MusicaLigada => Obter().musicalLigada;

// Liga/desliga a música de fundo (o botão de som chama isto)
public static void AlternarMusica()
{
    var g = Obter();
    g.musicalLigada = !g.musicalLigada;
    PlayerPrefs.SetInt("musicalLigada", g.musicalLigada ? 1 : 0);
    if (g.musicalLigada) g.fonteMusica.Play();
    else g.fonteMusica.Pause();
}

public static void TocarMusica()
{
    var g = Obter();
    if (g.musicalLigada && !g.fonteMusica.isPlaying && g.musicas.Count > 0)
    {
        g.fonteMusica.clip = g.musicas[g.musicaAtual];
        g.fonteMusica.Play();
    }
}

public static void PararMusica() { Obter().fonteMusica.Stop(); }

// Fixa uma faixa (0 = mais calma ... 4 = mais agitada) - usada pelas
// fases do jogo para acelerar a música conforme a dificuldade sobe.
// Passe -1 para voltar ao rodízio livre.
public static void TocarFaixa(int indice)
{
    var g = Obter();
    g.faixaFixa = indice;
    if (indice < 0 || indice >= g.musicas.Count) return;
    g.musicaAtual = indice;
    if (g.musicalLigada)
    {
        g.fonteMusica.clip = g.musicas[indice];
        g.fonteMusica.Play();
    }
}

// Fanfarra de vitória (toca uma vez; depois volta a faixa atual)
public static void TocarMusicaVitoria()
{
    var g = Obter();
    if (!g.musicalLigada) return;
    g.fonteMusica.clip = g.musicaVitoria;
    g.fonteMusica.Play();
}

// — Síntese de som —————

// Cria um clipe tocando as notas em sequência (uma após a outra)
AudioClip ClipDeNotas(float[] frequencias, float duracaoNota, float cauda)
{
    int totalAmostras = Mathf.CeilToInt((frequencias.Length * duracaoNota + cauda) * TAXA);
    float[] dados = new float[totalAmostras];

    for (int n = 0; n < frequencias.Length; n++)
        EscreverNota(dados, frequencias[n], n * duracaoNota, duracaoNota + cauda, 0.5f);

    return CriarClip("sfx", dados);
}

// Escreve uma nota no buffer: senoide com ataque rápido e decaimento suave
// (parecido com uma nota de xilofone/marimba)
void EscreverNota(float[] dados, float freq, float inicio, float duracao, float volume)
{
    int primeiraAmostra = Mathf.FloorToInt(inicio * TAXA);
    int totalAmostras = Mathf.FloorToInt(duracao * TAXA);

    for (int i = 0; i < totalAmostras && primeiraAmostra + i < dados.Length; i++)
    {
        float t = (float)i / TAXA;
        float ataque = Mathf.Clamp01(t / 0.008f); // evita "clique" no início
        float decaimento = Mathf.Exp(-4.5f * t / duracao); // volume cai suavemente
        float amostra = Mathf.Sin(2f * Mathf.PI * freq * t)
            + 0.3f * Mathf.Sin(4f * Mathf.PI * freq * t); // harmônico: timbre mais rico

        dados[primeiraAmostra + i] += amostra * ataque * decaimento * volume * 0.5f;
    }
}

AudioClip CriarClip(string nome, float[] dados)
{
    // Limita o volume para não estourar quando notas se sobrepõem
    for (int i = 0; i < dados.Length; i++)
        dados[i] = Mathf.Clamp(dados[i], -1f, 1f);

    var clip = AudioClip.Create(nome, dados.Length, 1, TAXA, false);
    clip.SetData(dados, 0);
    return clip;
}

```

```

// Gera as músicas de fundo. Cada número é a frequência da nota em Hz;
// 0 = pausa. Notas usadas (escala pentatônica, sempre soa bem):
// C4=261.63 D4=293.66 E4=329.63 G4=392 A4=440 C5=523.25 D5=587.33 E5=659.25
void GerarMusicas()
{
    // Faixa 1: calma, a original
    musicas.Add(RenderizarMusica(0.3f, new[]
    {
        523.25f, 0, 392f, 0, 440f, 0, 392f, 0,
        329.63f, 0, 392f, 0, 440f, 0, 0, 0,
        523.25f, 0, 587.33f, 0, 523.25f, 0, 440f, 0,
        392f, 0, 329.63f, 0, 293.66f, 0, 0, 0,
        329.63f, 392f, 440f, 0, 523.25f, 0, 440f, 0,
        392f, 0, 329.63f, 0, 261.63f, 0, 0, 0,
    },
    new[] { 130.81f, 98f, 110f, 98f, 130.81f, 98f }));

    // Faixa 2: mais animada (arpejos subindo, andamento mais rápido)
    musicas.Add(RenderizarMusica(0.24f, new[]
    {
        261.63f, 329.63f, 392f, 523.25f, 392f, 329.63f, 261.63f, 0,
        293.66f, 392f, 440f, 587.33f, 440f, 392f, 293.66f, 0,
        329.63f, 392f, 523.25f, 659.25f, 523.25f, 392f, 329.63f, 0,
        261.63f, 329.63f, 392f, 440f, 523.25f, 0, 523.25f, 0,
    },
    new[] { 130.81f, 110f, 98f, 130.81f }));

    // Faixa 3: bem suave, tipo canção de ninar (notas longas)
    musicas.Add(RenderizarMusica(0.4f, new[]
    {
        440f, 0, 0, 523.25f, 0, 0, 440f, 0,
        392f, 0, 0, 329.63f, 0, 0, 0, 0,
        440f, 0, 0, 587.33f, 0, 0, 523.25f, 0,
        440f, 0, 0, 392f, 0, 0, 0, 0,
    },
    new[] { 110f, 98f, 110f, 130.81f }));

    // Faixa 4: BEM animada! Andamento rápido, melodia saltitante
    musicas.Add(RenderizarMusica(0.17f, new[]
    {
        523.25f, 523.25f, 0, 659.25f, 0, 523.25f, 783.99f, 0,
        659.25f, 0, 523.25f, 0, 440f, 440f, 523.25f, 0,
        587.33f, 587.33f, 0, 659.25f, 0, 587.33f, 523.25f, 0,
        440f, 0, 392f, 0, 440f, 523.25f, 0, 0,
        659.25f, 659.25f, 0, 783.99f, 0, 659.25f, 1046.5f, 0,
        783.99f, 0, 659.25f, 0, 587.33f, 523.25f, 0, 0,
    },
    new[] { 130.81f, 130.81f, 98f, 110f, 130.81f, 98f }));

    // Faixa 5: festiva, com saltos de oitava (energia de festa junina!)
    musicas.Add(RenderizarMusica(0.2f, new[]
    {
        261.63f, 523.25f, 261.63f, 523.25f, 329.63f, 659.25f, 0, 0,
        293.66f, 587.33f, 293.66f, 587.33f, 392f, 783.99f, 0, 0,
        440f, 523.25f, 587.33f, 659.25f, 587.33f, 523.25f, 440f, 0,
        392f, 440f, 392f, 329.63f, 261.63f, 0, 261.63f, 0,
    },
    new[] { 130.81f, 110f, 98f, 130.81f }));

    // Fanfarra de vitória: escala subindo + acordes triunfais
    musicaVitoria = RenderizarMusica(0.16f, new[]
    {
        523.25f, 587.33f, 659.25f, 783.99f, 880f, 1046.5f, 0, 1046.5f,
        0, 1046.5f, 0, 0, 783.99f, 880f, 1046.5f, 0,
        1046.5f, 0, 880f, 0, 1046.5f, 0, 0, 0,
        523.25f, 659.25f, 783.99f, 1046.5f, 0, 1046.5f, 1046.5f, 0,
    },
    new[] { 130.81f, 98f, 130.81f, 130.81f }));
}

// Transforma um padrão de notas + baixo num clipe de áudio
AudioClip RenderizarMusica(float passo, float[] melodia, float[] baixo)
{
    int celulasPorBaixo = Mathf.Max(1, melodia.Length / baixo.Length);
    int totalAmostras = Mathf.CeilToInt(melodia.Length * passo * TAXA);
    float[] dados = new float[totalAmostras];

    for (int i = 0; i < melodia.Length; i++)
        if (melodia[i] > 0f)
            EscreverNota(dados, melodia[i], i * passo, passo * 3.2f, 0.22f);

    for (int b = 0; b < baixo.Length; b++)
        EscreverNota(dados, baixo[b], b * celulasPorBaixo * passo,
            passo * celulasPorBaixo * 0.9f, 0.16f);

    return CriarClip("musica", dados);
}
}

```

Assets/GerenciadorDeJogo.cs (672 linhas)

```
using UnityEngine;
using System.Collections;
using System.Collections.Generic;

// GerenciadorDeJogo: controla fases, palavras, pontos, vidas e tempo.
//
// FASES: 1-Frutas, 2-Animais, 3-Transporte, 4-Comidas (dificuldade crescente:
// menos tempo por letra a cada fase). Os modelos 3D são carregados pelo nome
// de dentro de Assets/Resources e redimensionados automaticamente.
//
// ECONOMIA: +10 por letra, +50 por palavra; pular letra custa 5, pular
// palavra custa 10. VIDAS: começa com 3, perde ao estourar o tempo,
// ganha 1 a cada 100 pontos (até o máximo).
public class GerenciadorDeJogo : MonoBehaviour
{
    // — Estrutura das fases —————
    [System.Serializable]
    public class ItemDePalavra
    {
        public string palavra;
        public string caminhoPrefab; // caminho dentro de Assets/Resources (sem extensão)
        public string caminhoTextura; // opcional: textura a aplicar no modelo
        public float escala = 0f; // 0 = tamanho automático (recomendado)
    }

    [System.Serializable]
    public class FaseDoJogo
    {
        public string nome;
        public float tempoPorPalavra = 20f; // segundos para soletrar cada palavra
        public List<ItemDePalavra> itens = new List<ItemDePalavra>();
    }

    [Header("Fases (vazio = usa as fases padrao do codigo)")]
    public List<FaseDoJogo> fases = new List<FaseDoJogo>();

    // Quando as fases padrão do código mudam, este número sobe e a cena
    // esquece a versão antiga que tinha memorizado
    const int VERSAO_DAS_FASES = 3;
    public int versaoDasFases = 0;

    // Música por fase: índice da faixa (0 = calma ... 4 = agitada)
    public int[] faixaMusicalPorFase = { 0, 1, 3, 4 };

    [Header("Ponto de exibicao do objeto 3D")]
    public Transform pontoDeExibicao;

    [Header("Aparencia do objeto 3D")]
    public float tamanhoDoObjeto = 4.2f; // tamanho no mundo (auto-ajuste mede e aplica)
    public Vector3 rotacaoObjeto = new Vector3(0f, 160f, 0f); // só gira no eixo Y

    [Header("Pontuacao e economia")]
    public int pontosPorLetra = 0; // letras nao dao pontos...
    public int bonusPorPalavra = 10; // ...a PALAVRA completa da 10!
    public int custoPularLetra = 5;
    public int custoPularPalavra = 10;

    [Header("Vidas")]
    public int vidasIniciais = 3;
    public int pontosPorVidaExtra = 100;
    public int maximoDeVidas = 5;

    // Modo sem pressao: desliga o tempo e as vidas (bom para iniciantes
    // e criancas que frustram facil). Controlado pelo painel de opcoes.
    public bool modoSemPressao = false;

    [Header("Tempo de celebracao (s)")]
    public float tempoCelebracao = 2.5f;

    // — Eventos —————
    public System.Action<string> OnPalavraCompleta;
    public System.Action<int> OnPontuacaoAtualizada;
    public System.Action<int> OnVidasAtualizadas;
    public System.Action<string> OnNovaFase;
    public System.Action<int> OnPontosGastos; // animação "-5"/"-10" na tela
    public System.Action OnLetraCorreta; // flash verde (feedback visual)
    public System.Action OnVidaPerdida; // flash vermelho
    public System.Action OnVidaGanha; // animação "+1 coracao" na tela
    public System.Action OnSemSaldo; // tremida vermelha na pontuação

    // — Estado interno —————
    private List<ItemDePalavra> listaEmbaralhada;
    private int indicePalavra = 0;
    private int indiceLetra = 0;
    private int pontuacao = 0;
    private int vidas = 0;
    private int faseAtual = 0;
    private int proximaVidaEm = 100;
```

```

private float tempoRestante = 0f;
private bool aguardandoCelebracao = false;
private bool fimDeJogo = false;
private GameObject objetoAtual;

// — Propriedades públicas —————
public string PalavraAtual =>
    (JogoIniciado && !fimDeJogo && listaEmbaralhada != null &&
     indicePalavra < listaEmbaralhada.Count)
    ? listaEmbaralhada[indicePalavra].palavra : "";

public int     IndiceLetraAtual => indiceLetra;
public int     Pontuacao        => pontuacao;
public int     Vidas            => vidas;
public float   TempoRestante   => tempoRestante;
public bool    Venceu           { get; private set; }
public string  NomeDaFase       =>
    (fases != null && faseAtual < fases.Count) ? fases[faseAtual].nome : "";

public bool JogoIniciado { get; private set; }
public bool JogoTerminado => JogoIniciado && !aguardandoCelebracao && fimDeJogo;

// Pausa (menu aberto no meio do jogo): congela tempo e reconhecimento
// SEM perder pontos, vidas nem a fase. O objeto 3D fica na frente da
// interface, então precisa ser escondido enquanto o menu está aberto.
public bool Pausado { get; private set; }

public void Pausar()
{
    if (!JogoIniciado || JogoTerminado) return;
    Pausado = true;
    if (objetoAtual != null) objetoAtual.SetActive(false);
}

public void Retomar()
{
    Pausado = false;
    if (objetoAtual != null) objetoAtual.SetActive(true);
}

// A letra que o jogador precisa fazer AGORA ("" fora do jogo).
// O ControladorCamera usa isto para saber se espera um sinal parado
// ou um MOVIMENTO (letras dinâmicas).
public string LetraEsperada
{
    get
    {
        if (Pausado) return ""; // pausado = reconhecimento dorme
        string p = PalavraAtual;
        return (p.Length > 0 && indiceLetra < p.Length) ? p[indiceLetra].ToString() : "";
    }
}

void Awake()
{
    if (fases == null || fases.Count == 0 || versaoDasFases != VERSAO_DAS_FASES)
    {
        fases = CriarFasesPadrao();
        versaoDasFases = VERSAO_DAS_FASES;
    }
}

// — Relógio da palavra —————
void Update()
{
    if (modoSemPressao) return; // sem tempo, sem vidas perdidas
    if (!JogoIniciado || Pausado || fimDeJogo || aguardandoCelebracao) return;
    if (string.IsNullOrEmpty(PalavraAtual)) return;

    tempoRestante -= Time.deltaTime;
    if (tempoRestante <= 0f)
    {
        tempoRestante = 0f;
        Debug.Log("Tempo esgotado! Perdeu uma vida.");
        PerderVida();
    }
}

// — Controle do jogo (chamado pelo MenuPrincipal) —————
public void IniciarJogo()
{
    if (fases == null || fases.Count == 0) fases = CriarFasesPadrao();

    faseAtual      = 0;
    pontuacao      = 0;
    vidas          = vidasIniciais;
    proximaVidaEm  = pontosPorVidaExtra;
    fimDeJogo      = false;
    venceu         = false;
    Pausado        = false;
    aguardandoCelebracao = false;
    JogoIniciado   = true;
}

```

```

        OnPontuacaoAtualizada?.Invoke(pontuacao);
        OnVidasAtualizadas?.Invoke(vidas);
        ComecarFase();
    }

    public void PararJogo()
    {
        StopAllCoroutines();
        aguardandoCelebracao = false;
        JogoIniciado = false;
        Pausado = false;
        if (objetoAtual != null) Destroy(objetoAtual);
    }

    void ComecarFase()
    {
        var fase = fases[faseAtual];
        listaEmbaralhada = new List<ItemDePalavra>(fase.itens);
        Embaralhar(listaEmbaralhada);
        indicePalavra = 0;
        OnNovaFase?.Invoke("FASE " + (faseAtual + 1));

        // Música fica mais agitada a cada fase
        int faixa = faixaMusicalPorFase[Mathf.Min(faseAtual, faixaMusicalPorFase.Length - 1)];
        GerenciadorDeAudio.TocarFaixa(faixa);

        ExibirItemAtual();
    }

    void Embaralhar(List<ItemDePalavra> lista)
    {
        for (int i = lista.Count - 1; i > 0; i--)
        {
            int j = Random.Range(0, i + 1);
            var tmp = lista[i]; lista[i] = lista[j]; lista[j] = tmp;
        }
    }

    void ExibirItemAtual()
    {
        if (objetoAtual != null) Destroy(objetoAtual);

        // Acabaram as palavras desta fase? Vai para a próxima
        if (indicePalavra >= listaEmbaralhada.Count)
        {
            AvancarFase();
            return;
        }

        var item = listaEmbaralhada[indicePalavra];
        var prefab = Resources.Load<GameObject>(item.caminhoPrefab);
        if (prefab == null)
        {
            Debug.LogError("Modelo nao encontrado em Resources: " + item.caminhoPrefab);
            indicePalavra++;
            ExibirItemAtual();
            return;
        }

        Vector3 pos = pontoDeExibicao != null ? pontoDeExibicao.position : Vector3.zero;

        // IMPORTANTE: multiplica pela rotação ORIGINAL do modelo - muitos FBX
        // trazem uma correção de eixo embutida (ignorá-la deixava alguns
        // modelos de cabeça para baixo!)
        Quaternion rotacao = Quaternion.Euler(rotacaoObjeto) * prefab.transform.rotation;
        objetoAtual = Instantiate(prefab, pos, rotacao);

        CorrigirMateriaisParaURP(objetoAtual); // sem materiais rosas/brancos
        PintarPorNomeDeMaterial(objetoAtual, prefab.name); // devolve as cores perdidas
        AplicarTexturaSeDefinida(objetoAtual, item.caminhoTextura);
        AjustarTamanhoECentro(objetoAtual, item.escala, pos);
        objetoAtual.AddComponent<Flutuar>(); // balanço suave, como se boiasse

        // Modelo animado toca a animação em loop; estático ganha "respiração"
        bool animou = ReproduzirAnimacao.Tocar(objetoAtual, item.caminhoPrefab);
        if (!animou) objetoAtual.AddComponent<RespirarSuave>();

        indiceLetra = 0;
        tempoRestante = fases[faseAtual].tempoPorPalavra; // reinicia a cada palavra
        Debug.Log("Palavra: [" + item.palavra + "] - " +
            Mathf.RoundToInt(tempoRestante) + "s para soletrar");
    }

    // Materiais dos pacotes vêm no shader antigo ("Standard"), que o URP
    // não renderiza (fica rosa/branco). Troca pelo URP Lit preservando a
    // cor e a textura que o material tinha.
    static void CorrigirMateriaisParaURP(GameObject go)
    {
        Shader urp = Shader.Find("Universal Render Pipeline/Lit");
        if (urp == null) return;
    }

```

```

foreach (var renderizador in go.GetComponentsInChildren<Renderer>())
    foreach (var material in renderizador.materials)
    {
        if (material == null || material.shader == null) continue;
        if (material.shader.name.Contains("Universal")) continue; // já é URP

        Color cor = material.HasProperty("_Color") ? material.color : Color.white;
        Texture tex = material.HasProperty("_MainTex") ? material.mainTexture : null;

        material.shader = urp;
        material.SetColor("_BaseColor", cor);
        if (tex != null) material.SetTexture("_BaseMap", tex);
    }
}

// O pacote de transporte foi exportado SEM as cores (todos os materiais
// vieram cinza 0.64) - mas os NOMES dos materiais revelam a cor que o
// artista queria ("Red", "Windows", "Wheel"...). Pintamos por nome.
// Chave "Modelo:Material" tem prioridade sobre a chave só do material.
static readonly Dictionary<string, Color> CORES_POR_NOME = new Dictionary<string, Color>
{
    // Genéricos (qualquer modelo)
    { "Red", new Color(0.85f, 0.15f, 0.15f) },
    { "White", new Color(0.95f, 0.95f, 0.95f) },
    { "Green", new Color(0.15f, 0.75f, 0.25f) },
    { "Yellow", new Color(1f, 0.80f, 0.10f) },
    { "Black", new Color(0.08f, 0.08f, 0.08f) },
    { "Grey", new Color(0.55f, 0.55f, 0.55f) },
    { "Lights", new Color(1f, 0.95f, 0.60f) }, // faróis
    { "Windows", new Color(0.55f, 0.80f, 0.95f) }, // vidro azulado
    { "Bumper", new Color(0.30f, 0.30f, 0.32f) },
    { "Wheel", new Color(0.12f, 0.12f, 0.12f) }, // pneu
    { "Details", new Color(0.40f, 0.40f, 0.42f) },
    { "Handle", new Color(0.15f, 0.15f, 0.15f) },

    // Específicos por modelo
    { "Bicycle:Bike", new Color(0.80f, 0.20f, 0.20f) }, // quadro vermelho
    { "Bicycle:Material.003", new Color(0.50f, 0.50f, 0.52f) },
    { "Bus:Top", new Color(0.95f, 0.95f, 0.95f) },
    { "Bus:Bottom", new Color(0.20f, 0.45f, 0.85f) }, // ônibus azul
    { "Bus:Material", new Color(0.35f, 0.35f, 0.38f) },
    { "Train:Outside", new Color(0.75f, 0.20f, 0.20f) }, // trem vermelho
    { "Train:Top", new Color(0.85f, 0.85f, 0.88f) },
    { "Ambulance:Material", new Color(0.95f, 0.95f, 0.95f) },
    { "TrafficLight:TrafficLight", new Color(0.20f, 0.20f, 0.22f) }, // poste
};

static void PintarPorNomeDeMaterial(GameObject go, string nomeModelo)
{
    foreach (var renderizador in go.GetComponentsInChildren<Renderer>())
        foreach (var material in renderizador.materials)
        {
            if (material == null) continue;
            string nome = material.name.Replace(" (Instance)", "").Trim();

            Color cor;
            if (CORES_POR_NOME.TryGetValue(nomeModelo + ":" + nome, out cor) ||
                CORES_POR_NOME.TryGetValue(nome, out cor))
            {
                material.color = cor;
                if (material.HasProperty("_BaseColor"))
                    material.SetColor("_BaseColor", cor);
            }
        }
}

// Alguns modelos (comidas) têm a textura num arquivo separado que o
// importador não conecta sozinho - aplicamos manualmente
static void AplicarTexturaSeDefinida(GameObject go, string caminhoTextura)
{
    if (string.IsNullOrEmpty(caminhoTextura)) return;

    var textura = Resources.Load<Texture2D>(caminhoTextura);
    if (textura == null)
    {
        Debug.LogWarning("Textura nao encontrada em Resources: " + caminhoTextura);
        return;
    }

    foreach (var renderizador in go.GetComponentsInChildren<Renderer>())
        foreach (var material in renderizador.materials)
        {
            material.SetTexture("_BaseMap", textura);
            material.mainTexture = textura;
            if (material.HasProperty("_BaseColor"))
                material.SetColor("_BaseColor", Color.white);
        }
}

// Auto-ajuste: mede o modelo (bounds) e o redimensiona para
// 'tamanhoDoObjeto'; depois centraliza - muitos modelos têm a origem
// nos "pés", o que os deixaria flutuando alto na tela

```

```

void AjustarTamanhoECentro(GameObject go, float escalaManual, Vector3 centroDesejado)
{
    var renderizadores = go.GetComponentsInChildren<Renderer>();

    if (escalaManual > 0f)
    {
        go.transform.localScale = Vector3.one * escalaManual;
    }
    else if (renderizadores.Length > 0)
    {
        Bounds b = renderizadores[0].bounds;
        for (int i = 1; i < renderizadores.Length; i++)
            b.Encapsulate(renderizadores[i].bounds);

        float maiorLado = Mathf.Max(b.size.x, Mathf.Max(b.size.y, b.size.z));
        if (maiorLado > 0.0001f)
            go.transform.localScale *= tamanhoDoObjeto / maiorLado;
    }

    if (renderizadores.Length > 0)
    {
        // Recalcula os bounds já na escala nova e centraliza de verdade
        Bounds b2 = renderizadores[0].bounds;
        for (int i = 1; i < renderizadores.Length; i++)
            b2.Encapsulate(renderizadores[i].bounds);
        go.transform.position += centroDesejado - b2.center;
    }
}

// — Tentativa de letra (chamado pelo ControladorCamera) —————
public bool TentarLetra(string letraFeita)
{
    if (!JogoIniciado || Pausado || fimDeJogo || aguardandoCelebracao) return false;
    string palavra = PalavraAtual;
    if (string.IsNullOrEmpty(palavra)) return false;

    if (letraFeita != palavra[indiceLetra].ToString()) return false;

    AdicionarPontos(pontosPorLetra);
    indiceLetra++;
    Debug.Log("ACERTO! [" + letraFeita + "] +" + pontosPorLetra + " pts");

    if (indiceLetra >= palavra.Length)
    {
        AdicionarPontos(bonusPorPalavra);
        aguardandoCelebracao = true;
        GerenciadorDeAudio.TocarVitoria();
        OnPalavraCompleta?.Invoke(palavra);
        StartCoroutine(AvancarAposCelebracao());
    }
    else
    {
        GerenciadorDeAudio.TocarAcerto();
    }
    OnLetraCorreta?.Invoke(); // flash verde na tela
    return true;
}

IEnumerator AvancarAposCelebracao()
{
    yield return new WaitForSeconds(tempoCelebracao);
    aguardandoCelebracao = false;
    indicePalavra++;
    ExibirItemAtual();
}

void AvancarFase()
{
    faseAtual++;
    if (faseAtual >= fases.Count)
    {
        // Completou TODAS as fases: vitória!
        Venceu = true;
        fimDeJogo = true;
        SalvarRecorde();
        if (objetoAtual != null) Destroy(objetoAtual);
        GerenciadorDeAudio.TocarMusicaVitoria(); // fanfarra completa!
        Debug.Log("VENCEU O JOGO! Pontuacao final: " + pontuacao);
        return;
    }
    GerenciadorDeAudio.TocarVitoria();
    ComecarFase();
}

void PerderVida()
{
    vidas--;
    OnVidasAtualizadas?.Invoke(vidas);
    OnVidaPerdida?.Invoke(); // flash vermelho na tela
    GerenciadorDeAudio.TocarErro();

    if (vidas <= 0)

```

```

        {
            fimDeJogo = true;
            SalvarRecorde();
            if (objetoAtual != null) Destroy(objetoAtual);
            Debug.Log("Fim de jogo (acabaram as vidas). Pontuacao: " + pontuacao);
            return;
        }

        // Perdeu a vida mas o jogo segue: troca a palavra
        indicePalavra++;
        ExibirItemAtual();
    }

    // — Ações do jogador (custam pontos!) —————

    // Pula a palavra inteira (custa pontos)
    public void PularPalavra()
    {
        if (!JogoIniciado || Pausado || fimDeJogo || aguardandoCelebracao) return;
        if (pontuacao < custoPularPalavra)
        {
            GerenciadorDeAudio.TocarErro();
            OnSemSaldo?.Invoke(); // feedback VISUAL também (público surdo!)
            Debug.Log("Pontos insuficientes para pular a palavra (" + custoPularPalavra + ")");
            return;
        }
        AdicionarPontos(-custoPularPalavra);
        OnPontosGastos?.Invoke(custoPularPalavra); // animação "-10" na tela
        GerenciadorDeAudio.TocarClique();
        StopAllCoroutines();
        aguardandoCelebracao = false;
        indicePalavra++;
        ExibirItemAtual();
    }

    // Avança para a próxima letra (custa pontos)
    public void PularLetra()
    {
        if (!JogoIniciado || Pausado || fimDeJogo || aguardandoCelebracao) return;
        if (string.IsNullOrEmpty(PalavraAtual)) return;
        if (pontuacao < custoPularLetra)
        {
            GerenciadorDeAudio.TocarErro();
            OnSemSaldo?.Invoke(); // feedback VISUAL também (público surdo!)
            Debug.Log("Pontos insuficientes para pular a letra (" + custoPularLetra + ")");
            return;
        }
        AdicionarPontos(-custoPularLetra);
        OnPontosGastos?.Invoke(custoPularLetra); // animação "-5" na tela
        GerenciadorDeAudio.TocarClique();

        if (indiceLetra < PalavraAtual.Length - 1)
        {
            indiceLetra++;
        }
        else
        {
            // Era a última letra: a palavra acaba SEM bônus nem celebração
            indicePalavra++;
            ExibirItemAtual();
        }
    }

    // — Supercontrole do apresentador (setas do teclado, SEM custo) —————
    // Útil em demonstrações: navega pelas palavras sem gastar pontos.

    public void AvancarPalavraGratis()
    {
        if (!JogoIniciado || Pausado || fimDeJogo || aguardandoCelebracao) return;
        GerenciadorDeAudio.TocarClique();
        indicePalavra++;
        ExibirItemAtual();
    }

    public void VoltarPalavraGratis()
    {
        if (!JogoIniciado || Pausado || fimDeJogo || aguardandoCelebracao) return;
        if (indicePalavra == 0) return; // já está na primeira palavra da fase
        GerenciadorDeAudio.TocarClique();
        indicePalavra--;
        ExibirItemAtual();
    }

    // Volta uma letra (de graça)
    public void VoltarLetra()
    {
        if (!JogoIniciado || Pausado || fimDeJogo || aguardandoCelebracao) return;
        if (indiceLetra > 0)
        {
            GerenciadorDeAudio.TocarClique();
            indiceLetra--;
        }
    }

```



```

}

// — Helpers —————
void AdicionarPontos(int valor)
{
    pontuacao = Mathf.Max(0, pontuacao + valor);
    OnPontuacaoAtualizada?.Invoke(pontuacao);

    // Vida extra a cada 'pontosPorVidaExtra' pontos alcançados
    while (pontuacao >= proximaVidaEm)
    {
        proximaVidaEm += pontosPorVidaExtra;
        if (vidas < maximoDeVidas)
        {
            vidas++;
            OnVidasAtualizadas?.Invoke(vidas);
            OnVidaGanha?.Invoke(); // animação "+1 coração"
            GerenciadorDeAudio.TocarVidaExtra();
            Debug.Log("VIDA EXTRA! Vidas: " + vidas);
        }
    }
}

// Guarda o recorde entre sessões (mostrado no menu)
void SalvarRecorde()
{
    int recorde = PlayerPrefs.GetInt("recorde", 0);
    if (pontuacao > recorde)
    {
        PlayerPrefs.SetInt("recorde", pontuacao);
        PlayerPrefs.Save(); // grava agora, sem depender do fechamento normal
        Debug.Log("NOVO RECORDE: " + pontuacao);
    }
}

// — As fases padrão do jogo —————
static List<FaseDoJogo> CriarFasesPadrao()
{
    ItemDePalavra Item(string palavra, string caminho, string textura = null) =>
        new ItemDePalavra { palavra = palavra, caminhoPrefab = caminho,
                             caminhoTextura = textura };

    const string FRUTAS      = "Low Poly Fruits/Prefabs/";
    const string FAZENDA     = "Farm Animals by @Quaternius/FBX/";
    const string SELVAGEM    = "Ultimate Animated Animals - July 2021/FBX/";
    // Transporte usa os OBJ: as CORES dos veículos vêm nos materiais .mtl
    // (a exportação FBX deste pacote perde as cores)
    const string TRANSPORTE  = "Public Transport Pack - Feb 2017/OBJ/";
    const string COMIDA      = "Junk Food Pack - Apr 2017/FBX/";
    const string TEXTURAS    = "Junk Food Pack - Apr 2017/Blender/Textures/";

    return new List<FaseDoJogo>
    {
        new FaseDoJogo
        {
            nome = "FRUTAS", tempoPorPalavra = 25f,
            itens = new List<ItemDePalavra>
            {
                Item("MAÇA",      FRUTAS + "apple"),
                Item("BANANA",    FRUTAS + "banana"),
                Item("ABACATE",   FRUTAS + "avocado"),
                Item("LIMAO",     FRUTAS + "lemon"),
                Item("PERA",      FRUTAS + "pear"),
                Item("PESSEGO",   FRUTAS + "peach"),
                Item("MELANCIA",  FRUTAS + "watermelon"),
                Item("MORANGO",   FRUTAS + "strawberry"),
                Item("AMENDOIM",  FRUTAS + "peanut"),
                Item("CEREJA",    FRUTAS + "cherries"),
            }
        },
        new FaseDoJogo
        {
            nome = "ANIMAIS", tempoPorPalavra = 20f,
            itens = new List<ItemDePalavra>
            {
                // Animados (pacote Ultimate): VACA, CAVALO e LHAMA trocados
                // pelas versões COM animação
                Item("VACA",      SELVAGEM + "Cow"),
                Item("CAVALO",    SELVAGEM + "Horse"),
                Item("LHAMA",     SELVAGEM + "Alpaca"),
                Item("TOURO",     SELVAGEM + "Bull"),
                Item("CERVO",     SELVAGEM + "Deer"),
                Item("BURRO",     SELVAGEM + "Donkey"),
                Item("RAPOSA",    SELVAGEM + "Fox"),
                Item("LOBO",      SELVAGEM + "Wolf"),
                Item("CACHORRO",  SELVAGEM + "Husky"),
                // Estáticos (Farm, sem esqueleto): ganham "respiração"
                Item("PORCO",     FAZENDA + "Pig"),
                Item("OVELHA",    FAZENDA + "Sheep"),
                Item("ZEBRA",     FAZENDA + "Zebra"),
            }
        },
    },

```

```

new FaseDoJogo
{
    nome = "TRANSPORTE", tempoPorPalavra = 15f,
    itens = new List<ItemDePalavra>
    {
        Item("ONIBUS",      TRANSPORTE + "Bus"),
        Item("TREM",        TRANSPORTE + "Train"),
        Item("TAXI",        TRANSPORTE + "Taxi",
            "Public Transport Pack - Feb 2017/Blends/TaxiTexture"),
        Item("AMBULANCIA", TRANSPORTE + "Ambulance"),
        Item("BICICLETA",  TRANSPORTE + "Bicycle"),
        Item("SEMAFORO",   TRANSPORTE + "TrafficLight"),
    }
},
new FaseDoJogo
{
    nome = "COMIDAS", tempoPorPalavra = 10f,
    itens = new List<ItemDePalavra>
    {
        Item("PIZZA",      COMIDA + "Pizza",      TEXTURAS + "PizzaTexture"),
        Item("BOLO",       COMIDA + "Cake",        TEXTURAS + "CakeTexture"),
        Item("BISCOITO",   COMIDA + "Cookie",      TEXTURAS + "CookieTexture"),
        Item("SORVETE",    COMIDA + "Icecream",    TEXTURAS + "Icecream"),
        Item("ROSQUINHA",  COMIDA + "Donut",       TEXTURAS + "DonutTexture"),
        Item("HAMBURGUER", COMIDA + "Burger",     TEXTURAS + "BurgerTexture"),
        Item("REFRIGERANTE", COMIDA + "Soda",     TEXTURAS + "SodaTexture"),
    }
},
};
}
}

```

Assets/HoverButton.cs (141 linhas)

```
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.Events;

// HoverButton: seleciona um botão quando a mão fica parada sobre ele por 3 segundos.
//
// COMO CONFIGURAR:
// 1) Crie um GameObject filho deste botão chamado "CirculoHover"
// 2) Adicione um componente Image nele (Image Type = Filled, Fill Method = Radial 360,
//    Fill Origin = Top, Fill Amount = 0, Clockwise = true)
// 3) Arraste esse Image para o campo "imagemCirculo" no Inspector
// 4) Arraste o objeto que tem ControladorCamera para o campo "controlador"
// 5) Configure o evento "Ao Selecionar" com o método que deve ser chamado
//    (ex: GerenciadorDeJogo.PularPalavra)
public class HoverButton : MonoBehaviour
{
    [Header("Fonte dos dados da mao")]
    public ControladorCamera controlador;

    [Header("Circulo de progresso")]
    public Image imagemCirculo; // Image com Fill Method = Radial 360
    public float tempoParaSelecionar = 3f;

    [Header("Evento disparado ao selecionar")]
    public UnityEvent aoSelecionar;

    [Header("Visual")]
    public Color corCirculoNormal = new Color(0.2f, 0.8f, 1f, 0.8f);
    public Color corCirculoCompleto = new Color(0.1f, 1f, 0.3f, 1f);

    private float tempoHover = 0f;
    private bool selecionando = false;
    private RectTransform meuRect;
    private Canvas canvas;

    void Awake()
    {
        meuRect = GetComponent<RectTransform>();
        canvas = GetComponentInParent<Canvas>();

        if (imagemCirculo != null)
        {
            imagemCirculo.type = Image.Type.Filled;
            imagemCirculo.fillMethod = Image.FillMethod.Radial360;
            imagemCirculo.fillOrigin = (int)Image.Origin360.Top;
            imagemCirculo.fillAmount = 0f;
            imagemCirculo.color = corCirculoNormal;
        }
    }

    void Update()
    {
        // Só ativa se houver mão detectada
        if (controlador == null || !controlador.MaoDetectada ||
            controlador.PontosDaMaoAtuais == null)
        {
            ResetarHover();
            return;
        }

        Vector2 posicaoMao = ObterPosicaoMaoNaTela();
        bool sobre = EstaSobreEste(posicaoMao);

        if (sobre)
        {
            tempoHover += Time.deltaTime;
            float progresso = Mathf.Clamp01(tempoHover / tempoParaSelecionar);

            if (imagemCirculo != null)
            {
                imagemCirculo.fillAmount = progresso;
                imagemCirculo.color = Color.Lerp(corCirculoNormal, corCirculoCompleto, progresso);
            }

            if (!selecionando && tempoHover >= tempoParaSelecionar)
            {
                selecionando = true;
                Selecionar();
            }
        }
        else
        {
            // Fora do botão: recua o círculo mais rápido do que avança
            tempoHover = Mathf.Max(0f, tempoHover - Time.deltaTime * 1.5f);
            if (imagemCirculo != null)
                imagemCirculo.fillAmount = Mathf.Clamp01(tempoHover / tempoParaSelecionar);

            if (tempoHover <= 0f)
            {
                ResetarHover();
            }
        }
    }

    void ResetarHover()
    {
        tempoHover = 0f;
        selecionando = false;
    }

    void Selecionar()
    {
        aoSelecionar.Invoke();
    }
}
```

```

        selecionando = false;
    }
}

void Selecionar()
{
    Debug.Log("HoverButton: " + gameObject.name + " selecionado pela mao!");
    aoSelecionar?.Invoke();

    // Dispara também o onClick do Button padrão, se existir
    GetComponent<Button>()?.onClick?.Invoke();

    // Reseta após um breve delay para evitar duplo-disparo
    Invoke(nameof(ResetarHover), 0.5f);
}

void ResetarHover()
{
    tempoHover = 0f;
    selecionando = false;
    if (imagemCirculo != null) imagemCirculo.fillAmount = 0f;
}

// Pega a posição do dedo indicador (ponto 8) em coordenadas de tela.
// Usa a conversão do ControladorCamera, que já desconta recorte e espelho -
// assim o círculo enche exatamente onde a mão APARECE na tela.
Vector2 ObterPosicaoMaoNaTela()
{
    Vector3 kp = controlador.PontosDaMaoAtuais[8]; // ponta do indicador
    return controlador.PontoParaTela(kp);
}

// Verifica se a posição de tela está dentro dos limites deste RectTransform
bool EstaSobreEste(Vector2 screenPos)
{
    Camera cam = (canvas != null && canvas.renderMode != RenderMode.ScreenSpaceOverlay)
        ? canvas.worldCamera : null;
    Vector2 localPos;
    return RectTransformUtility.ScreenPointToLocalPointInRectangle(
        meuRect, screenPos, cam, out localPos)
        && meuRect.rect.Contains(localPos);
}

// Desenha um ponto no editor para visualizar onde a mão está
void OnDrawGizmosSelected()
{
    if (!Application.isPlaying || controlador?.PontosDaMaoAtuais == null) return;
    Gizmos.color = Color.cyan;
    Vector2 sp = ObterPosicaoMaoNaTela();
    Gizmos.DrawSphere(new Vector3(sp.x, sp.y, 0), 10f);
}
}

```

Assets/MenuPrincipal.cs (1084 linhas)

```
using UnityEngine;
using UnityEngine.UI;
using TMPro;
using System.Collections.Generic;

// MenuPrincipal: constrói o menu inicial e o HUD do jogo por código.
// Fica no Canvas. Controla a troca entre MENU <-> JOGO <-> TREINAMENTO.
//
// Todos os botões funcionam de 3 formas: toque na tela, clique do mouse,
// ou mão parada sobre o botão por 3 segundos (HoverButton).
public class MenuPrincipal : MonoBehaviour
{
    [Header("Referências (arrastar no Inspector)")]
    public GerenciadorDeJogo gerenciador;
    public ControladorCamera controlador;
    public GameObject painelPalavra; // Panel com o texto da palavra
    public GameObject botaoPular;    // Botão PULAR PALAVRA da cena

    // Paleta de cores do jogo
    // Fundo TRANSLÚCIDO (alfa < 1): a câmera aparece atrás do menu,
    // então o jogador se vê e consegue selecionar botões com a mão
    static readonly Color COR_FUNDO_TOPO = new Color(0.09f, 0.11f, 0.32f, 0.72f); // azul-noite
    static readonly Color COR_FUNDO_BASE = new Color(0.32f, 0.16f, 0.46f, 0.72f); // roxo
    static readonly Color COR_TITULO     = new Color(0.969f, 0.878f, 0.455f, 1f); // #F7E074 amarelo-estrela
    static readonly Color COR_JOGAR      = new Color(0.149f, 0.816f, 0.486f, 1f); // #26D07C verde vibrante
    static readonly Color COR_SINAIS     = new Color(0.922f, 0.596f, 0.184f, 1f); // #EB982F laranja aprendizado
    static readonly Color COR_TREINAR    = new Color(0.149f, 0.584f, 0.816f, 1f); // #2695D0 azul foco
    static readonly Color COR_SAIR       = new Color(0.816f, 0.282f, 0.227f, 1f); // #D0483A vermelho sair
    static readonly Color COR_HUD        = new Color(0.102f, 0.137f, 0.494f, 1f); // #1A237E

    // Ajuste livre do menu pelo Inspector. Cada botao tem posicao e tamanho
    // para PC (paisagem) e para Tablet/Celular (retrato), independentes: mexer
    // num nao mexe no outro. Deixe (0,0) para manter o padrao do codigo.
    [System.Serializable] public class LayBotao { public Vector2 pos; public Vector2 tam; }
    [Header("Layout PC (paisagem) - 0,0 = padrao")]
    public LayBotao pcContinuar, pcJogar, pcAprender, pcTreinar, pcSair;
    [Header("Layout Tablet/Celular (retrato) - 0,0 = padrao")]
    public LayBotao tbContinuar, tbJogar, tbAprender, tbTreinar, tbSair;

    [Header("Mover o objeto 3D (a fruta/animal). PC e Tablet separados (em pixels)")]
    public Vector2 off3dPC;
    public Vector2 off3dTablet;

    [Header("Logo Universo IF (canto superior; desmarque para remover)")]
    public bool mostrarLogo = true;
    GameObject logoUniverso;

    [Header("Senha da area do professor (modo treinamento)")]
    public string senhaAdmin = "1234";

    GameObject telaMenu;
    GameObject botaoMenuHud;
    GameObject headerBar; // barra unica #1A237E no topo, durante o jogo
    GameObject dicaTreinamento;
    TextMeshProUGUI textoContagem; // contador de amostras por letra (treinamento)

    GameObject botaoSom; // liga/desliga a música (sempre visível)
    GameObject riscoSom; // risco vermelho = música desligada
    GameObject botaoPularLetra; // "PULAR LETRA -5" (aparece junto do outro)
    GameObject botaoContinuar; // "CONTINUAR" (só quando há jogo pausado)
    TextMeshProUGUI rotuloJogar; // vira "RECOMEÇAR" quando há jogo pausado

    GameObject painelSenha; // teclado numérico da área do professor
    TextMeshProUGUI displaySenha;
    TextMeshProUGUI textoRecorde; // "RECORDE: X" no menu
    string senhaDigitada = "";

    bool menuAberto;
    bool jogoPausado;
    float timerFimDeJogo;
    float timerContagem;

    // Distancia da camera em que o objeto 3D e exibido
    const float DISTANCIA_DO_OBJETO = 15f;
    float larguraDoSlot = 300f; // espaco reservado na esquerda do cartao
    int larguraDaTelaAnterior, alturaDaTelaAnterior;

    // Orientacao da tela: false = celular (retrato), true = PC/tablet (paisagem)
    bool telaHorizontal;
    // Verdadeiro quando o usuario escolheu o formato a dedo, nas opcoes.
    // Enquanto for falso, o formato acompanha a janela.
    bool escolhaDeTelaFeita;
    int larguraJanelaAnterior, alturaJanelaAnterior;
    CanvasScaler escalador;

    // Painel de opções (engrenagem): tela, tempo/vidas, espelho da câmera
    GameObject painelOpcoes;
    TextMeshProUGUI rotuloTela;
```

```

TextMeshProUGUI rotuloPressao;
TextMeshProUGUI rotuloEspelho;
TextMeshProUGUI tmpTitulo1; // texto do título (muda entre os modos)
UIControle uiControle; // para abrir o espaço do objeto no cartão

// Referências para reposicionar o menu conforme a orientacao
RectTransform rtTitulo1, rtTitulo2, rtSubtitulo, rtRecorde;
RectTransform rtContinuar, rtJogar, rtAprender, rtTreinar, rtSair, rtDicaMenu, rtCreditos;
ModoEstudo telaEstudo;
readonly List<RectTransform> estrelasFundo = new List<RectTransform>();
readonly List<Vector2> estrelasBase = new List<Vector2>();
readonly List<RectTransform> brilhosTitulo = new List<RectTransform>();
readonly List<Vector2> brilhosBase = new List<Vector2>();

void Awake()
{
    ConstruirMenu();
    ConstruirHud();
    CriarLogo();
}

void Start()
{
    // Recupera as escolhas da última vez (tela, tempo/vidas, espelho)
    // Sem escolha salva, o formato segue o formato REAL da janela. 0
    // padrao antigo era sempre retrato, o que no computador montava um
    // menu de celular numa janela larga e empilhava os elementos uns
    // sobre os outros. So passa a valer a preferencia depois que alguem
    // troca de propósito, pelas opcoes.
    escolhaDeTelaFeita = PlayerPrefs.GetInt("telaEscolhida", 0) == 1;
    telaHorizontal = escolhaDeTelaFeita
        ? PlayerPrefs.GetInt("telaHorizontal", 0) == 1
        : Screen.width >= Screen.height;
    if (gerenciador != null)
        gerenciador.modoSemPressao = PlayerPrefs.GetInt("semPressao", 0) == 1;
    AplicarEspelho(PlayerPrefs.GetInt("espelharImagem", 1) == 1);
    AplicarOrientacao();

    AbrirMenu();
    TrazerLogoAFrente();
    GerenciadorDeAudio.TocarMusica();
    // O jogador pode ter desligado a música na última vez - reflete no ícone
    riscoSom.SetActive(!GerenciadorDeAudio.Musicaligada);
}

void Update()
{
    // Janela redimensionada no computador: enquanto ninguem tiver
    // escolhido o formato a dedo, o layout acompanha a janela
    if (Screen.width != larguraJanelaAnterior || Screen.height != alturaJanelaAnterior)
    {
        larguraJanelaAnterior = Screen.width;
        alturaJanelaAnterior = Screen.height;

        if (!escolhaDeTelaFeita)
        {
            bool deitada = Screen.width >= Screen.height;
            if (deitada != telaHorizontal)
            {
                telaHorizontal = deitada;
                AplicarOrientacao();
            }
        }
    }

    // Quando o jogo termina, espera e volta ao menu
    // (na vitória espera mais: tempo de curtir os confetes!)
    if (!menuAberto && gerenciador != null && gerenciador.JogoTerminado)
    {
        timerFimDeJogo += Time.deltaTime;
        if (timerFimDeJogo >= (gerenciador.Venceu ? 8f : 4f)) AbrirMenu();
    }
    else
    {
        timerFimDeJogo = 0f;
    }

    // Se a janela mudar de tamanho, o objeto 3D reencontra o cartão
    if (Screen.width != larguraDaTelaAnterior || Screen.height != alturaDaTelaAnterior)
    {
        larguraDaTelaAnterior = Screen.width;
        alturaDaTelaAnterior = Screen.height;
        PosicionarObjeto3D();
    }

    // No treinamento, atualiza o contador de amostras a cada meio segundo
    if (dicaTreinamento != null && dicaTreinamento.activeSelf &&
        controlador != null && controlador.reconhecedor != null)
    {
        timerContagem -= Time.deltaTime;
        if (timerContagem <= 0f)
        {

```

```

        timerContagem = 0.5f;
        textoContagem.text = controlador.reconhecedor.ResumoDoBanco();
    }
}

// — Construção da interface —————

void ConstruirMenu()
{
    // Fundo em gradiente (azul-noite -> roxo) cobrindo a tela inteira
    var fundo = UIFabrica.CriarImagem(transform, "TelaMenu", Color.white,
        Vector2.zero, Vector2.zero, UIFabrica.Gradient(COR_FUNDO_TOPO, COR_FUNDO_BASE));
    var rt = fundo.rectTransform;
    rt.anchorMin = Vector2.zero;
    rt.anchorMax = Vector2.one;
    rt.sizeDelta = Vector2.zero;
    telaMenu = fundo.gameObject;

    // Estrelas decorativas cintilando (cada uma no seu ritmo)
    var sorteio = new System.Random(7);
    for (int i = 0; i < 45; i++)
    {
        float x = (float)sorteio.NextDouble() * 1000f - 500f;
        float y = (float)sorteio.NextDouble() * 1780f - 890f;
        float tam = 4f + (float)sorteio.NextDouble() * 10f;
        float alfa = 0.25f + (float)sorteio.NextDouble() * 0.6f;
        var estrela = UIFabrica.CriarImagem(telaMenu.transform, "Estrela",
            new Color(1f, 1f, 1f, alfa),
            new Vector2(x, y), new Vector2(tam, tam), UIFabrica.Circulo());
        estrela.gameObject.AddComponent<Cintilar>();
        estrelasFundo.Add(estrela.rectTransform);
        estrelasBase.Add(new Vector2(x, y));
    }

    rtTitulo1 = UIFabrica.CriarTexto(telaMenu.transform, "Titulo1", "DITADO",
        130f, COR_TITULO, new Vector2(0, 620), new Vector2(1000, 150)).rectTransform;
    rtTitulo2 = UIFabrica.CriarTexto(telaMenu.transform, "Titulo2", "ESTRELADO",
        130f, COR_TITULO, new Vector2(0, 480), new Vector2(1000, 150)).rectTransform;

    // Pontos de luz SOBRE o título, pulsando como estrelas no céu
    for (int i = 0; i < 16; i++)
    {
        float x = (float)sorteio.NextDouble() * 680f - 340f;
        float y = 425f + (float)sorteio.NextDouble() * 265f;
        float tam = 5f + (float)sorteio.NextDouble() * 9f;
        var brilho = UIFabrica.CriarImagem(telaMenu.transform, "BrilhoTitulo",
            new Color(1f, 1f, 0.85f, 0.9f),
            new Vector2(x, y), new Vector2(tam, tam), UIFabrica.Circulo());
        brilho.raycastTarget = false;
        var cintilar = brilho.gameObject.AddComponent<Cintilar>();
        cintilar.alfaMinimo = 0f; // some completamente...
        cintilar.escalaExtra = 0.8f; // ...e volta inchando, bem "estrela"
        brilhosTitulo.Add(brilho.rectTransform);
        brilhosBase.Add(new Vector2(x, y));
    }

    rtSubtitulo = UIFabrica.CriarTexto(telaMenu.transform, "Subtitulo", "Aprenda o alfabeto em LIBRAS",
        46f, new Color(1f, 1f, 1f, 0.9f), new Vector2(0, 360), new Vector2(1000, 80), false).rectTransform;

    // Recorde do jogador (salvo entre sessões; atualizado ao abrir o menu)
    textoRecorde = UIFabrica.CriarTexto(telaMenu.transform, "Recorde", "",
        36f, COR_TITULO, new Vector2(0, 295), new Vector2(700, 55));
    rtRecorde = textoRecorde.rectTransform;

    // Engrenagem de opções no canto superior esquerdo do menu
    // (alinhada com o botão de som, que fica no canto direito)
    var engrenagem = UIFabrica.CriarBotao(telaMenu.transform, "BotaoOpcoes", "",
        COR_HUD, new Vector2(30, -30), new Vector2(150, 90), 30f, controlador, AbrirOpcoes);
    UIFabrica.Ancorar(engrenagem, new Vector2(0f, 1f), new Vector2(0f, 1f));
    var iconeEngrenagem = UIFabrica.CriarImagem(engrenagem.transform, "Icône",
        Color.white, Vector2.zero, new Vector2(56, 56), UIFabrica.Engrenagem());
    iconeEngrenagem.raycastTarget = false;

    ConstruirPainelOpcoes();

    // CONTINUAR: aparece apenas quando o menu foi aberto no MEIO de um
    // jogo (pausa) - retoma exatamente de onde parou
    var continuar = UIFabrica.CriarBotao(telaMenu.transform, "BotaoContinuar",
        "CONTINUAR", new Color(0.10f, 0.78f, 0.55f, 1f),
        new Vector2(0, 195), new Vector2(560, 130), 52f, controlador, Continuar);
    botaoContinuar = continuar.gameObject;
    botaoContinuar.SetActive(false);
    rtContinuar = continuar.GetComponent<RectTransform>();
    AdicionarIcône(continuar.transform, UIFabrica.Seta(), false, 46f);

    // Acao primaria: maior, no topo da piramide, com a seta de "play"
    var jogar = UIFabrica.CriarBotao(telaMenu.transform, "BotaoJogar", "JOGAR", COR_JOGAR,
        new Vector2(0, 60), new Vector2(560, 130), 56f, controlador, Jogar);
    rotuloJogar = jogar.transform.Find("Rotulo").GetComponent<TextMeshProUGUI>();
    rtJogar = jogar.GetComponent<RectTransform>();
    AdicionarIcône(jogar.transform, UIFabrica.Seta(), false, 50f); // ► play (JOGAR)

```

```

var iconeVoltar = UIFabrica.CriarImagem(jogar.transform, "IcôneVoltar", Color.white,
    Vector2.zero, new Vector2(54, 54), Resources.Load<Sprite>("ui/recomecar"));
iconeVoltar.raycastTarget = false; iconeVoltar.preserveAspect = true;
var ivr = iconeVoltar.rectTransform;
ivr.anchorMin = ivr.anchorMax = new Vector2(0f, 0.5f); ivr.pivot = new Vector2(0f, 0.5f);
ivr.anchoredPosition = new Vector2(28f, 0f);
iconeVoltar.gameObject.SetActive(false);

// Acoes secundarias: lado a lado, tamanho intermediario
var aprender = UIFabrica.CriarBotao(telaMenu.transform, "BotaoAprender", "APRENDA OS SINAIS",
    COR_SINAIS, new Vector2(0, -75), new Vector2(560, 125), 46f, controlador, AbrirEstudo);
rtAprender = aprender.GetComponent<RectTransform>();
AdicionarIcône(aprender.transform, Resources.Load<Sprite>("simbolo_libras"), false, 54f);

var treinar = UIFabrica.CriarBotao(telaMenu.transform, "BotaoTreinar", "TREINAMENTO", COR_TREINAR,
    new Vector2(0, -110), new Vector2(560, 130), 52f, controlador, PedirSenha);
rtTreinar = treinar.GetComponent<RectTransform>();
AdicionarIcône(treinar.transform, UIFabrica.Estrela(), false, 46f);

// Acao negativa: menor e na base, para evitar toque acidental
var sair = UIFabrica.CriarBotao(telaMenu.transform, "BotaoSair", "SAIR", COR_SAIR,
    new Vector2(0, -280), new Vector2(560, 130), 52f, controlador, Sair);
rtSair = sair.GetComponent<RectTransform>();
AdicionarIcône(sair.transform, UIFabrica.Xis(), false, 40f);

ConstruirPainelSenha();

rtDicaMenu = UIFabrica.CriarTexto(telaMenu.transform, "Dica",
    "Toque no botão ou aponte o dedo por 3 segundos",
    32f, new Color(1f, 1f, 1f, 0.7f), new Vector2(0, -460), new Vector2(1000, 60), false).rectTransform;

// — Rodapé: uma única linha com a identificação do autor —
// Fundo escuro discreto só para a frase ficar legível sobre a câmera.
var cartaoCreditos = UIFabrica.CriarImagem(telaMenu.transform, "Creditos",
    new Color(0f, 0f, 0f, 0.28f), new Vector2(0, -700), new Vector2(1600, 92),
    UIFabrica.Arredondado(), true);
cartaoCreditos.raycastTarget = false;
rtCreditos = cartaoCreditos.rectTransform;

// O texto estica junto com o cartão e encolhe a fonte até caber; em
// telas estreitas (retrato) ele quebra em duas linhas por conta própria.
var autoria = UIFabrica.CriarTexto(cartaoCreditos.transform, "Autoria",
    "Desenvolvido por: Marcus Vinicius Souza Batista Strabello, mestrando em Computação " +
    "Aplicada (PPGCA) pelo IFMA - Campus Monte Castelo. " +
    "Coordenador: Dr. Daniel Lima Gomes Junior.",
    26f, new Color(1f, 1f, 1f, 0.92f), Vector2.zero, new Vector2(1540, 80), false);
autoria.enableAutoSize = true;
autoria.fontSizeMin = 15f;
autoria.fontSizeMax = 26f;
var autoriaRt = autoria.rectTransform;
autoriaRt.anchorMin = Vector2.zero;
autoriaRt.anchorMax = Vector2.one;
autoriaRt.sizeDelta = new Vector2(-44, -10);
autoriaRt.anchoredPosition = Vector2.zero;
}

// Coloca um ícone branco dentro de um botão, preso a uma das laterais para
// não se deslocar quando o botão muda de tamanho entre retrato e paisagem.
void AdicionarIcône(Transform botao, Sprite sprite, bool aDireita, float tam)
{
    if (sprite == null) return; // sem sprite, não cria um quadrado branco
    var icone = UIFabrica.CriarImagem(botao, "Icône", Color.white,
        Vector2.zero, new Vector2(tam, tam), sprite);
    icone.raycastTarget = false;
    var rt = icone.rectTransform;
    rt.anchorMin = rt.anchorMax = new Vector2(aDireita ? 1f : 0f, 0.5f);
    rt.pivot = new Vector2(aDireita ? 1f : 0f, 0.5f);
    rt.anchoredPosition = new Vector2(aDireita ? -30f : 30f, 0f);
}

// Estrela com um "X" no centro (ícone de "pular esta letra")
void AdicionarEstrelaX(Transform botao)
{
    var estrela = UIFabrica.CriarImagem(botao, "Icône", Color.white,
        Vector2.zero, new Vector2(56, 56), UIFabrica.Estrela());
    estrela.raycastTarget = false;
    var rt = estrela.rectTransform;
    rt.anchorMin = rt.anchorMax = new Vector2(0f, 0.5f);
    rt.pivot = new Vector2(0f, 0.5f);
    rt.anchoredPosition = new Vector2(30f, 0f);

    var x = UIFabrica.CriarImagem(estrela.transform, "X",
        new Color(0.102f, 0.137f, 0.494f, 1f), Vector2.zero, new Vector2(24, 24), UIFabrica.Xis());
    x.raycastTarget = false;
    var xr = x.rectTransform;
    xr.anchorMin = Vector2.zero; xr.anchorMax = Vector2.one;
    xr.sizeDelta = new Vector2(-24, -24); xr.anchoredPosition = Vector2.zero;
}

// Pinta o rótulo de um botão (texto), sem mexer no resto
void PintarRotulo(Button b, Color c)
{

```



```

        var t = b.transform.Find("Rotulo");
        if (t != null)
        {
            var tmp = t.GetComponent<TextMeshProUGUI>();
            if (tmp != null) tmp.color = c;
        }
    }

    // Troca o visual de um botao pela imagem-asset (pilula pronta), escondendo
    // qualquer rotulo antigo. Mantem o Button e o HoverButton funcionando.
    void UsarImagemDeBotao(Button b, string sprite)
    {
        if (b == null) return;
        var img = b.GetComponent<Image>();
        if (img != null)
        {
            var sp = Resources.Load<Sprite>(sprite);
            if (sp != null) { img.sprite = sp; img.type = Image.Type.Simple; img.preserveAspect = true; }
            img.color = Color.white;
        }
        // esconde textos antigos do botao (o rotulo ja esta desenhado na imagem)
        foreach (var t in b.GetComponentsInChildren<TextMeshProUGUI>(true))
            t.gameObject.SetActive(false);
        // remove icones que eu tinha desenhado por codigo
        foreach (Transform ch in b.transform)
            if (ch.name == "Icone" || ch.name == "X") Destroy(ch.gameObject);
    }

    // Texto pequeno do custo (-5 / -10) na parte de baixo do botao
    void AdicionarCusto(Transform botao, string custo, Color cor)
    {
        var t = UIFabrica.CriarTexto(botao, "Custo", custo, 44f, cor,
            new Vector2(0, -46), new Vector2(220, 56));
        t.raycastTarget = false;
        var rt = t.rectTransform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0f);
        rt.pivot = new Vector2(0.5f, 0f);
        rt.anchoredPosition = new Vector2(0, 14f);
    }

    static Vector2 EscolherOu(Vector2 valor, Vector2 padrao) => valor == Vector2.zero ? padrao : valor;

    void Por(RectTransform rt, LayBotao l)
    {
        if (rt == null || l == null) return;
        if (l.pos != Vector2.zero) rt.anchoredPosition = l.pos;
        if (l.tam != Vector2.zero) rt.sizeDelta = l.tam;
    }

    // Aplica o que voce ajustou no Inspector por cima do layout automatico.
    void AplicarAjustesInspector(bool h)
    {
        Por(rtContinuar, h ? pcContinuar : tbContinuar);
        Por(rtJogar, h ? pcJogar : tbJogar);
        Por(rtAprender, h ? pcAprender : tbAprender);
        Por(rtTreinar, h ? pcTreinar : tbTreinar);
        Por(rtSair, h ? pcSair : tbSair);
    }

    // Quando ha jogo pausado, CONTINUAR e RECOMEÇAR ficam lado a lado (no lugar
    // do JOGAR), alinhados com os botoes de baixo. Continuar leva a seta de
    // play; Recomeçar leva a mesma seta invertida (voltar).
    void PosicionarBotoesContinuar(bool pausado)
    {
        bool h = telaHorizontal;
        if (pausado)
        {
            // Paisagem: lado a lado. Retrato: empilhado (coluna), largura cheia.
            if (rtContinuar != null) { rtContinuar.anchoredPosition = h ? new Vector2(-330, 110) : new Vector2(0,
200);
                rtContinuar.sizeDelta = h ? new Vector2(610, 130) : new Vector2(820, 120);
            }

            if (rtJogar != null) { rtJogar.anchoredPosition = h ? new Vector2(330, 110) : new Vector2(0, 55);
                rtJogar.sizeDelta = h ? new Vector2(610, 130) : new Vector2(820, 130); }
            FlipIconeJogar(true);
        }
        else
        {
            if (rtJogar != null) { rtJogar.anchoredPosition = h ? new Vector2(0, 110) : new Vector2(0, 55);
                rtJogar.sizeDelta = h ? new Vector2(540, 140) : new Vector2(820, 130); }
            FlipIconeJogar(false);
        }
    }

    // Inverte o icone do JOGAR: play (->) quando "JOGAR/RECOMEÇAR" avança,
    // seta invertida (<-) para dar sentido de "voltar" no RECOMEÇAR pausado.
    void FlipIconeJogar(bool voltar)
    {
        if (rtJogar == null) return;
        var play = rtJogar.Find("Icone");
        var back = rtJogar.Find("IconeVoltar");
        if (play != null) play.gameObject.SetActive(!voltar);
    }

```

```

        if (back != null) back.gameObject.SetActive(voltar);
    }

    void CriarLogo()
    {
        if (!mostrarLogo) return;
        var logo = UIFabrica.CriarImagem(transform, "LogoUniverso", Color.white,
            new Vector2(0, -16), new Vector2(150, 129), Resources.Load<Sprite>("ui/logo_universo"));
        logo.preserveAspect = true; logo.raycastTarget = false;
        UIFabrica.Ancorar(logo, new Vector2(0.5f, 1f), new Vector2(0.5f, 1f));
        logoUniverso = logo.gameObject;
    }

    void TrazerLogoAFrente() { if (logoUniverso != null) logoUniverso.transform.SetAsLastSibling(); }

    // Ajuste específico do modo estudo (logo menor e mais no topo, para não
    // cobrir o título APRENDA OS SINAIS) sem afetar os outros modos.
    void AjustarLogo(bool estudo)
    {
        if (logoUniverso == null) return;
        var rt = (RectTransform)logoUniverso.transform;
        rt.sizeDelta = estudo ? new Vector2(86, 74) : new Vector2(100, 86);
        rt.anchoredPosition = estudo ? new Vector2(0, -4) : new Vector2(0, -8);
    }

    void ConstruirHud()
    {
        // Barra única do topo: uma faixa cheia #1A237E atrás de pontos, vidas,
        // tempo, menu e som, unificando tudo num só cabeçalho. Fica ATRÁS de
        // todos (SetAsFirstSibling), então os chips e botões continuam por cima
        // e funcionando; a cor deles virou a mesma da barra, formando uma faixa
        // contínua. A barra só aparece no jogo, junto com o botão MENU.
        var header = UIFabrica.CriarImagem(transform, "HeaderBar",
            new Color(0f, 0f, 0f, 0f), Vector2.zero, new Vector2(0, 200),
            UIFabrica.Arredondado(), true);
        var hr = header.rectTransform;
        hr.anchorMin = new Vector2(0f, 1f);
        hr.anchorMax = new Vector2(1f, 1f);
        hr.pivot = new Vector2(0.5f, 1f);
        hr.sizeDelta = new Vector2(0, 200);
        hr.anchoredPosition = Vector2.zero;
        header.raycastTarget = false;
        // A barra vai para o MESMO pai da camera (o Canvas), logo ACIMA dela e
        // ABAIXO dos chips (pontos/vidas/tempo, criados depois). Sem isto a
        // barra ficava atrás da imagem da camera e não aparecia.
        if (controlador != null && controlador.fundoDoEcra != null)
        {
            var camera = controlador.fundoDoEcra.transform;
            header.transform.SetParent(camera.parent, false);
            camera.SetAsFirstSibling(); // camera sempre no fundo
            header.transform.SetSiblingIndex(1); // barra logo acima da camera
        }
        else
        {
            header.transform.SetAsFirstSibling();
        }
        headerBar = header.gameObject;
        headerBar.SetActive(false);

        // Botão MENU no canto superior direito (visível durante jogo/treinamento)
        var botao = UIFabrica.CriarBotao(transform, "BotaoMenuHud", "MENU", COR_HUD,
            new Vector2(-30, -30), new Vector2(220, 90), 40f, controlador, AbrirMenuComSom);
        UIFabrica.Ancorar(botao, new Vector2(1f, 1f), new Vector2(1f, 1f));
        botaoMenuHud = botao.gameObject;

        // Botão de SOM logo abaixo do MENU - visível SEMPRE (menu e jogo)
        var som = UIFabrica.CriarBotao(transform, "BotaoSom", "", COR_HUD,
            new Vector2(-30, -135), new Vector2(150, 90), 38f, controlador, AlternarSom);
        UIFabrica.Ancorar(som, new Vector2(1f, 1f), new Vector2(1f, 1f));
        botaoSom = som.gameObject;

        // Ícone de alto-falante desenhado por código (na estética do jogo)
        var iconeSom = UIFabrica.CriarImagem(som.transform, "Icône",
            Color.white, Vector2.zero, new Vector2(58, 58), UIFabrica.AltoFalante());
        iconeSom.raycastTarget = false;

        // Risco vermelho na diagonal = música desligada
        var risco = UIFabrica.CriarImagem(som.transform, "Risco",
            new Color(0.9f, 0.2f, 0.2f, 0.95f), Vector2.zero, new Vector2(120, 10),
            UIFabrica.Arredondado(), true);
        risco.rectTransform.localEulerAngles = new Vector3(0, 0, 20f);
        risco.raycastTarget = false;
        riscoSom = risco.gameObject;
        riscoSom.SetActive(false);

        // PULAR LETRA e PULAR PALAVRA usam os assets prontos (a pilula com o
        // rotulo e o icone já desenhados). O botao vira apenas a imagem; por
        // cima, um texto pequeno mostra o custo (-5 / -10) na parte de baixo.
        var pularLetra = UIFabrica.CriarBotao(transform, "BotaoPularLetra",
            "", Color.white, new Vector2(-250, 175), new Vector2(500, 175), 1f,
            controlador, () => gerenciador.PularLetra());
        UIFabrica.Ancorar(pularLetra, new Vector2(0.5f, 0f), new Vector2(0.5f, 0.5f));
    }

```

```

botaoPularLetra = pularLetra.gameObject;
UsarImagemDeBotao(pularLetra, "ui/pular_letra");
AdicionarCusto(pularLetra.transform, "-5", new Color(0.42f, 0.30f, 0f, 1f));

if (botaoPular != null)
{
    UsarImagemDeBotao(botaoPular.GetComponent<Button>(), "ui/pular_palavra");
    AdicionarCusto(botaoPular.transform, "-10", new Color(0.03f, 0.28f, 0.06f, 1f));
}

// Cartão de instruções do modo treinamento
var painel = UIFabrica.CriarImagem(transform, "DicaTreinamento",
    new Color(1f, 1f, 1f, 0.88f), new Vector2(0, -160), new Vector2(980, 470),
    UIFabrica.Arredondado(), true);
UIFabrica.Ancorar(painel, new Vector2(0.5f, 1f), new Vector2(0.5f, 1f));
UIFabrica.CriarTexto(painel.transform, "Texto",
    "MODO TREINAMENTO\n\n" +
    "Faça o sinal e pressione a tecla da letra\n" +
    "para gravar (grave várias vezes!)\n" +
    "H J K W X Z Ç gravam o MOVIMENTO por 1,3s\n" +
    "(o Ç grava pela tecla Ç do teclado)\n" +
    "Shift + tecla apaga a letra",
    36f, new Color(0.15f, 0.15f, 0.22f, 1f), new Vector2(0, 80), new Vector2(940, 290), false);

// Contador ao vivo: quantas amostras cada letra tem no banco
// (várias linhas, com quebra automática feita pelo ResumoDoBanco)
textoContagem = UIFabrica.CriarTexto(painel.transform, "Contagem", "",
    34f, new Color(0.1f, 0.35f, 0.7f, 1f), new Vector2(0, -155), new Vector2(940, 150));
textoContagem.enableAutoSizing = true;
textoContagem.fontSizeMin = 24f;
textoContagem.fontSizeMax = 36f;

dicaTreinamento = painel.gameObject;
}

// — Painel de senha (área do professor) —————
void ConstruirPainelSenha()
{
    var fundo = UIFabrica.CriarImagem(telaMenu.transform, "PainelSenha",
        new Color(0.07f, 0.09f, 0.25f, 0.97f), Vector2.zero, new Vector2(680, 1000),
        UIFabrica.Arredondado(), true);
    painelSenha = fundo.gameObject;

    UIFabrica.CriarTexto(fundo.transform, "Titulo", "ÁREA DO PROFESSOR",
        48f, COR_TITULO, new Vector2(0, 410), new Vector2(640, 80));
    UIFabrica.CriarTexto(fundo.transform, "Sub", "Digite a senha para o treinamento",
        30f, new Color(1f, 1f, 1f, 0.8f), new Vector2(0, 345), new Vector2(640, 50), false);

    // Visor da senha (mostra bolinhas)
    var visor = UIFabrica.CriarImagem(fundo.transform, "Visor",
        new Color(1f, 1f, 1f, 0.92f), new Vector2(0, 260), new Vector2(420, 90),
        UIFabrica.Arredondado(), true);
    displaySenha = UIFabrica.CriarTexto(visor.transform, "Texto", "",
        52f, new Color(0.1f, 0.1f, 0.2f, 1f), Vector2.zero, new Vector2(400, 90));

    // Teclado numérico 3x4: 1-9, C (limpar), 0, OK
    string[] teclas = { "1", "2", "3", "4", "5", "6", "7", "8", "9", "C", "0", "OK" };
    for (int i = 0; i < teclas.Length; i++)
    {
        string tecla = teclas[i]; // cópia local: cada botão guarda a sua!
        int coluna = i % 3, linha = i / 3;
        Vector2 pos = new Vector2((coluna - 1) * 200f, 130f - linha * 155f);
        Color cor = (tecla == "OK") ? COR_JOGAR : (tecla == "C") ? COR_SAIR : COR_TREINAR;
        UIFabrica.CriarBotao(fundo.transform, "Tecla" + tecla, tecla, cor,
            pos, new Vector2(180, 135), 48f, controlador, () => TeclaSenha(tecla));
    }

    UIFabrica.CriarBotao(fundo.transform, "Voltar", "VOLTAR",
        new Color(0.4f, 0.4f, 0.5f, 1f), new Vector2(0, -420), new Vector2(300, 90),
        36f, controlador, FecharPainelSenha);

    painelSenha.SetActive(false);
}

void PedirSenha()
{
    GerenciadorDeAudio.TocarClique();
    senhaDigitada = "";
    AtualizarVisorSenha();
    painelSenha.SetActive(true);
    painelSenha.transform.SetAsLastSibling(); // por cima dos botões do menu
}

void FecharPainelSenha()
{
    GerenciadorDeAudio.TocarClique();
    painelSenha.SetActive(false);
}

void TeclaSenha(string tecla)
{
    GerenciadorDeAudio.TocarClique();

```

```

        if (tecla == "C")
        {
            senhaDigitada = "";
        }
        else if (tecla == "OK")
        {
            if (senhaDigitada == senhaAdmin)
            {
                painelSenha.SetActive(false);
                Treinar();
                return;
            }
            displaySenha.text = "ERRADA!";
            displaySenha.color = new Color(0.8f, 0.15f, 0.15f, 1f);
            senhaDigitada = "";
            return;
        }
        else if (senhaDigitada.Length < 8)
        {
            senhaDigitada += tecla;
        }
        AtualizarVisorSenha();
    }

    void AtualizarVisorSenha()
    {
        displaySenha.color = new Color(0.1f, 0.1f, 0.2f, 1f);
        displaySenha.text = new string('*', senhaDigitada.Length);
    }

    public void AlternarSom()
    {
        GerenciadorDeAudio.TocarClique();
        GerenciadorDeAudio.AlternarMusica();
        riscoSom.SetActive(!GerenciadorDeAudio.Musicaligada);
    }

    // — Painel de opções (engrenagem) —————

    void ConstruirPainelOpcoes()
    {
        var fundo = UIFabrica.CriarImagem(telaMenu.transform, "PainelOpcoes",
            new Color(0.07f, 0.09f, 0.25f, 0.97f), Vector2.zero, new Vector2(700, 640),
            UIFabrica.Arredondado(), true);
        painelOpcoes = fundo.gameObject;

        UIFabrica.CriarTexto(fundo.transform, "Titulo", "OPÇÕES",
            52f, COR_TITULO, new Vector2(0, 250), new Vector2(640, 80));

        rotuloTela = CriarLinhaDeOpcao(fundo.transform, new Vector2(0, 140), AlternarTela);
        rotuloPressao = CriarLinhaDeOpcao(fundo.transform, new Vector2(0, 10), AlternarPressao);
        rotuloEspelho = CriarLinhaDeOpcao(fundo.transform, new Vector2(0, -120), AlternarEspelho);

        UIFabrica.CriarBotao(fundo.transform, "Fechar", "FECHAR",
            new Color(0.4f, 0.4f, 0.5f, 1f), new Vector2(0, -250), new Vector2(300, 90),
            36f, controlador, FecharOpcoes);

        painelOpcoes.SetActive(false);
    }

    TextMeshProUGUI CriarLinhaDeOpcao(Transform pai, Vector2 pos,
        UnityEngine.Events.UnityAction acao)
    {
        var botao = UIFabrica.CriarBotao(pai, "Opcao", "", COR_TREINAR, pos,
            new Vector2(600, 110), 32f, controlador, acao);
        return botao.transform.Find("Rotulo").GetComponent<TextMeshProUGUI>();
    }

    void AbrirOpcoes()
    {
        GerenciadorDeAudio.TocarClique();
        // Clicar na engrenagem alterna: se o painel ja esta aberto, fecha.
        if (painelOpcoes.activeSelf) { painelOpcoes.SetActive(false); return; }
        AtualizarRotulosOpcoes();
        painelOpcoes.SetActive(true);
        painelOpcoes.transform.SetAsLastSibling();
    }

    void FecharOpcoes()
    {
        GerenciadorDeAudio.TocarClique();
        painelOpcoes.SetActive(false);
    }

    void AtualizarRotulosOpcoes()
    {
        if (rotuloTela != null)
            rotuloTela.text = telaHorizontal ? "TELA: PC / TABLET" : "TELA: CELULAR";
        if (rotuloPressao != null && gerenciador != null)
            rotuloPressao.text = gerenciador.modosSemPressao
                ? "TEMPO E VIDAS: NÃO" : "TEMPO E VIDAS: SIM";
        if (rotuloEspelho != null && controlador != null)
    }

```

```

        rotuloEspelho.text = controlador.espelharImagem
        ? "ESPELHAR CÂMERA: SIM" : "ESPELHAR CÂMERA: NÃO";
    }

    void AlternarTela()
    {
        GerenciadorDeAudio.TocarClique();
        telaHorizontal = !telaHorizontal;
        escolhaDeTelaFeita = true;
        PlayerPrefs.SetInt("telaHorizontal", telaHorizontal ? 1 : 0);
        PlayerPrefs.SetInt("telaEscolhida", 1);
        AplicarOrientacao();
        AtualizarRotulosOpcoes();
    }

    // Liga/desliga o tempo e as vidas (modo tranquilo para iniciantes)
    void AlternarPressao()
    {
        GerenciadorDeAudio.TocarClique();
        gerenciador.modosSemPressao = !gerenciador.modosSemPressao;
        PlayerPrefs.SetInt("semPressao", gerenciador.modosSemPressao ? 1 : 0);
        AtualizarRotulosOpcoes();
    }

    // Espelha (ou não) a imagem da câmera e a leitura da mão junto
    void AlternarEspelho()
    {
        GerenciadorDeAudio.TocarClique();
        AplicarEspelho(!controlador.espelharImagem);
        PlayerPrefs.SetInt("espelharImagem", controlador.espelharImagem ? 1 : 0);
        AtualizarRotulosOpcoes();
    }

    void AplicarEspelho(bool espelhar)
    {
        controlador.espelharImagem = espelhar;
        if (controlador.fundoDoEcra != null)
        {
            var escala = controlador.fundoDoEcra.rectTransform.localScale;
            escala.x = espelhar ? -Mathf.Abs(escala.x) : Mathf.Abs(escala.x);
            controlador.fundoDoEcra.rectTransform.localScale = escala;
        }
    }

    // Reposiciona TUDO conforme a orientação.
    // Retrato: menu em coluna única (como no celular), em pirâmide de tamanhos.
    // Paisagem: título numa linha no topo, JOGAR em destaque, as duas
    // secundárias lado a lado, SAIR na base, e a autoria no rodapé.
    // Nos dois modos o objeto 3D fica DENTRO do cartão da palavra, no espaço
    // à esquerda, ao lado das letras (não cobre o rosto nem as mãos).
    void AplicarOrientacao()
    {
        bool h = telaHorizontal;

        if (escalador == null) escalador = GetComponent<CanvasScaler>();
        if (escalador != null)
            escalador.referenceResolution = h ? new Vector2(1920, 1080)
            : new Vector2(1080, 1920);
        if (telaEstudo != null) telaEstudo.AplicarLayout(h);

        void Pos(RectTransform rt, Vector2 retrato, Vector2 paisagem)
        {
            if (rt != null) rt.anchoredPosition = h ? paisagem : retrato;
        }

        void PosTam(RectTransform rt, Vector2 posR, Vector2 tamR,
                    Vector2 posP, Vector2 tamP)
        {
            if (rt == null) return;
            rt.anchoredPosition = h ? posP : posR;
            rt.sizeDelta = h ? tamP : tamR;
        }

        // — Menu —
        // Na paisagem o título vira UMA linha ("DITADO ESTRELADO") no topo
        if (tmpTitulo1 == null && rtTitulo1 != null)
            tmpTitulo1 = rtTitulo1.GetComponent<TextMeshProUGUI>();
        if (tmpTitulo1 != null) tmpTitulo1.text = h ? "DITADO ESTRELADO" : "DITADO";
        if (rtTitulo1 != null) rtTitulo1.sizeDelta = h ? new Vector2(1800, 150)
            : new Vector2(1000, 150);
        if (rtTitulo2 != null) rtTitulo2.gameObject.SetActive(!h);

        Pos(rtTitulo1, new Vector2(0, 585), new Vector2(0, 385));
        Pos(rtSubtitulo, new Vector2(0, 345), new Vector2(0, 250));
        Pos(rtRecorde, new Vector2(0, 295), new Vector2(0, 268));

        // Hierarquia piramidal: JOGAR (primaria) maior no topo; APRENDA e
        // TREINAMENTO (secundarias) no meio, tamanho intermediario - em coluna
        // no retrato, lado a lado na paisagem; SAIR (negativa) menor na base,
        // para reduzir toque acidental. Primeiro par = retrato, segundo = paisagem.
        PosTam(rtContinuar, new Vector2(0, 200), new Vector2(820, 120),
            new Vector2(0, 235), new Vector2(500, 90));
        PosTam(rtJogar, new Vector2(0, 55), new Vector2(820, 130),

```

```

        new Vector2(0, 110), new Vector2(540, 140));
PostTam(rtAprender, new Vector2(0, -90), new Vector2(820, 120),
new Vector2(-330, -45), new Vector2(610, 120));
PostTam(rtTreinar, new Vector2(0, -235), new Vector2(820, 120),
new Vector2( 330, -45), new Vector2(610, 120));
PostTam(rtSair, new Vector2(0, -380), new Vector2(820, 120),
new Vector2(0, -190), new Vector2(380, 90));
// Bloco do rodape: a linha de autoria colada na base e a dica logo
// acima dela. Presos a base para continuarem visiveis em qualquer
// proporcao de janela, inclusive quando ela nao corresponde a
// orientacao escolhida. A posicao da dica sai da ALTURA REAL do cartao
// (Ancorar poe o pivo na base, entao a dica fica no topo do cartao mais
// uma folga), e nao de um numero fixo que quebraria se o cartao mudasse.
const float MARGEM_DA_BASE = 24f; // folga entre o cartao e a borda
const float FOLGA_DA_DICA = 18f; // folga entre o cartao e a dica

float topoDosCreditos = MARGEM_DA_BASE;
if (rtCreditos != null)
{
    // Na paisagem a frase cabe numa linha larga e baixa; no retrato ela
    // quebra em duas, entao o cartao fica mais estreito e mais alto.
    rtCreditos.sizeDelta = h ? new Vector2(1700, 92) : new Vector2(1000, 150);
    UIFabrica.Ancorar(rtCreditos, new Vector2(0.5f, 0f), new Vector2(0.5f, 0f));
    rtCreditos.anchoredPosition = new Vector2(0, MARGEM_DA_BASE);
    topoDosCreditos = MARGEM_DA_BASE + rtCreditos.sizeDelta.y;
}
if (rtDicaMenu != null)
{
    UIFabrica.Ancorar(rtDicaMenu, new Vector2(0.5f, 0f), new Vector2(0.5f, 0f));
    rtDicaMenu.anchoredPosition = new Vector2(0, topoDosCreditos + FOLGA_DA_DICA);
}

// Luzes acompanham o título; estrelas se espalham na largura nova
for (int i = 0; i < brilhosTitulo.Count; i++)
{
    Vector2 b = brilhosBase[i];
    brilhosTitulo[i].anchoredPosition = h
        ? new Vector2(b.x * 2.4f, 390f + (b.y - 425f) * 0.30f)
        : b;
}
for (int i = 0; i < estrelasFundo.Count; i++)
    estrelasFundo[i].anchoredPosition = h
        ? new Vector2(estrelasBase[i].x * 1.8f, estrelasBase[i].y * 0.55f)
        : estrelasBase[i];

// — Jogo —
// Cartão da palavra com "slot" à esquerda para o objeto 3D.
//
// Estes elementos sao ancorados a BASE da tela com o pivo no centro,
// entao a altura informada e a distancia da borda de baixo ate o MEIO
// do elemento. Valores negativos jogam o cartao para fora da tela, que
// era o que acontecia na paisagem: so a beirada de cima aparecia.
//
// Na horizontal o cartao encolheu de 1200 para 1000. Com 1200, somado
// aos dois botoes de 390, dava 1980 numa tela de 1920: nao cabiam lado
// a lado e se invadiam.
// Foco central: o painel da palavra sobe e fica em destaque; logo abaixo,
// os dois botoes de acao formam um par centralizado, maiores e proximos
// um do outro - longe dos cantos, para o dedo alcancar sem toque acidental.
if (painelPalavra != null)
{
    PostTam((RectTransform)painelPalavra.transform,
        new Vector2(0, 450), new Vector2(880, 240),
        new Vector2(0, 405), new Vector2(980, 180));

    if (uiControle == null)
        uiControle = painelPalavra.GetComponentInChildren<UIControle>(true);
    if (uiControle != null)
        uiControle.DefinirEspacoDoObjeto(h ? 300f : 260f, h ? 145f : 120f);
}
if (botaoPular != null) // PULAR PALAVRA, a direita do centro
    PostTam((RectTransform)botaoPular.transform,
        new Vector2( 270, 175), new Vector2(500, 178),
        new Vector2( 300, 175), new Vector2(540, 192));
if (botaoPularLetra != null) // PULAR LETRA, a esquerda do centro
    PostTam((RectTransform)botaoPularLetra.transform,
        new Vector2(-270, 175), new Vector2(500, 178),
        new Vector2(-300, 175), new Vector2(540, 192));

larguraDoSlot = h ? 300f : 260f;
PosicionarObjeto3D();
PosicionarBotoesContinuar(jogoPausado); // mantem CONTINUAR/RECOMEÇAR no lugar
AplicarAjustesInspector(h);
if (uiControle != null) uiControle.AjustarTextos(h);
}

// O objeto 3D vive no mundo e o cartão vive na interface. Se a posição do
// objeto for fixa, os dois só coincidem numa proporção de tela específica
// e o objeto "escapa" da caixa em qualquer outra. Aqui a posição é
// calculada a partir do cartão de verdade, então eles andam sempre juntos.
void PosicionarObjeto3D()
{

```

```

if (gerenciador == null || gerenciador.pontoDeExibicao == null) return;
if (painelPalavra == null) return;

var camera = Camera.main;
if (camera == null) return;

var rt = (RectTransform)painelPalavra.transform;
var cantos = new Vector3[4];
rt.GetWorldCorners(cantos); // 0=baixo-esq 1=cima-esq 2=cima-dir 3=baixo-dir

// Centro do espaço reservado na esquerda do cartão
float fatia = Mathf.Clamp01(larguraDoSlot / Mathf.Max(1f, rt.rect.width));
Vector3 meioEsquerdo = Vector3.Lerp(cantos[0], cantos[1], 0.5f);
Vector3 meioDireito = Vector3.Lerp(cantos[3], cantos[2], 0.5f);
Vector3 alvo = Vector3.Lerp(meioEsquerdo, meioDireito, fatia * 0.5f);

// Leva esse ponto para a distância em que o objeto é exibido
Vector3 naTela = camera.WorldToScreenPoint(alvo);
Vector2 o3 = telaHorizontal ? off3dPC : off3dTablet;
float esc = Screen.height / (telaHorizontal ? 1080f : 1920f);
naTela.x += o3.x * esc; naTela.y += o3.y * esc;
gerenciador.pontoDeExibicao.position =
    camera.ScreenToWorldPoint(new Vector3(naTela.x, naTela.y, DISTANCIA_DO_OBJETO));

// Tamanho proporcional à altura do cartão, para caber no espaço
Vector3 topo = camera.ScreenToWorldPoint(new Vector3(naTela.x,
    camera.WorldToScreenPoint(cantos[1]).y, DISTANCIA_DO_OBJETO));
Vector3 baixo = camera.ScreenToWorldPoint(new Vector3(naTela.x,
    camera.WorldToScreenPoint(cantos[0]).y, DISTANCIA_DO_OBJETO));
float alturaNoMundo = Vector3.Distance(topo, baixo);
if (alturaNoMundo > 0.01f)
    gerenciador.tamanhoDoObjeto = alturaNoMundo * 0.78f;
}

// — Ações dos botões —————

// Abre a tela de consulta do alfabeto (criada na primeira vez que e usada)
void AbrirEstudo()
{
    GerenciadorDeAudio.TocarClique();
    if (telaEstudo == null)
        telaEstudo = ModoEstudo.Criar(transform, controlador, FecharEstudo);
    telaMenu.SetActive(false);
    telaEstudo.Abrir(telaHorizontal);
    TrazerLogoAFrente(); AjustarLogo(true);
}

void FecharEstudo()
{
    telaMenu.SetActive(true);
    telaMenu.transform.SetAsLastSibling();
    if (botaoSom != null) botaoSom.transform.SetAsLastSibling();
}

// Retoma o jogo pausado exatamente de onde parou
void Continuar()
{
    GerenciadorDeAudio.TocarClique();
    FecharMenu();
    controlador.MODO_TREINAMENTO = false;
    dicaTreinamento.SetActive(false);
    if (painelPalavra != null) painelPalavra.SetActive(true);
    if (botaoPular != null) botaoPular.SetActive(true);
    if (botaoPularLetra != null) botaoPularLetra.SetActive(true);
    gerenciador.Retomar();
}

void Jogar()
{
    GerenciadorDeAudio.TocarClique();
    FecharMenu();
    controlador.MODO_TREINAMENTO = false;
    dicaTreinamento.SetActive(false);
    if (painelPalavra != null) painelPalavra.SetActive(true);
    if (botaoPular != null) botaoPular.SetActive(true);
    if (botaoPularLetra != null) botaoPularLetra.SetActive(true);
    gerenciador.IniciarJogo();
}

void Treinar()
{
    GerenciadorDeAudio.TocarClique();
    FecharMenu();
    controlador.MODO_TREINAMENTO = true;
    gerenciador.PararJogo();
    if (painelPalavra != null) painelPalavra.SetActive(false);
    if (botaoPular != null) botaoPular.SetActive(false);
    if (botaoPularLetra != null) botaoPularLetra.SetActive(false);
    dicaTreinamento.SetActive(true);
}

void Sair()

```

```

    {
        GerenciadorDeAudio.TocarClique();
    }
    #if UNITY_EDITOR
        UnityEditor.EditorApplication.isPlaying = false;
    #else
        Application.Quit();
    #endif
}

public void AbrirMenuComSom()
{
    GerenciadorDeAudio.TocarClique();
    AbrirMenu();
}

public void AbrirMenu()
{
    menuAberto = true;
    timerFimDeJogo = 0f;

    // Jogo em andamento? PAUSA (não perde pontos/fase/vidas).
    // Fora isso (fim de jogo, treinamento, início), para de verdade.
    bool podeContinuar = gerenciador != null &&
        gerenciador.JogoIniciado && !gerenciador.JogoTerminado;
    jogoPausado = podeContinuar;
    if (podeContinuar) gerenciador.Pausar();
    else gerenciador.PararJogo();
    if (botaoContinuar != null) botaoContinuar.SetActive(podeContinuar);
    if (rotuloJogar != null) rotuloJogar.text = podeContinuar ? "RECOMEÇAR" : "JOGAR";
    // Com o jogo pausado o CONTINUAR ocupa o topo da pilha; o recorde (uma
    // estatística de menu ocioso) sai de cena para não disputar espaço.
    if (rtRecorde != null) rtRecorde.gameObject.SetActive(!podeContinuar);
    PosicionarBotoesContinuar(podeContinuar);

    controlador.MODO_TREINAMENTO = false;

    telaMenu.SetActive(true);
    telaMenu.transform.SetAsLastSibling(); // menu por cima de tudo
    if (painelSenha != null) painelSenha.SetActive(false);
    if (painelOpcoes != null) painelOpcoes.SetActive(false);
    if (telaEstudo != null) telaEstudo.gameObject.SetActive(false);

    // No menu, o som sobe para ficar alinhado com a engrenagem
    if (botaoSom != null)
        ((RectTransform)botaoSom.transform).anchoredPosition = new Vector2(-30, -30);

    // Atualiza o recorde exibido (pode ter acabado de bater um novo!)
    if (textoRecorde != null)
    {
        int recorde = PlayerPrefs.GetInt("recorde", 0);
        textoRecorde.text = recorde > 0 ? "RECORDE: " + recorde : "";
    }
    if (botaoSom != null) botaoSom.transform.SetAsLastSibling(); // som clicável até no menu
    botaoMenuHud.SetActive(false);
    if (headerBar != null) headerBar.SetActive(false);
    if (botaoSom != null) botaoSom.SetActive(true); // som do menu
    dicaTreinamento.SetActive(false);
    if (painelPalavra != null) painelPalavra.SetActive(false);
    if (botaoPular != null) botaoPular.SetActive(false);
    if (botaoPularLetra != null) botaoPularLetra.SetActive(false);
    TrazerLogoAFrente(); AjustarLogo(false);
}

void FecharMenu()
{
    menuAberto = false;
    telaMenu.SetActive(false);
    botaoMenuHud.SetActive(false); // MENU agora faz parte da faixa HUD
    if (botaoSom != null) botaoSom.SetActive(false); // SOM também está na faixa
    if (headerBar != null) headerBar.SetActive(true);
    TrazerLogoAFrente(); AjustarLogo(false);

    // No jogo, o som volta para baixo do botão MENU
    if (botaoSom != null)
        ((RectTransform)botaoSom.transform).anchoredPosition = new Vector2(-30, -135);
}
}

```


Assets/ModoEstudo.cs (675 linhas)

```
using UnityEngine;
using UnityEngine.UI;
using TMPro;
using System.Collections;
using System.Collections.Generic;

// ModoEstudo: tela de consulta do alfabeto em LIBRAS.
//
// Mostra uma letra por vez com o desenho de uma mao reproduzindo o sinal.
// Nas letras com movimento (H, J, K, W, X, Z e C-cedilha) a sequencia
// gravada e reproduzida em laco, para o usuario ver o gesto completo.
//
// A mao e montada a partir dos 21 pontos do banco de sinais do jogo
// (MeuAlfabeto.asset): a palma e os dedos sao desenhados como faixas
// arredondadas com contorno, formando uma silhueta em vez de um traco.
//
// O usuario tambem pode praticar: se ele fizer o sinal da letra exibida
// diante da camera, a tela avisa com som e destaque verde.
public class ModoEstudo : MonoBehaviour
{
    const string ALFABETO = "ABCDEFGHIIJKLMNOPQRSTUVWXYZÇ";

    // A mao e montada em 6 pecas independentes (palma e os cinco dedos).
    // Cada peca tem contorno proprio e as pecas sao reordenadas por
    // profundidade a cada quadro, entao um dedo que passa na frente da palma
    // aparece destacado em vez de se fundir com ela.
    class ParteDaMao
    {
        public RectTransform raiz;
        public int[,] ossos; // ligacoes desenhadas
        public int[] juntas; // pontos que recebem circulo
        public float[] escalaDoOsso; // afinamento ao longo do dedo
        public bool ehPalma = false;
        public readonly List<Image> contorno = new List<Image>();
        public readonly List<Image> pele = new List<Image>();
        public readonly List<Image> detalhes = new List<Image>(); // linhas da palma
    }

    static readonly Color COR_FUNDO = new Color(0.07f, 0.09f, 0.25f, 0.88f);
    static readonly Color COR_PELE = new Color(0.99f, 0.82f, 0.66f, 1f);
    static readonly Color COR_PELE_FUNDO = new Color(0.72f, 0.53f, 0.40f, 1f); // partes ao fundo
    static readonly Color COR_LINHA_PALMA = new Color(0.60f, 0.42f, 0.32f, 0.55f);
    static readonly Color COR_CONTORNO = new Color(0.24f, 0.13f, 0.09f, 1f);
    static readonly Color COR_TITULO = new Color(1f, 0.85f, 0.25f, 1f);
    static readonly Color COR_BOTAO = new Color(0.15f, 0.50f, 0.90f, 1f);
    static readonly Color COR_ACERTO = new Color(0.15f, 0.85f, 0.35f, 1f);

    ControladorCamera controlador;
    ReconhecedorLibras reconhecedor;
    System.Action aoVoltar;

    RectTransform rtTitulo, rtLetra, rtSubtitulo, rtArea, rtContador, rtPratica;
    TextMeshProUGUI letraGrande, subtitulo, contador, aviso, textoPratica;
    Image fundoDaTela, brilhoAcerto, fotoDoSinal;
    RectTransform areaDaMao;

    readonly List<ParteDaMao> partes = new List<ParteDaMao>();

    int indiceLetra = 0;
    bool horizontal = false;

    readonly List<Vector3[]> quadros = new List<Vector3[]>();
    int quadroAtual = 0;
    float tempoDoProximoQuadro = 0f;
    Vector2 centroDoDesenho = Vector2.zero;
    float escalaDoDesenho = 1f;
    float tamanhoAlvo = 430f;
    float grossuraDedo = 34f;
    float grossuraPalma = 60f;

    // Pratica: o usuario reproduz o sinal diante da camera
    bool letraAtualDinamica = false;
    float tempoSinalCerto = 0f;
    float tempoDoProximoAviso = 0f;
    float tempoDaProximaChecagem = 0f;
    bool comemorando = false;

    // Guarda as imagens ja convertidas, para nao refazer a cada troca de letra
    readonly Dictionary<string, Sprite> fotosCarregadas = new Dictionary<string, Sprite>();

    // — Criacao da tela —————

    public static ModoEstudo Criar(Transform canvas, ControladorCamera controlador,
        System.Action aoVoltar)
    {
        var fundo = UIFabrica.CriarImagem(canvas, "TelaEstudo", COR_FUNDO,
            Vector2.zero, Vector2.zero);
        var rt = fundo.rectTransform;
```

```

rt.anchorMin = Vector2.zero;
rt.anchorMax = Vector2.one;
rt.sizeDelta = Vector2.zero;

var tela = fundo.gameObject.AddComponent<ModoEstudo>();
tela.fundoDaTela = fundo;
tela.controlador = controlador;
tela.reconhecedor = controlador.reconhecedor;
tela.aoVoltar = aoVoltar;
tela.Construir();
return tela;
}

void Construir()
{
    var titulo = UIFabrica.CriarTexto(transform, "Titulo", "APRENDA OS SINAIS",
        42f, COR_TITULO, Vector2.zero, new Vector2(900, 60));
    UIFabrica.Ancorar(titulo, new Vector2(0.5f, 1f), new Vector2(0.5f, 1f));
    rtTitulo = titulo.rectTransform;

    letraGrande = UIFabrica.CriarTexto(transform, "Letra", "A",
        200f, Color.white, Vector2.zero, new Vector2(420, 220));
    rtLetra = letraGrande.rectTransform;

    subtitulo = UIFabrica.CriarTexto(transform, "Subtitulo", "",
        32f, new Color(1f, 1f, 1f, 0.85f), Vector2.zero, new Vector2(560, 46), false);
    rtSubtitulo = subtitulo.rectTransform;

    // Area onde a mao e desenhada
    var area = UIFabrica.CriarImagem(transform, "AreaDaMao",
        new Color(1f, 1f, 1f, 0.06f), Vector2.zero, new Vector2(560, 560),
        UIFabrica.Arredondado(), true);
    area.raycastTarget = false;
    areaDaMao = area.rectTransform;
    rtArea = areaDaMao;

    // Brilho verde que pisca quando o usuario acerta o sinal
    brilhoAcerto = UIFabrica.CriarImagem(areaDaMao, "BrilhoAcerto",
        new Color(0.15f, 0.85f, 0.35f, 0f), Vector2.zero, new Vector2(560, 560),
        UIFabrica.Arredondado(), true);
    brilhoAcerto.raycastTarget = false;

    MontarPartesDaMao();

    // Imagem de referencia (opcional). Se existir um arquivo com o nome da
    // letra em Assets/Resources/SinaisLibras, ele e mostrado no lugar do
    // desenho. Basta uma foto da propria mao fazendo o sinal.
    fotoDoSinal = UIFabrica.CriarImagem(areaDaMao, "FotoDoSinal", Color.white,
        Vector2.zero, new Vector2(520, 520));
    fotoDoSinal.preserveAspect = true;
    fotoDoSinal.raycastTarget = false;
    fotoDoSinal.gameObject.SetActive(false);

    // Credito da fonte dos desenhos do alfabeto (pequeno, no rodape)
    var fonte = UIFabrica.CriarTexto(transform, "FonteCredito", "Fonte: libras.com.br (2026)",
        22f, new Color(1f, 1f, 1f, 0.55f), new Vector2(0, 14), new Vector2(700, 34), false);
    UIFabrica.Ancorar(fonte, new Vector2(0.5f, 0f), new Vector2(0.5f, 0f));

    aviso = UIFabrica.CriarTexto(areaDaMao, "Aviso", "",
        34f, new Color(1f, 1f, 1f, 0.8f), Vector2.zero, new Vector2(500, 220), false);

    CriarSeta("SetaEsquerda", new Vector2(0f, 0.5f), 180f, LetraAnterior);
    CriarSeta("SetaDireita", new Vector2(1f, 0.5f), 0f, ProximaLetra);

    textoPratica = UIFabrica.CriarTexto(transform, "Pratica", "",
        34f, new Color(1f, 1f, 1f, 0.85f), Vector2.zero, new Vector2(900, 50), false);
    rtPratica = textoPratica.rectTransform;

    contador = UIFabrica.CriarTexto(transform, "Contador", "",
        30f, new Color(1f, 1f, 1f, 0.7f), Vector2.zero, new Vector2(500, 44), false);
    UIFabrica.Ancorar(contador, new Vector2(0.5f, 0f), new Vector2(0.5f, 0f));
    rtContador = contador.rectTransform;

    var voltar = UIFabrica.CriarBotao(transform, "Voltar", "VOLTAR AO MENU",
        new Color(0.4f, 0.4f, 0.5f, 1f), new Vector2(0, 85), new Vector2(440, 100),
        36f, controlador, Voltar);
    UIFabrica.Ancorar(voltar, new Vector2(0.5f, 0f), new Vector2(0.5f, 0f));

    AplicarLayout(false);
}

void CriarSeta(string nome, Vector2 ancora, float giro,
    UnityEngine.Events.UnityAction acao)
{
    var botao = UIFabrica.CriarBotao(transform, nome, "", COR_BOTA0,
        new Vector2(ancora.x < 0.5f ? 75 : -75, 0), new Vector2(120, 190),
        30f, controlador, acao);
    UIFabrica.Ancorar(botao, ancora, new Vector2(0.5f, 0.5f));

    var icone = UIFabrica.CriarImagem(botao.transform, "Icone", Color.white,
        Vector2.zero, new Vector2(66, 66), UIFabrica.Seta());
    icone.rectTransform.localEulerAngles = new Vector3(0, 0, giro);
}

```

```

        icone.raycastTarget = false;
    }

    // — Montagem das pecas da mao —————

void MontarPartesDaMao()
{
    // Palma: leque do pulso ate os nos dos dedos, com a borda fechada,
    // para o preenchimento nao deixar falhas
    CriarParte("Palma",
        new int[,] { {0,1},{1,5},{5,9},{9,13},{13,17},{17,0},
                     {0,5},{0,9},{0,13} },
        new int[] { 0, 1, 5, 9, 13, 17 },
        new float[] { 1f, 1f, 1f, 1f, 1f, 1f, 1f, 1f, 1f }, true);

    // Dedos: cada segmento afina em direcao a ponta
    float[] afinamento = { 0.92f, 0.82f, 0.72f };
    CriarParte("Polegar", new int[,] { {1,2},{2,3},{3,4} },
        new int[] { 2, 3, 4 }, afinamento, false);
    CriarParte("Indicador", new int[,] { {5,6},{6,7},{7,8} },
        new int[] { 6, 7, 8 }, afinamento, false);
    CriarParte("Medio", new int[,] { {9,10},{10,11},{11,12} },
        new int[] { 10, 11, 12 }, afinamento, false);
    CriarParte("Anelar", new int[,] { {13,14},{14,15},{15,16} },
        new int[] { 14, 15, 16 }, afinamento, false);
    CriarParte("Minimo", new int[,] { {17,18},{18,19},{19,20} },
        new int[] { 18, 19, 20 }, afinamento, false);
}

void CriarParte(string nome, int[,] ossos, int[] juntas,
    float[] escalaDoOsso, bool ehPalma)
{
    var go = new GameObject(nome, typeof(RectTransform));
    go.layer = 5;
    var raiz = go.GetComponent<RectTransform>();
    raiz.SetParent(areaDaMao, false);
    raiz.anchorMin = raiz.anchorMax = new Vector2(0.5f, 0.5f);
    raiz.sizeDelta = Vector2.zero;

    var parte = new ParteDaMao
    {
        raiz = raiz, ossos = ossos, juntas = juntas,
        escalaDoOsso = escalaDoOsso, ehPalma = ehPalma
    };

    // Primeiro TODO o contorno, depois TODA a pele: assim a peca ganha um
    // contorno continuo, sem emendas visiveis por dentro
    for (int i = 0; i < ossos.GetLength(0); i++)
        parte.contorno.Add(CriarForma(raiz, COR_CONTORNO, true));
    for (int i = 0; i < juntas.Length; i++)
        parte.contorno.Add(CriarForma(raiz, COR_CONTORNO, false));
    for (int i = 0; i < ossos.GetLength(0); i++)
        parte.pele.Add(CriarForma(raiz, COR_PELE, true));
    for (int i = 0; i < juntas.Length; i++)
        parte.pele.Add(CriarForma(raiz, COR_PELE, false));

    // Linhas da palma: dao a leitura de que se ve a palma da mao,
    // e nao o dorso (que era como o desenho estava sendo entendido)
    if (ehPalma)
        for (int i = 0; i < 3; i++)
            parte.detalhes.Add(CriarForma(raiz, COR_LINHA_PALMA, true));

    partes.Add(parte);
}

Image CriarForma(Transform pai, Color cor, bool alongada)
{
    var img = UIFabrica.CriarImagem(pai, alongada ? "Osso" : "Junta", cor,
        Vector2.zero, new Vector2(10, 10),
        alongada ? UIFabrica.Arredondado() : UIFabrica.Circulo(), alongada);
    img.raycastTarget = false;
    return img;
}

// — Layout: muda conforme a orientacao da tela —————

public void AplicarLayout(bool telaHorizontal)
{
    horizontal = telaHorizontal;

    if (horizontal)
    {
        // Deitado: texto a esquerda, mao a direita
        rtTitulo.anchoredPosition = new Vector2(0, -120);
        UIFabrica.Ancorar(letraGrande, new Vector2(0.5f, 0.5f), new Vector2(0.5f, 0.5f));
        rtLetra.anchoredPosition = new Vector2(-470, 130);
        UIFabrica.Ancorar(subtitulo, new Vector2(0.5f, 0.5f), new Vector2(0.5f, 0.5f));
        rtSubtitulo.anchoredPosition = new Vector2(-470, -90);
        rtArea.anchoredPosition = new Vector2(430, -25);
        rtArea.sizeDelta = new Vector2(520, 520);
        rtPratica.anchoredPosition = new Vector2(-470, -175);
        rtContador.anchoredPosition = new Vector2(0, 205);
    }
}

```

```

        tamanhoAlvo = 380f;
    }
    else
    {
        // Em pe: tudo empilhado, com espaço entre os blocos
        rtTitulo.anchoredPosition = new Vector2(0, -128);
        UIFabrica.Ancorar(letraGrande, new Vector2(0.5f, 1f), new Vector2(0.5f, 1f));
        rtLetra.anchoredPosition = new Vector2(0, -140);
        UIFabrica.Ancorar(subtitulo, new Vector2(0.5f, 1f), new Vector2(0.5f, 1f));
        rtSubtitulo.anchoredPosition = new Vector2(0, -365);
        rtArea.anchoredPosition = new Vector2(0, -40);
        rtArea.sizeDelta = new Vector2(600, 600);
        UIFabrica.Ancorar(textoPratica, new Vector2(0.5f, 0f), new Vector2(0.5f, 0f));
        rtPratica.anchoredPosition = new Vector2(0, 280);
        rtContador.anchoredPosition = new Vector2(0, 205);
        tamanhoAlvo = 430f;
    }

    if (horizontal)
        UIFabrica.Ancorar(textoPratica, new Vector2(0.5f, 0.5f), new Vector2(0.5f, 0.5f));

    brilhoAcerto.rectTransform.sizeDelta = rtArea.sizeDelta;
    grossuraDedo = tamanhoAlvo * 0.088f;
    grossuraPalma = tamanhoAlvo * 0.125f;

    if (quadros.Count > 0)
    {
        PrepararEscala();
        DesenharQuadro(quadros[quadroAtual]);
    }
}

// — Navegacao —————

public void Abrir(bool telaHorizontal)
{
    gameObject.SetActive(true);
    transform.SetAsLastSibling();
    AplicarLayout(telaHorizontal);
    CarregarLetra();
}

void Voltar()
{
    GerenciadorDeAudio.TocarClique();
    gameObject.SetActive(false);
    if (aoVoltar != null) aoVoltar();
}

void ProximaLetra()
{
    GerenciadorDeAudio.TocarClique();
    indiceLetra = (indiceLetra + 1) % ALFABETO.Length;
    CarregarLetra();
}

void LetraAnterior()
{
    GerenciadorDeAudio.TocarClique();
    indiceLetra = (indiceLetra - 1 + ALFABETO.Length) % ALFABETO.Length;
    CarregarLetra();
}

void Update()
{
    if (Input.GetKeyDown(KeyCode.RightArrow)) ProximaLetra();
    if (Input.GetKeyDown(KeyCode.LeftArrow)) LetraAnterior();

    // Reproduz a sequencia de movimento em laco, com pausa no fim
    // (so quando a mao desenhada esta em uso, e nao a foto)
    bool usandoFoto = fotoDoSinal != null && fotoDoSinal.gameObject.activeSelf;

    if (!usandoFoto && quadros.Count > 1 && Time.unscaledTime >= tempoDoProximoQuadro)
    {
        quadroAtual = (quadroAtual + 1) % quadros.Count;
        tempoDoProximoQuadro = Time.unscaledTime +
            ((quadroAtual == 0) ? 0.8f : 1f / 12f);
        DesenharQuadro(quadros[quadroAtual]);
    }

    VerificarPratica();
}

// — Pratica: confere se o usuario esta fazendo o sinal —————

void VerificarPratica()
{
    if (comemorando || quadros.Count == 0) return;
    if (controlador == null || reconhecedor == null) return;
    if (Time.unscaledTime < tempoDoProximoAviso) return;

    // Compara algumas vezes por segundo. Comparar a cada quadro repetiria

```

```

// um calculo pesado sem ganho nenhum para o usuario.
if (Time.unscaledTime < tempoDaProximaChecagem) return;
tempoDaProximaChecagem = Time.unscaledTime + 0.1f;

if (!controlador.MaoDetectada || controlador.PontosDaMaoAtuais == null)
{
    tempoSinalCerto = 0f;
    textoPratica.text = "Faça o sinal para praticar";
    textoPratica.color = new Color(1f, 1f, 1f, 0.6f);
    return;
}

string letra = ALFABETO[indiceLetra].ToString();
string feita = letraAtualDinamica
    ? reconhecedor.ClassificarSinalDinamico(controlador.JanelaDeMovimento)
    : reconhecedor.ClassificarLetra(controlador.PontosDaMaoAtuais);

if (feita == letra)
{
    if (tempoSinalCerto <= 0f) tempoSinalCerto = Time.unscaledTime;
    textoPratica.text = "quase lá...";
    textoPratica.color = new Color(1f, 1f, 1f, 0.9f);

    if (Time.unscaledTime - tempoSinalCerto >= 0.35f)
        StartCoroutine(Comemorar());
}
else
{
    tempoSinalCerto = 0f;
    textoPratica.text = "Faça o sinal para praticar";
    textoPratica.color = new Color(1f, 1f, 1f, 0.6f);
}
}

IEnumerator Comemorar()
{
    comemorando = true;
    GerenciadorDeAudio.TocarVitoria();

    textoPratica.text = "MUITO BEM! VOCÊ FEZ O SINAL";
    textoPratica.color = COR_ACERTO;
    letraGrande.color = COR_ACERTO;

    // O brilho verde cresce e some atras da mao
    float duracao = 1.4f;
    for (float t = 0f; t < duracao; t += Time.unscaledDeltaTime)
    {
        float p = t / duracao;
        var cor = brilhoAcerto.color;
        cor.a = Mathf.Sin(p * Mathf.PI) * 0.45f;
        brilhoAcerto.color = cor;
        yield return null;
    }

    brilhoAcerto.color = new Color(0.15f, 0.85f, 0.35f, 0f);
    letraGrande.color = Color.white;
    textoPratica.text = "Faça o sinal para praticar";
    textoPratica.color = new Color(1f, 1f, 1f, 0.6f);

    tempoSinalCerto = 0f;
    tempoDoProximoAviso = Time.unscaledTime + 1.2f;
    comemorando = false;
}

// — Carregamento e desenho da mao —————

void CarregarLetra()
{
    string letra = ALFABETO[indiceLetra].ToString();
    letraGrande.text = letra;
    letraGrande.color = Color.white;
    contador.text = (indiceLetra + 1) + " de " + ALFABETO.Length;

    quadros.Clear();
    quadroAtual = 0;
    tempoSinalCerto = 0f;
    tempoDoProximoQuadro = Time.unscaledTime + 0.8f;

    var banco = (reconhecedor != null) ? reconhecedor.bancoDeDados : null;
    letraAtualDinamica = reconhecedor != null && reconhecedor.EhLetraDinamica(letra);

    if (banco != null && banco.sinaisDinamicos != null)
        foreach (var sinal in banco.sinaisDinamicos)
            if (sinal.nome == letra && sinal.quadros != null && sinal.quadros.Count > 1)
            {
                foreach (var q in sinal.quadros) quadros.Add(q.pontos);
                break;
            }

    if (quadros.Count == 0 && banco != null && banco.letrasGravadas != null)
        foreach (var padrao in banco.letrasGravadas)
            if (padrao.nome == letra)

```

```

        {
            quadros.Add(padrao.pontosNormalizados);
            break;
        }

// Imagem de referencia tem prioridade sobre o desenho
bool temFoto = CarregarFoto(letra);
bool temSinal = quadros.Count > 0;

MostrarMao(temSinal && !temFoto);
aviso.gameObject.SetActive(!temSinal && !temFoto);
// A pratica depende das amostras gravadas, nao da imagem
textoPratica.gameObject.SetActive(temSinal);

if (!temSinal && !temFoto)
{
    subtitulo.text = "";
    aviso.text = "Sinal ainda não cadastrado.\n\n" +
                "Grave esta letra no modo\ntreinamento para vê-la aqui.";
    return;
}

subtitulo.text = letraAtualDinamica ? "sinal com movimento" : "sinal parado";
textoPratica.text = "Faça o sinal para praticar";
textoPratica.color = new Color(1f, 1f, 1f, 0.6f);

if (temSinal && !temFoto)
{
    PrepararEscala();
    DesenharLayout(quadros[0]);
}
}

void MostrarMao(bool visivel)
{
    foreach (var parte in partes) parte.raiz.gameObject.SetActive(visivel);
}

// Procura uma imagem da letra em Assets/Resources/SinaisLibras
bool CarregarFoto(string letra)
{
    if (fotoDoSinal == null) return false;

    Sprite figura;
    if (!fotosCarregadas.TryGetValue(letra, out figura))
    {
        string arquivo = (letra == "Ç") ? "CEDILHA" : letra;
        var textura = Resources.Load<Texture2D>("SinaisLibras/" + arquivo);
        if (textura != null) textura.filterMode = FilterMode.Bilinear;
        if (textura == null && letra == "Ç")
            textura = Resources.Load<Texture2D>("SinaisLibras/Ç");

        figura = (textura == null) ? null : Sprite.Create(textura,
            new Rect(0, 0, textura.width, textura.height), new Vector2(0.5f, 0.5f));
        fotosCarregadas[letra] = figura; // guarda ate o "sem imagem"
    }

    if (figura == null)
    {
        fotoDoSinal.gameObject.SetActive(false);
        return false;
    }

    fotoDoSinal.sprite = figura;
    fotoDoSinal.rectTransform.sizeDelta = rtArea.sizeDelta * 0.92f;
    fotoDoSinal.gameObject.SetActive(true);
    fotoDoSinal.transform.SetAsLastSibling();
    return true;
}

// Escala unica para TODOS os quadros, senao a mao mudaria de tamanho
// durante a animacao
void PrepararEscala()
{
    float minX = float.MaxValue, maxX = float.MinValue;
    float minY = float.MaxValue, maxY = float.MinValue;

    foreach (var quadro in quadros)
    {
        for (int i = 0; i < 21 && i < quadro.Length; i++)
        {
            if (quadro[i].x < minX) minX = quadro[i].x;
            if (quadro[i].x > maxX) maxX = quadro[i].x;
            if (quadro[i].y < minY) minY = quadro[i].y;
            if (quadro[i].y > maxY) maxY = quadro[i].y;
        }

        float maior = Mathf.Max(maxX - minX, maxY - minY);
        escalaDoDesenho = (maior > 0.0001f) ? tamanhoAlvo / maior : 1f;
        centroDoDesenho = new Vector2((minX + maxX) * 0.5f, (minY + maxY) * 0.5f);
    }
}

void DesenharLayout(Vector3[] quadro)

```

```

{
    if (quadro == null || quadro.Length < 21) return;

    // Profundidade da cena: serve para ordenar as pecas e escurecer o que
    // esta mais longe. Em z menor a parte esta MAIS PERTO da camera.
    float zMin = float.MaxValue, zMax = float.MinValue;
    for (int i = 0; i < 21; i++)
    {
        if (quadro[i].z < zMin) zMin = quadro[i].z;
        if (quadro[i].z > zMax) zMax = quadro[i].z;
    }
    bool temProfundidade = (zMax - zMin) > 0.0001f;

    // Ordena as pecas: o que esta atras e desenhado primeiro
    var ordem = new List<KeyValuePair<float, ParteDaMao>>();
    for (int i = 0; i < partes.Count; i++)
    {
        float z = ProfundidadeDaParte(partes[i], quadro);
        // Sem eixo z (amostras antigas): mantem a ordem natural da mao
        if (!temProfundidade) z = -i;
        ordem.Add(new KeyValuePair<float, ParteDaMao>(z, partes[i]));
    }
    ordem.Sort((a, b) => b.Key.CompareTo(a.Key));

    for (int i = 0; i < ordem.Count; i++)
    {
        var parte = ordem[i].Value;
        parte.raiz.SetSiblingIndex(i);
        // O brilho de acerto fica atras da mao; o aviso, na frente
        brilhoAcerto.transform.SetAsFirstSibling();
        aviso.transform.SetAsLastSibling();

        // Quanto mais longe, mais escura fica a pele (nocaio de volume)
        float tom = temProfundidade
            ? Mathf.InverseLerp(zMin, zMax, ordem[i].Key) : 0.25f;
        Color corDaPele = Color.Lerp(COR_PELI, COR_PELI_FUNDO, tom * 0.75f);
        DesenharaParte(parte, quadro, corDaPele);
    }
}

float ProfundidadeDaParte(ParteDaMao parte, Vector3[] quadro)
{
    float soma = 0f;
    for (int i = 0; i < parte.juntas.Length; i++) soma += quadro[parte.juntas[i]].z;
    return soma / Mathf.Max(1, parte.juntas.Length);
}

void DesenharaParte(ParteDaMao parte, Vector3[] quadro, Color corDaPele)
{
    bool ehPalma = parte.ehPalma;
    float baseDaParte = ehPalma ? grossuraPalma : grossuraDedo;
    float bordaExtra = grossuraDedo * 0.40f; // espessura do contorno

    int nOssos = parte.ossos.GetLength(0);
    for (int i = 0; i < nOssos; i++)
    {
        float g = baseDaParte * parte.escalaDoOsso[Mathf.Min(i, parte.escalaDoOsso.Length - 1)];
        Vector3 de = quadro[parte.ossos[i], 0];
        Vector3 ate = quadro[parte.ossos[i], 1];
        PosicionarFaixa(parte.contorno[i].rectTransform, de, ate, g + bordaExtra);
        PosicionarFaixa(parte.pele[i].rectTransform, de, ate, g);
        parte.pele[i].color = corDaPele;
    }

    for (int i = 0; i < parte.juntas.Length; i++)
    {
        int indice = parte.juntas[i];
        float g = ehPalma
            ? grossuraPalma
            : grossuraDedo * parte.escalaDoOsso[Mathf.Min(i, parte.escalaDoOsso.Length - 1)];

        var contorno = parte.contorno[nOssos + i].rectTransform;
        var pele = parte.pele[nOssos + i].rectTransform;
        Vector2 pos = ParaTela(quadro[indice]);

        contorno.anchoredPosition = pos;
        contorno.sizeDelta = new Vector2(g + bordaExtra, g + bordaExtra);
        pele.anchoredPosition = pos;
        pele.sizeDelta = new Vector2(g, g);
        parte.pele[nOssos + i].color = corDaPele;
    }

    // Tres linhas leves dentro da palma (as pregas da mao)
    if (ehPalma && parte.detalhes.Count >= 3)
    {
        float fina = grossuraDedo * 0.15f;
        DesenharaLinhaDaPalma(parte.detalhes[0], quadro, 0, 5, 0.45f, 0, 13, 0.55f, fina);
        DesenharaLinhaDaPalma(parte.detalhes[1], quadro, 0, 5, 0.62f, 0, 17, 0.50f, fina);
        DesenharaLinhaDaPalma(parte.detalhes[2], quadro, 0, 9, 0.30f, 0, 17, 0.30f, fina);
    }
}

```

```

// Traca uma linha entre dois pontos "no meio do caminho" de dois ossos
void DesenharLinhaDaPalma(Image linha, Vector3[] quadro,
                           int a1, int a2, float fA, int b1, int b2, float fB,
                           float grossura)
{
    Vector2 a = Vector2.Lerp(ParaTela(quadro[a1]), ParaTela(quadro[a2]), fA);
    Vector2 b = Vector2.Lerp(ParaTela(quadro[b1]), ParaTela(quadro[b2]), fB);
    Vector2 direcao = b - a;

    var rt = linha.rectTransform;
    rt.anchoredPosition = (a + b) * 0.5f;
    rt.sizeDelta = new Vector2(direcao.magnitude, grossura);
    rt.localEulerAngles = new Vector3(0, 0,
        Mathf.Atan2(direcao.y, direcao.x) * Mathf.Rad2Deg);
}

void PosicionarFaixa(RectTransform faixa, Vector3 de, Vector3 ate, float grossura)
{
    Vector2 a = ParaTela(de), b = ParaTela(ate);
    Vector2 direcao = b - a;

    faixa.anchoredPosition = (a + b) * 0.5f;
    faixa.sizeDelta = new Vector2(direcao.magnitude + grossura, grossura);
    faixa.localEulerAngles = new Vector3(0, 0,
        Mathf.Atan2(direcao.y, direcao.x) * Mathf.Rad2Deg);
}

// Espelha o X para a mao aparecer como o usuario ve a propria mao na tela
Vector2 ParaTela(Vector3 ponto)
{
    return new Vector2(-(ponto.x - centroDoDesenho.x),
        (ponto.y - centroDoDesenho.y)) * escalaDoDesenho;
}
}

```


Assets/Plugins/Android/RastreadorMediaPipe.java (187 linhas)

```
package br.edu.ifto.ditadoestrelado;

import android.content.Context;
import android.graphics.Bitmap;

import com.google.mediapipe.framework.image.BitmapImageBuilder;
import com.google.mediapipe.framework.image.MPImage;
import com.google.mediapipe.tasks.components.containers.NormalizedLandmark;
import com.google.mediapipe.tasks.core.BaseOptions;
import com.google.mediapipe.tasks.core.Delegate;
import com.google.mediapipe.tasks.vision.core.RunningMode;
import com.google.mediapipe.tasks.vision.handlandmarker.HandLandmarker;
import com.google.mediapipe.tasks.vision.handlandmarker.HandLandmarkerResult;

import java.io.File;
import java.io.FileInputStream;
import java.nio.ByteBuffer;
import java.util.List;

/**
 * Ponte entre o aplicativo e o MediaPipe do Android.
 *
 * E o mesmo motor de rastreamento de maos que o computador usa por fora,
 * pelo Python, aqui rodando dentro do proprio aplicativo. O Unity manda a
 * imagem ja endireitada e recebe de volta os 21 pontos da mao.
 *
 * O modo LIVE_STREAM trabalha em segundo plano: o envio do quadro nao trava
 * o jogo e o resultado fica guardado ate alguem vir buscar.
 */
public class RastreadorMediaPipe {

    private static final int PONTOS = 21;
    private static final int VALORES = PONTOS * 3;

    private HandLandmarker detector;

    // Tres imagens em rodizio: o MediaPipe ainda pode estar lendo a anterior
    // quando o quadro seguinte chega
    private final Bitmap[] quadros = new Bitmap[3];
    private int proximoQuadro = 0;
    private int larguraQuadro = 0, alturaQuadro = 0;
    private long relógio = 0;

    private final Object trava = new Object();
    private final float[] leitura = new float[VALORES];
    private final float[] entregue = new float[2 + VALORES];
    private float confianca = 0f;
    private int versao = 0;

    private volatile String ultimoErro = "";

    /** Carrega o modelo e prepara o detector. Devolve true se deu certo. */
    public boolean iniciar(Context contexto, String caminhoDoModelo) {
        try {
            ByteBuffer modelo = lerModelo(caminhoDoModelo);

            BaseOptions base = BaseOptions.builder()
                .setDelegate(Delegate.CPU)
                .setModelAssetBuffer(modelo)
                .build();

            HandLandmarker.HandLandmarkerOptions opcoes =
                HandLandmarker.HandLandmarkerOptions.builder()
                    .setBaseOptions(base)
                    .setRunningMode(RunningMode.LIVE_STREAM)
                    .setNumHands(1)
                    .setMinHandDetectionConfidence(0.5f)
                    .setMinHandPresenceConfidence(0.5f)
                    .setMinTrackingConfidence(0.5f)
                    .setResultListener(this::guardar)
                    .setErrorListener(e -> ultimoErro = String.valueOf(e.getMessage()))
                    .build();

            detector = HandLandmarker.createFromOptions(contexto, opcoes);
            return detector != null;
        } catch (Throwable e) {
            ultimoErro = e.getClass().getSimpleName() + ": " + e.getMessage();
            detector = null;
            return false;
        }
    }

    /** O MediaPipe exige o modelo num buffer direto, fora da memoria do Java. */
    private ByteBuffer lerModelo(String caminho) throws Exception {
        File arquivo = new File(caminho);
        byte[] dados = new byte[(int) arquivo.length()];
        FileInputStream fluxo = new FileInputStream(arquivo);
        try {
```

```

        int lidos = 0;
        while (lidos < dados.length) {
            int n = fluxo.read(dados, lidos, dados.length - lidos);
            if (n < 0) break;
            lidos += n;
        }
    } finally {
        fluxo.close();
    }
    ByteBuffer buffer = ByteBuffer.allocateDirect(dados.length);
    buffer.put(dados);
    buffer.rewind();
    return buffer;
}

/** Recebe um quadro do Unity (bytes RGBA, linha de cima primeiro). */
public void enviarQuadro(byte[] pixels, int largura, int altura) {
    if (detector == null) return;
    try {
        if (quadros[0] == null || larguraQuadro != largura || alturaQuadro != altura) {
            for (int i = 0; i < quadros.length; i++)
                quadros[i] = Bitmap.createBitmap(largura, altura, Bitmap.Config.ARGB_8888);
            larguraQuadro = largura;
            alturaQuadro = altura;
            proximoQuadro = 0;
        }

        Bitmap quadro = quadros[proximoQuadro];
        proximoQuadro = (proximoQuadro + 1) % quadros.length;

        quadro.copyPixelsFromBuffer(ByteBuffer.wrap(pixels));
        MPMImage imagem = new BitmapImageBuilder(quadro).build();

        relógio += 33; // o modo ao vivo exige tempos sempre crescentes
        detector.detectAsync(imagem, relógio);
    } catch (Throwable e) {
        ultimoErro = e.getClass().getSimpleName() + ": " + e.getMessage();
    }
}

/** Chamado pelo MediaPipe numa linha de execucao propria. */
private void guardar(HandLandmarkerResult resultado, MPMImage entrada) {
    synchronized (trava) {
        versao++;

        List<List<NormalizedLandmark>> maos = resultado.landmarks();
        if (maos == null || maos.isEmpty()) {
            confianca = 0f;
            return;
        }

        List<NormalizedLandmark> mao = maos.get(0);
        int total = Math.min(PONTOS, mao.size());
        for (int i = 0; i < total; i++) {
            NormalizedLandmark p = mao.get(i);
            leitura[i * 3] = p.x();
            leitura[i * 3 + 1] = 1f - p.y(); // o Unity conta o Y de baixo para cima
            leitura[i * 3 + 2] = p.z();
        }

        confianca = 1f;
        try {
            confianca = resultado.handednesses().get(0).get(0).score();
        } catch (Throwable ignorado) {
            // sem esse dado, vale a presenca da mao
        }
    }
}

/**
 * Devolve tudo de uma vez para poupar travessias entre C# e Java:
 * [0] contador de versao, [1] confianca, [2..] os 21 pontos (x, y, z).
 */
public float[] resultado() {
    synchronized (trava) {
        entregue[0] = versao;
        entregue[1] = confianca;
        System.arraycopy(leitura, 0, entregue, 2, VALORES);
        return entregue;
    }
}

public String erro() {
    return ultimoErro;
}

public void encerrar() {
    HandLandmarker atual = detector;
    detector = null;
    if (atual != null) {
        try {
            atual.close();
        }
    }
}

```

```
        } catch (Throwable ignorado) {  
            // encerrando de qualquer forma  
        }  
    }  
}
```

Assets/RastreadorExterno.cs (122 linhas)

```
using UnityEngine;
using System;
using System.Net;
using System.Net.Sockets;
using System.Threading;

// RastreadorExterno: recebe os pontos da mão e o vídeo enviados pelo script
// Python (RastreadorPython/rastreador_maos.py) por UDP em 127.0.0.1.
// É comunicação interna do próprio PC - funciona 100% offline.
//
// O ControladorCamera consulta este componente: se o Python estiver rodando,
// usa ele (MediaPipe completo); senão, usa o rastreador interno antigo.
public class RastreadorExterno : MonoBehaviour
{
    [Header("Portas (iguais as do script Python)")]
    public int portaLandmarks = 5052;
    public int portaVideo = 5053;

    [Header("Sem pacotes por este tempo = rastreador desligado")]
    public float tempoMaximoSemSinal = 1.5f;

    // — O que o resto do jogo lê —————
    public bool Ativo => Time.realtimeSinceStartup - tempoUltimoPacote < tempoMaximoSemSinal;
    public float Score { get; private set; }
    public Vector3[] Pontos { get; private set; } = new Vector3[21];
    public Texture2D Video { get; private set; }

    // — Recepção em segundo plano —————
    private UdpClient udpLandmarks;
    private UdpClient udpVideo;
    private Thread threadLandmarks;
    private Thread threadVideo;
    private readonly object trava = new object();
    private byte[] pacoteLandmarks;
    private byte[] pacoteVideo;
    private bool novoLandmarks;
    private bool novoVideo;
    private volatile bool rodando;
    private float tempoUltimoPacote = -999f;

    void Awake()
    {
        rodando = true;
        try
        {
            udpLandmarks = new UdpClient(portaLandmarks);
            udpVideo = new UdpClient(portaVideo);
        }
        catch (Exception e)
        {
            Debug.LogWarning("RastreadorExterno: nao consegui abrir as portas UDP (" + e.Message + ")");
            enabled = false;
            return;
        }

        // Threads de fundo: ficam bloqueadas esperando pacotes, sem travar o jogo
        threadLandmarks = new Thread(() => LacoDeRecepcao(udpLandmarks, true)) { IsBackground = true };
        threadVideo = new Thread(() => LacoDeRecepcao(udpVideo, false)) { IsBackground = true };
        threadLandmarks.Start();
        threadVideo.Start();
    }

    void LacoDeRecepcao(UdpClient udp, bool ehLandmarks)
    {
        IPEndPoint origem = new IPEndPoint(IPAddress.Any, 0);
        while (rodando)
        {
            try
            {
                byte[] dados = udp.Receive(ref origem);
                lock (trava)
                {
                    if (ehLandmarks) { pacoteLandmarks = dados; novoLandmarks = true; }
                    else { pacoteVideo = dados; novoVideo = true; }
                }
            }
            catch { /* socket fechado ao sair - normal */ }
        }
    }

    void Update()
    {
        // Copia os pacotes mais recentes (com trava, pois vêm de outra thread)
        byte[] lm = null, vid = null;
        lock (trava)
        {
            if (novoLandmarks) { lm = pacoteLandmarks; novoLandmarks = false; }
            if (novoVideo) { vid = pacoteVideo; novoVideo = false; }
        }
    }
}
```

```

    }

    // Pacote de pontos: byte 'M' (0x4D) + confiança + 21 × (x,y,z) float32
    if (lm != null && lm.Length >= 1 + 4 + 21 * 12 && lm[0] == 0x4D)
    {
        Score = BitConverter.ToSingle(lm, 1);
        for (int i = 0; i < 21; i++)
        {
            int b = 5 + i * 12;
            Pontos[i] = new Vector3(
                BitConverter.ToSingle(lm, b),
                BitConverter.ToSingle(lm, b + 4),
                BitConverter.ToSingle(lm, b + 8));
        }
        tempoUltimoPacote = Time.realtimeSinceStartup;
    }

    // Pacote de vídeo: bytes de um JPEG
    if (vid != null)
    {
        if (Video == null)
            Video = new Texture2D(2, 2, TextureFormat.RGB24, false);
        Video.LoadImage(vid); // decodifica o JPEG e atualiza a textura
    }
}

void OnDestroy()
{
    rodando = false;
    udpLandmarks?.Close();
    udpVideo?.Close();
}
}

```

Assets/RastreadorMediaPipeAndroid.cs (162 linhas)

```
using System.Collections;
using UnityEngine;
using UnityEngine.Rendering;

// MediaPipe nativo do celular.
//
// No computador o rastreamento bom vem do MediaPipe rodando por fora, no
// Python. Esta classe traz o MESMO motor para dentro do aplicativo Android:
// a biblioteca oficial do Google (tasks-vision 0.10.35, a mesma versao do
// Python) com o modelo hand_landmarker.
//
// O aplicativo entrega a imagem inteira, ja endireitada, e o MediaPipe cuida
// sozinho de achar a palma, recortar, girar e ler os 21 pontos - o trabalho
// que antes era feito na mao aqui dentro, e mal.
//
// Os pontos voltam normalizados no quadro inteiro, com o Y contado de baixo
// para cima: exatamente a mesma convencao do rastreador do computador.
public class RastreadorMediaPipeAndroid
{
    public const int PONTOS = 21;

    const string ARQUIVO_DO_MODELO = "hand_landmarker.task";
    const int LADO_MAIOR = 512; // resolucao enviada ao MediaPipe

    public bool Pronto { get; private set; }
    public float Score { get; private set; }
    public string Erro { get; private set; } = "";
    public Vector3[] Pontos { get; private set; } = new Vector3[PONTOS];

#if UNITY_ANDROID && !UNITY_EDITOR
    AndroidJavaObject ponte;
    RenderTexture reduzida;
    byte[] pixels;
    bool lendoQuadro = false;
    int ultimaVersao = -1;

    public IEnumerator Iniciar()
    {
        // O MediaPipe le o modelo de um arquivo comum. Dentro do APK ele esta
        // compactado, entao a primeira execucao copia para a pasta do aplicativo.
        string destino = System.IO.Path.Combine(Application.persistentDataPath, ARQUIVO_DO_MODELO);
        bool precisaCopiar = !System.IO.File.Exists(destino) ||
            new System.IO.FileInfo(destino).Length < 1000000;

        if (precisaCopiar)
        {
            string origem = Application.streamingAssetsPath + "/" + ARQUIVO_DO_MODELO;
            using (var pedido = UnityEngine.Networking.UnityWebRequest.Get(origem))
            {
                yield return pedido.SendWebRequest();
                if (pedido.result != UnityEngine.Networking.UnityWebRequest.Result.Success)
                {
                    Erro = "modelo nao encontrado: " + pedido.error;
                    yield break;
                }
                System.IO.File.WriteAllBytes(destino, pedido.downloadHandler.data);
            }
        }

        try
        {
            var jogador = new AndroidJavaClass("com.unity3d.player.UnityPlayer");
            var atividade = jogador.GetStatic<AndroidJavaObject>("currentActivity");

            ponte = new AndroidJavaObject("br.edu.iftoditadostrelado.RastreadorMediaPipe");
            Pronto = ponte.Call<bool>("iniciar", atividade, destino);
            if (!Pronto) Erro = ponte.Call<string>("erro");
        }
        catch (System.Exception e)
        {
            Erro = e.Message;
            Pronto = false;
        }
    }

    // Manda um quadro para analise. Nao trava o jogo: a leitura da placa de
    // video volta depois, e o MediaPipe trabalha em segundo plano.
    public void Enviar(Texture origem)
    {
        if (!Pronto || origem == null || lendoQuadro) return;

        int largura, altura;
        if (origem.width >= origem.height)
        {
            largura = LADO_MAIOR;
            altura = Mathf.RoundToInt(LADO_MAIOR * (float)origem.height / origem.width);
        }
        else
```

```

{
    altura = LADO_MAIOR;
    largura = Mathf.RoundToInt(LADO_MAIOR * (float)origem.width / origem.height);
}
largura = Mathf.Max(64, largura - largura % 4);
altura = Mathf.Max(64, altura - altura % 4);

if (reduzida == null || reduzida.width != largura || reduzida.height != altura)
{
    if (reduzida != null) reduzida.Release();
    reduzida = new RenderTexture(largura, altura, 0, RenderTextureFormat.ARGB32);
    reduzida.Create();
    pixels = new byte[largura * altura * 4];
}

// Espelha na vertical: o Unity guarda a imagem de baixo para cima e o
// Android le de cima para baixo
Graphics.Blit(origem, reduzida, new Vector2(1f, -1f), new Vector2(0f, 1f));

lendoQuadro = true;
AsyncGPUReadback.Request(reduzida, 0, TextureFormat.RGBA32, Receber);
}

void Receber(AsyncGPUReadbackRequest pedido)
{
    lendoQuadro = false;
    if (!Pronto || pedido.hasError || pixels == null) return;

    var dados = pedido.GetData<byte>();
    if (dados.Length != pixels.Length) return; // o tamanho mudou no meio do caminho
    dados.CopyTo(pixels);

    ponte.Call("enviarQuadro", pixels, reduzida.width, reduzida.height);
}

// Busca o ultimo resultado pronto. Devolve true quando ha novidade.
public bool Atualizar()
{
    if (!Pronto) return false;

    float[] resposta = ponte.Call<float[]>("resultado");
    if (resposta == null || resposta.Length < 2 + PONTOS * 3) return false;

    int versao = (int)resposta[0];
    if (versao == ultimaVersao) return false;
    ultimaVersao = versao;

    Score = resposta[1];
    for (int i = 0; i < PONTOS; i++)
        Pontos[i] = new Vector3(resposta[2 + i * 3],
                                resposta[3 + i * 3],
                                resposta[4 + i * 3]);

    return true;
}

public void Encerrar()
{
    Pronto = false;
    if (ponte != null)
    {
        try { ponte.Call("encerrar"); } catch (System.Exception) { }
        ponte = null;
    }
    if (reduzida != null) { reduzida.Release(); reduzida = null; }
}
#else
// No computador esta classe nao entra em acao: quem rastreia e o Python.
public IEnumerator Iniciar() { yield break; }
public void Enviar(Texture t) { }
public bool Atualizar() { return false; }
public void Encerrar() { }
#endif
}

```

Assets/ReconhecedorLibras.cs (559 linhas)

```
using UnityEngine;
using System.Collections.Generic;
#if UNITY_EDITOR
using UnityEditor;
#endif

public class ReconhecedorLibras : MonoBehaviour
{
    [Header("Conecte o 'MeuAlfabeto' aqui!")]
    public AlfabetoData bancoDeDados;

    [Header("Aceitacao do sinal")]
    // Limite de seguranca: acima disso a mao nao parece com NADA do banco.
    // Nao e mais o criterio principal - so evita aceitar uma mao qualquer.
    public float limiteDeSeguranca = 12f;

    // Criterio principal, e RELATIVO: a letra vencedora precisa estar este
    // tanto mais perto do que a segunda colocada. Como compara duas distancias
    // entre si, aguenta o dia ruim - quando a leitura piora, todas as
    // distancias sobem juntas e a comparacao continua valendo. O limite fixo
    // antigo simplesmente recusava tudo nessa situacao.
    [Range(0.5f, 1f)] public float vantagemNecessaria = 0.95f;

    // Abaixo desta distancia o sinal e obvio e o jogo aceita mais rapido
    public float distanciaDeCerteza = 3f;

    [Header("Giro da mao")]
    // Endireita a palma antes de comparar, ate este limite. Absorve a
    // inclinacao natural de quem sinaliza sem confundir letras que se
    // distinguem justamente PELA orientacao: D e G tem quase a mesma forma,
    // giradas cerca de 90 graus uma da outra, e por isso o giro e limitado.
    [Range(0f, 90f)] public float giroMaximoCorrigido = 40f;

    [Header("Classificacao kNN")]
    [Range(1, 9)] public int vizinhosK = 5; // quantas amostras próximas votam
    [Range(0f, 3f)] public float pesoDosAngulos = 0.25f; // importância dos ângulos vs posições
    [Range(0f, 3f)] public float pesoDosContatos = 0.5f; // importância do que encosta em quê

    // — Compatibilidade do celular (nao afeta o computador) —
    //
    // O banco de sinais foi gravado no computador, com o rastreador completo:
    // quadro 4:3 e com profundidade. No celular o quadro fica em pe e o
    // rastreador nao fornece profundidade, entao os pontos precisam ser
    // levados ao mesmo formato do banco antes de comparar.
    //
    // Em qualquer plataforma que nao seja o celular, os metodos abaixo
    // devolvem o ponto sem tocar em nada: o computador roda exatamente o
    // mesmo caminho de antes desta compatibilizacao existir.
    const float ASPECTO_DO_BANCO = 4f / 3f;

    [HideInInspector] public float aspectoDaCamera = ASPECTO_DO_BANCO;
    [HideInInspector] public bool temProfundidade = true;

    Vector3 DaCamera(Vector3 p)
    {
        #if UNITY_ANDROID && !UNITY_EDITOR
            // A profundidade tem as mesmas unidades do eixo X, entao recebe o
            // mesmo fator. Sem isso ela ficaria com peso exagerado e passaria a
            // atrapalhar a comparacao em vez de ajudar.
            float fator = aspectoDaCamera / ASPECTO_DO_BANCO;
            return new Vector3(p.x * fator, p.y, temProfundidade ? p.z * fator : 0f);
        #else
            return p;
        #endif
    }

    Vector3 DoBanco(Vector3 p)
    {
        #if UNITY_ANDROID && !UNITY_EDITOR
            return temProfundidade ? p : new Vector3(p.x, p.y, 0f);
        #else
            return p;
        #endif
    }

    [Header("Letras dinamicas (gravadas como MOVIMENTO, nao como foto)")]
    // (campo renomeado para o Unity aplicar a lista nova - W e Ç também se movem)
    public string[] letrasComMovimento = { "H", "J", "K", "W", "X", "Z", "Ç" };
    // Mesmo criterio das letras paradas: limite de seguranca largo + a folga
    // para a rival e que decide (ver Aceita)
    public float limiteDinamicoDeSeguranca = 9f;

    public bool EhLetraDinamica(string letra)
    {
        foreach (var l in letrasComMovimento)
            if (l == letra) return true;
        return false;
    }
}
```



```

// — Ângulos das articulações —————
// Cada trio (a, b, c) mede o ângulo NO ponto b entre os vetores (a-b) e (c-b).
// Ângulo não muda quando a mão gira ou muda de tamanho -> complementa as posições.
private static readonly int[,] TRIOS =
{
    // Polegar
    {0,1,2}, {1,2,3}, {2,3,4},
    // Indicador
    {0,5,6}, {5,6,7}, {6,7,8},
    // Médio
    {0,9,10}, {9,10,11}, {10,11,12},
    // Anelar
    {0,13,14}, {13,14,15}, {14,15,16},
    // Mínimo
    {0,17,18}, {17,18,19}, {18,19,20},
};

// Abertura entre dedos vizinhos: ângulo entre os vetores pulso->base de cada dedo
private static readonly int[,] PARES_ABERTURA =
{
    {1,5}, {5,9}, {9,13}, {13,17}
};

// Contatos: distâncias entre pontas de dedos e entre elas e o polegar.
// Boa parte das letras se distingue justamente por ISSO - o que encosta em
// quê. C e O têm a mesma forma geral: no O o polegar fecha o círculo, no C
// fica um vão. Medido junto com as outras 21 posições, esse detalhe se
// dilui; medido à parte, ele pesa no resultado.
private static readonly int[,] PARES_CONTATO =
{
    // Polegar até cada ponta de dedo (fecha o círculo? encosta?)
    {4,8}, {4,12}, {4,16}, {4,20},
    // Pontas vizinhas (dedos juntos ou separados: U, V, R)
    {8,12}, {12,16}, {16,20},
    // Polegar até a base de cada dedo (polegar por dentro ou por fora: A, S, M, N)
    {4,5}, {4,9}, {4,13}, {4,17},
    // Ponta até a própria base (dedo esticado ou dobrado)
    {8,5}, {12,9}, {16,13}, {20,17},
};

// Recebe as posições JÁ normalizadas pelo tamanho da mão, então as
// distâncias saem na mesma escala para mão grande ou pequena, perto ou longe
static float[] ExtrairContatos(Vector3[] normalizados)
{
    int n = PARES_CONTATO.GetLength(0);
    var contatos = new float[n];
    for (int i = 0; i < n; i++)
        contatos[i] = Vector3.Distance(normalizados[PARES_CONTATO[i, 0]],
                                       normalizados[PARES_CONTATO[i, 1]]);
    return contatos;
}

// Extrai todos os ângulos (em radianos) de uma mão
static float[] ExtrairAngulos(Vector3[] p)
{
    int nTrios = TRIOS.GetLength(0);
    int nPares = PARES_ABERTURA.GetLength(0);
    float[] angulos = new float[nTrios + nPares];

    for (int i = 0; i < nTrios; i++)
    {
        Vector3 v1 = p[TRIOS[i, 0]] - p[TRIOS[i, 1]];
        Vector3 v2 = p[TRIOS[i, 2]] - p[TRIOS[i, 1]];
        angulos[i] = Vector3.Angle(v1, v2) * Mathf.Deg2Rad;
    }
    for (int i = 0; i < nPares; i++)
    {
        Vector3 v1 = p[PARES_ABERTURA[i, 0]] - p[0];
        Vector3 v2 = p[PARES_ABERTURA[i, 1]] - p[0];
        angulos[nTrios + i] = Vector3.Angle(v1, v2) * Mathf.Deg2Rad;
    }
    return angulos;
}

// Distância combinada: formato (posições) + curvatura dos dedos (ângulos)
// + contatos entre dedos
float DistanciaEntre(Vector3[] posA, float[] angA, float[] conA,
                    Vector3[] posB, float[] angB, float[] conB)
{
    float distPosicoes = 0f;
    for (int i = 0; i < 21; i++)
        distPosicoes += Vector3.Distance(posA[i], posB[i]);

    float distAngulos = 0f;
    for (int i = 0; i < angA.Length; i++)
        distAngulos += Mathf.Abs(angA[i] - angB[i]);

    float distContatos = 0f;
    for (int i = 0; i < conA.Length; i++)
        distContatos += Mathf.Abs(conA[i] - conB[i]);
}

```

```

        return distPosicoes + pesoDosAngulos * distAngulos + pesoDosContatos * distContatos;
    }

    [Header("Liga o log 'Mais parecido: X (distancia Y)' no Console")]
    public bool mostrarDebug = false;
    private float tempoUltimoDebug = 0f;

    // Distância da última classificação - quanto MENOR, mais parecido o sinal
    // está com o gravado (o ControladorCamera usa para aceitar mais rápido)
    public float UltimaDistancia { get; private set; } = float.MaxValue;

    // Letra mais parecida da ultima comparacao, mesmo quando recusada.
    // Serve para descobrir, no proprio aparelho, se o problema esta em
    // reconhecer a letra errada ou em recusar a letra certa.
    public string UltimaLetra { get; private set; } = "-";

    // Resumo tipo "A: 7 B: 5 C: 4" - mostrado na tela de treinamento
    public string ResumoDoBanco()
    {
        if (bancoDeDados == null || bancoDeDados.letrasGravadas.Count == 0)
            return "Nenhuma letra gravada ainda.";

        var contagem = new SortedDictionary<string, int>();
        foreach (var l in bancoDeDados.letrasGravadas)
            contagem[l.nome] = contagem.ContainsKey(l.nome) ? contagem[l.nome] + 1 : 1;

        // Letras dinâmicas ganham um * para diferenciar (ex: "H*: 3")
        foreach (var s in bancoDeDados.sinaisDinamicos)
        {
            string chave = s.nome + "*";
            contagem[chave] = contagem.ContainsKey(chave) ? contagem[chave] + 1 : 1;
        }

        // Monta em linhas de 6 itens para caber legível no cartão
        var sb = new System.Text.StringBuilder();
        int itensNaLinha = 0;
        foreach (var par in contagem)
        {
            sb.Append(par.Key).Append(": ").Append(par.Value);
            itensNaLinha++;
            if (itensNaLinha % 6 == 0) sb.Append('\n');
            else sb.Append(" ");
        }
        return sb.ToString().TrimEnd();
    }

    // Normaliza pontos (relativos ao pulso) pelo TAMANHO da mão - a distância
    // do pulso até a base do dedo médio (ponto 9). Assim a mesma letra é
    // reconhecida perto OU longe da câmera, pois a escala deixa de importar.
    // Os dados já gravados continuam válidos: eles também passam por aqui.
    // Endireita a palma: gira os pontos para que o eixo pulso -> base do dedo
    // médio aponte para cima, mas nunca mais que 'giroMaximoCorrigido'.
    // Sem isso, inclinar a mão 20 graus já era suficiente para o sistema
    // deixar de reconhecer a letra.
    Vector3[] EndireitarPalma(Vector3[] relativosAoPulso)
    {
        if (giroMaximoCorrigido <= 0f) return relativosAoPulso;

        Vector2 eixo = new Vector2(relativosAoPulso[9].x, relativosAoPulso[9].y);
        if (eixo.sqrMagnitude < 1e-10f) return relativosAoPulso;

        // Quanto a palma está torta em relação ao "para cima" (0 = já está em pé)
        float desvio = Mathf.Atan2(eixo.x, eixo.y);
        float limite = giroMaximoCorrigido * Mathf.Deg2Rad;
        desvio = Mathf.Clamp(desvio, -limite, limite);

        float co = Mathf.Cos(desvio), se = Mathf.Sin(desvio);
        var girados = new Vector3[21];
        for (int i = 0; i < 21; i++)
        {
            Vector3 p = relativosAoPulso[i];
            girados[i] = new Vector3(p.x * co - p.y * se,
                                    p.x * se + p.y * co,
                                    p.z);
        }
        return girados;
    }

    static Vector3[] NormalizarEscala(Vector3[] relativosAoPulso)
    {
        float tamanhoMao = relativosAoPulso[9].magnitude;
        if (tamanhoMao < 0.0001f) tamanhoMao = 1f; // proteção contra divisão por zero

        var resultado = new Vector3[21];
        for (int i = 0; i < 21; i++)
            resultado[i] = relativosAoPulso[i] / tamanhoMao;
        return resultado;
    }

    // Grava UMA amostra da letra. Pode chamar várias vezes para a mesma letra
    // - quanto mais amostras, melhor o reconhecimento.
    public void GravarLetra(string nomeDaLetra, Vector3[] pontosAtuais)
    {

```

```

{
    if (bancoDeDados == null)
    {
        Debug.LogError("Banco de dados nao conectado no Inspector!");
        return;
    }
}

#if UNITY_EDITOR
    Undo.RecordObject(bancoDeDados, "Gravar Letra LIBRAS");
#endif

    var novaLetra = new AlfabetoData.LetraPadrao();
    novaLetra.nome = nomeDaLetra;
    novaLetra.pontosNormalizados = new Vector3[21];

    // Normaliza pela posição do pulso (ponto 0)
    Vector3 pulso = pontosAtuais[0];
    for (int i = 0; i < 21; i++)
        novaLetra.pontosNormalizados[i] = pontosAtuais[i] - pulso;

    bancoDeDados.letrasGravadas.Add(novaLetra);

    // Conta quantas amostras desta letra já existem
    int total = 0;
    foreach (var l in bancoDeDados.letrasGravadas)
        if (l.nome == nomeDaLetra) total++;

#if UNITY_EDITOR
    EditorUtility.SetDirty(bancoDeDados);
    AssetDatabase.SaveAssets();
#endif

    Debug.Log("Letra [" + nomeDaLetra + "] gravada! Total de amostras desta letra: " + total);
}

// Chamado automaticamente quando o jogador acerta uma letra no jogo.
// Acumula até 30 amostras por letra sem logs excessivos.
// O aprendizado automatico foi retirado.
//
// Cada acerto durante o jogo virava uma nova amostra do banco. Parecia
// bom, mas gravava a mao como ela estava no instante do acerto - inclinada,
// meio fechada, com a leitura ruim - e o banco ia se enchendo de amostras
// deformadas. Quinze das vinte letras chegaram ao teto de 30 amostras por
// esse caminho, sem ninguem conferir nenhuma delas.
//
// O banco agora so muda no treinamento, com quem grava vendo o que gravou.

// Grava um MOVIMENTO completo (sequência de quadros) para letra dinâmica
public void GravarSinalDinamico(string nome, List<Vector3[]> quadrosAbsolutos)
{
    if (bancoDeDados == null) return;

#if UNITY_EDITOR
    Undo.RecordObject(bancoDeDados, "Gravar Sinal Dinamico");
#endif

    var sinal = new AlfabetoData.SinalDinamico
    {
        nome = nome,
        quadros = new List<AlfabetoData.QuadroDeMao>()
    };
    foreach (var absoluto in quadrosAbsolutos)
    {
        // Cada quadro fica relativo ao pulso (igual às letras estáticas)
        var relativo = new Vector3[21];
        Vector3 pulso = absoluto[0];
        for (int i = 0; i < 21; i++) relativo[i] = absoluto[i] - pulso;
        sinal.quadros.Add(new AlfabetoData.QuadroDeMao { pontos = relativo });
    }
    bancoDeDados.sinaisDinamicos.Add(sinal);

    int total = 0;
    foreach (var s in bancoDeDados.sinaisDinamicos)
        if (s.nome == nome) total++;

#if UNITY_EDITOR
    EditorUtility.SetDirty(bancoDeDados);
    AssetDatabase.SaveAssets();
#endif

    Debug.Log("Movimento [" + nome + "] gravado com " + sinal.quadros.Count +
        " quadros! Amostras deste movimento: " + total);
}

// Apaga TODAS as amostras de uma letra (use Shift+Tecla no treinamento)
public void ApagarLetra(string nomeDaLetra)
{
    if (bancoDeDados == null) return;

#if UNITY_EDITOR
    Undo.RecordObject(bancoDeDados, "Apagar Letra LIBRAS");
#endif
}

```

```

        int removidos = bancoDeDados.letrasGravadas.RemoveAll(l => l.nome == nomeDaLetra);
        removidos += bancoDeDados.sinaisDinamicos.RemoveAll(s => s.nome == nomeDaLetra);

#if UNITY_EDITOR
        EditorUtility.SetDirty(bancoDeDados);
        AssetDatabase.SaveAssets();
#endif

        Debug.Log("Letra [" + nomeDaLetra + "] apagada. " + removidos + " amostra(s) removida(s).");
    }

    public string ClassificarLetra(Vector3[] pontosAtuais)
    {
        if (bancoDeDados == null || bancoDeDados.letrasGravadas.Count == 0) return "Nenhuma";

        // Características da mão atual: posições normalizadas + ângulos
        Vector3[] corrigidos = new Vector3[21];
        for (int i = 0; i < 21; i++) corrigidos[i] = DaCamera(pontosAtuais[i]);

        Vector3 pulso = corrigidos[0];
        Vector3[] rel = new Vector3[21];
        for (int i = 0; i < 21; i++) rel[i] = corrigidos[i] - pulso;
        Vector3[] atualPos = NormalizarEscala(EndireitarPalma(rel));
        float[] atualAng = ExtrairAngulos(corrigidos);
        float[] atualCon = ExtrairContatos(atualPos);

        // Distância da mão atual até TODAS as amostras gravadas
        var candidatos = new List<KeyValuePair<float, string>>();
        var amostra = new Vector3[21];
        foreach (var padrao in bancoDeDados.letrasGravadas)
        {
            for (int i = 0; i < 21; i++) amostra[i] = DoBanco(padrao.pontosNormalizados[i]);
            Vector3[] padraoPos = NormalizarEscala(EndireitarPalma(amostra));
            float[] padraoAng = ExtrairAngulos(amostra);
            float[] padraoCon = ExtrairContatos(padraoPos);
            float dist = DistanciaEntre(atualPos, atualAng, atualCon,
                                       padraoPos, padraoAng, padraoCon);
            candidatos.Add(new KeyValuePair<float, string>(dist, padrao.nome));
        }
        candidatos.Sort((a, b) => a.Key.CompareTo(b.Key));

        // Votação KNN com peso: cada uma das K amostras mais próximas vota com
        // força 1/distância, então uma amostra bem parecida pesa mais que outra
        // que só entrou na lista por falta de concorrência.
        int k = Mathf.Min(vizinhosK, candidatos.Count);
        var forca = new Dictionary<string, float>();
        var melhorDistDaLetra = new Dictionary<string, float>();
        for (int i = 0; i < k; i++)
        {
            string nome = candidatos[i].Value;
            float peso = 1f / (candidatos[i].Key + 0.5f);
            forca[nome] = forca.ContainsKey(nome) ? forca[nome] + peso : peso;
            if (!melhorDistDaLetra.ContainsKey(nome))
                melhorDistDaLetra[nome] = candidatos[i].Key; // lista ordenada -> 1ª é a menor
        }

        string vencedora = "";
        float maiorForca = -1f;
        foreach (var par in forca)
            if (par.Value > maiorForca) { maiorForca = par.Value; vencedora = par.Key; }

        float menorDistancia = melhorDistDaLetra[vencedora];

        // Melhor distância de uma letra DIFERENTE: a rival mais próxima
        float distanciaDaRival = float.MaxValue;
        foreach (var c in candidatos)
            if (c.Value != vencedora) { distanciaDaRival = c.Key; break; }

        UltimaDistancia = menorDistancia;
        UltimaLetra = vencedora;

        // Debug: mostra a cada 0.5s a eleição, a distância e a da rival.
        // O que importa é a FOLGA entre as duas: quanto maior, mais seguro.
        if (mostrarDebug && Time.time - tempoUltimoDebug > 0.5f)
        {
            tempoUltimoDebug = Time.time;
            Debug.Log("Mais parecido: [" + vencedora + "] entre " + k +
                    " vizinhos, distancia " + menorDistancia.ToString("F2") +
                    ", rival a " + distanciaDaRival.ToString("F2") +
                    " -> " + (Aceita(menorDistancia, distanciaDaRival) ? "ACEITA" : "recusada"));
        }

        return Aceita(menorDistancia, distanciaDaRival) ? vencedora : "Desconhecido";
    }

    // Duas condições para dar a letra por feita:
    // 1. a mão parece com alguma coisa do banco (limite de segurança);
    // 2. a vencedora ganha da rival com folga.
    // A segunda é o critério de verdade, e é relativa: se a leitura piorar,
    // as duas distâncias sobem juntas e a comparação continua honesta. Era
    // exatamente aí que o limite fixo falhava - bastava a mão inclinar um
    // pouco para TUDO passar do limite e nada mais ser reconhecido.

```

```

bool Aceita(float distanciaDaVencedora, float distanciaDaRival)
{
    return distanciaDaVencedora < limiteDeSeguranca &&
           distanciaDaVencedora < vantagemNecessaria * distanciaDaRival;
}

// — Letras dinâmicas: comparação de MOVIMENTOS via DTW —————

private float tempoUltimoDebugDinamico = 0f;

// Compara a janela de movimento atual com os movimentos gravados.
// Retorna a letra vencedora ou "Desconhecido".
public string ClassificarSinalDinamico(List<Vector3[]> janelaAbsoluta)
{
    if (bancoDeDados == null || bancoDeDados.sinaisDinamicos == null ||
        bancoDeDados.sinaisDinamicos.Count == 0 ||
        janelaAbsoluta == null || janelaAbsoluta.Count < 6)
        return "Desconhecido";

    // Normaliza a janela atual uma única vez (pulso + tamanho da mão)
    var janela = new List<Vector3[]>(janelaAbsoluta.Count);
    foreach (var absoluto in janelaAbsoluta)
    {
        var relativo = new Vector3[21];
        Vector3 pulso = DaCamera(absoluto[0]);
        for (int i = 0; i < 21; i++) relativo[i] = DaCamera(absoluto[i]) - pulso;
        janela.Add(NormalizarEscala(EndireitarPalma(relativo)));
    }

    string melhor = "Desconhecido";
    float menor = float.MaxValue;
    // Melhor custo de um movimento de OUTRA letra, para medir a folga
    var menorPorLetra = new Dictionary<string, float>();

    foreach (var sinal in bancoDeDados.sinaisDinamicos)
    {
        var amostra = new List<Vector3[]>(sinal.quadros.Count);
        foreach (var quadro in sinal.quadros)
        {
            var q = new Vector3[21];
            for (int i = 0; i < 21; i++) q[i] = DoBanco(quadro.pontos[i]);
            amostra.Add(NormalizarEscala(EndireitarPalma(q)));
        }

        float custo = CustoDTW(janela, amostra);
        if (!menorPorLetra.ContainsKey(sinal.nome) || custo < menorPorLetra[sinal.nome])
            menorPorLetra[sinal.nome] = custo;
        if (custo < menor)
        {
            menor = custo;
            melhor = sinal.nome;
        }
    }

    float custoDaRival = float.MaxValue;
    foreach (var par in menorPorLetra)
        if (par.Key != melhor && par.Value < custoDaRival) custoDaRival = par.Value;

    if (mostrarDebug && menor < float.MaxValue &&
        Time.time - tempoUltimoDebugDinamico > 0.6f)
    {
        tempoUltimoDebugDinamico = Time.time;
        Debug.Log("Movimento mais parecido: [" + melhor + "] custo " +
            menor.ToString("F2") + ", rival a " + custoDaRival.ToString("F2"));
    }

    bool aceita = menor < limiteDinamicoDeSeguranca &&
                 menor < vantagemNecessaria * custoDaRival;
    return aceita ? melhor : "Desconhecido";
}

static float DistanciaEntreQuadros(Vector3[] a, Vector3[] b)
{
    float d = 0f;
    for (int i = 0; i < 21; i++) d += Vector3.Distance(a[i], b[i]);
    return d;
}

// DTW (Dynamic Time Warping): alinha duas sequências no tempo antes de
// comparar - o MESMO gesto feito mais rápido ou mais devagar ainda casa.
// Retorna o custo médio por passo (independe do tamanho das sequências).
static float CustoDTW(List<Vector3[]> a, List<Vector3[]> b)
{
    int n = a.Count, m = b.Count;
    float[,] D = new float[n + 1, m + 1];
    for (int i = 0; i <= n; i++)
        for (int j = 0; j <= m; j++)
            D[i, j] = float.PositiveInfinity;
    D[0, 0] = 0f;

    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)

```

```
        {
            float custo = DistanciaEntreQuadros(a[i - 1], b[j - 1]);
            float menorCaminho = Mathf.Min(D[i - 1, j],
                                             Mathf.Min(D[i, j - 1], D[i - 1, j - 1]));
            D[i, j] = custo + menorCaminho;
        }
    }
    return D[n, m] / (n + m);
}
```

Assets/ReproduzirAnimacao.cs (61 linhas)

```
using UnityEngine;
using UnityEngine.Playables;
using UnityEngine.Animations;

// ReproduzirAnimacao: toca um clipe de animação de um modelo em LOOP,
// sem precisar de Animator Controller nem de importação Legacy.
// Usa a Playables API do Unity e reinicia o clipe manualmente ao terminar.
//
// Uso: ReproduzirAnimacao.Tocar(objeto, "caminho/do/modelo/em/Resources");
public class ReproduzirAnimacao : MonoBehaviour
{
    private AnimationClip      clipe;
    private PlayableGraph      grafo;
    private AnimationClipPlayable playable;
    private bool               pronto = false;

    // Retorna true se encontrou e tocou uma animação (false = modelo estático)
    public static bool Tocar(GameObject alvo, string caminhoResources)
    {
        // Os clipes moram DENTRO do arquivo FBX - LoadAll pega os sub-assets
        var clipes = Resources.LoadAll<AnimationClip>(caminhoResources);
        if (clipes == null || clipes.Length == 0) return false; // sem animação

        // Prefere um clipe chamado "Idle" (parado respirando); senão o primeiro
        AnimationClip escolhido = clipes[0];
        foreach (var c in clipes)
            if (c.name.ToLower().Contains("idle")) { escolhido = c; break; }

        var animator = alvo.GetComponentInChildren<Animator>();
        if (animator == null) animator = alvo.AddComponent<Animator>();

        var reprodutor = alvo.AddComponent<ReproduzirAnimacao>();
        reprodutor.Iniciar(animator, escolhido);
        return true;
    }

    void Iniciar(Animator animator, AnimationClip clipeEscolhido)
    {
        clipe      = clipeEscolhido;
        grafo      = PlayableGraph.Create("AnimacaoDoObjeto");
        playable = AnimationClipPlayable.Create(grafo, clipe);

        var saida = AnimationPlayableOutput.Create(grafo, "saida", animator);
        saida.SetSourcePlayable(playable);
        grafo.Play();
        pronto = true;
    }

    void Update()
    {
        // Loop manual: quando o clipe chega ao fim, volta ao começo
        if (pronto && playable.IsValid() && playable.GetTime() >= clipe.length)
            playable.SetTime(0);
    }

    void OnDestroy()
    {
        if (grafo.IsValid()) grafo.Destroy();
    }
}
```

Assets/RespirarSuave.cs (31 linhas)

```
using UnityEngine;

// RespirarSuave: dá "vida" a modelos SEM animação - o corpo infla e
// desinfla de leve, como uma respiração calma. Usado como plano B
// quando o modelo não tem clipes de animação (ex: Farm Animals).
public class RespirarSuave : MonoBehaviour
{
    [Range(0.01f, 0.1f)] public float intensidade = 0.035f; // 3,5% de variação
    [Range(0.3f, 4f)] public float velocidade = 1.6f;

    private Vector3 escalaBase;
    private float fase;

    void Start()
    {
        escalaBase = transform.localScale; // captura DEPOIS do auto-ajuste
        fase = Random.Range(0f, Mathf.PI * 2f);
    }

    void Update()
    {
        float s = Mathf.Sin(Time.time * velocidade + fase) * intensidade;

        // Sobe no Y e encolhe de leve nos lados (conserva o "volume")
        transform.localScale = new Vector3(
            escalaBase.x * (1f - s * 0.5f),
            escalaBase.y * (1f + s),
            escalaBase.z * (1f - s * 0.5f));
    }
}
```


Assets/UIControle.cs (606 linhas)

```
using UnityEngine;
using UnityEngine.UI;
using TMPro;
using System.Collections;
using System.Text;
using UnityEngine.Events;

[RequireComponent(typeof(TextMeshProUGUI))]
public class UIControle : MonoBehaviour
{
    [Header("Ajuste do HUD superior (fracoes 0..1 da faixa)")]
    [Tooltip("X,Y do numero de PONTOS sobre o desenho")] public Vector2 ancoraPontos = new Vector2(0.168f,
0.71f);
    [Tooltip("X,Y do 1o coracao (as VIDAS)")] public Vector2 ancoraVidas = new Vector2(0.060f,
0.31f);
    [Tooltip("X,Y do numero do TEMPO sobre a estrela")] public Vector2 ancoraTempo = new Vector2(0.260f,
0.463f);
    [Tooltip("Tamanho de cada coracao")] public float tamanhoCoracao = 28f;
    [Tooltip("Distancia entre os coracoes")] public float espacoCoracao = 32f;

    [Header("Mover os textos do jogo (PALAVRA e tracejado). PC e Tablet separados")]
    public Vector2 offTextoPC; // X: +direita / -esquerda ; Y: +cima / -baixo
    public Vector2 offTextoTablet;

    [Header("Gerenciador do jogo")]
    public GerenciadorDeJogo gerenciador;

    [Header("Controlador da camera (para a barra do sinal)")]
    public ControladorCamera controlador;

    [Header("Texto da pontuacao (deixe vazio: e criado sozinho)")]
    public TextMeshProUGUI textoScore;

    private TextMeshProUGUI tmp;
    private TextMeshProUGUI rotulo; // "PALAVRA:" pequeno no topo do cartão
    private GameObject chipScore; // agora aponta para o painel de stats (imagem HUD)
    private GameObject painelStats; // imagem HUD_Superior (cluster esquerdo)
    private GameObject containerVidas;
    private RectTransform hudRect; // faixa HUD (mantem o aspecto)
    private MenuPrincipal menuPrincipal; // para o botao MENU invisivel
    private Image riscoSomHud; // risco do som na faixa

    // Barra "reconhecendo o sinal X..." (feedback em tempo real)
    private GameObject barraSinal;
    private Image preenchimentoSinal;
    private TextMeshProUGUI letraSinal;
    private static readonly Color COR_BARRA_INICIO = new Color(0.2f, 0.8f, 1f, 0.9f);
    private static readonly Color COR_BARRA_FIM = new Color(0.1f, 0.85f, 0.3f, 1f);

    // Vidas (corações) e relógio da palavra
    private GameObject chipVidas;
    private readonly System.Collections.Generic.List<GameObject> coracoes =
        new System.Collections.Generic.List<GameObject>();
    private GameObject chipTempo;
    private TextMeshProUGUI textoTempo;
    private Transform raizCanvas; // raiz do Canvas (para confetes/flashes)
    private Image flashTela; // flash verde/vermelho (feedback visual)
    private static readonly Color COR_VIDA = new Color(0.95f, 0.25f, 0.35f, 1f);
    private static readonly Color COR_TEMPO_OK = Color.white;
    private static readonly Color COR_TEMPO_FIM = new Color(1f, 0.35f, 0.25f, 1f);
    private Vector2 rotuloAuto, tmpBase; private bool basesPegas;
    private string palavraAnterior = null;
    private int indiceAnterior = -1;
    private bool celebrando = false;
    private bool inscrito = false;

    private static readonly Color COR_NORMAL = new Color(0.12f, 0.12f, 0.12f, 1f);
    private static readonly Color COR_CEBRACAO = new Color(0.1f, 0.65f, 0.15f, 1f);
    private static readonly Color COR_PARABENS = new Color(0.85f, 0.55f, 0f, 1f);
    private static readonly Color COR_ROTULO = new Color(0.45f, 0.45f, 0.52f, 1f);

    void Awake()
    {
        tmp = GetComponent<TextMeshProUGUI>();
        tmp.richText = false;
        tmp.enableAutoSizing = true;
        tmp.fontSizeMin = 20f;
        tmp.fontSizeMax = 130f;
        tmp.alignment = TextAlignmentOptions.Center;
        tmp.enableWordWrapping = false;
        tmp.overflowMode = TextOverflowModes.Overflow;
        tmp.color = COR_NORMAL;
        tmp.margin = new Vector4(0, 42, 0, 64); // tracejado mais alto (a letra identificada fica
visivel)
        tmp.text = "";
        tmpBase = tmp.rectTransform.anchoredPosition; rotuloAuto = Vector2.zero; basesPegas = true;

        // Rótulo "PALAVRA:" preso ao topo do cartão (igual ao app de referência)
```

```

rotulo = UIFabrica.CriarTexto(transform.parent, "RotuloPalavra", "PALAVRA:",
    36f, COR_ROTULO, new Vector2(0, 4), new Vector2(900, 50));
UIFabrica.Ancorar(rotulo, new Vector2(0.5f, 1f), new Vector2(0.5f, 1f));

// Paine de estatisticas: imagem HUD_Superior (recorte do cluster
// esquerdo) com PONTOS, VIDAS e a estrela do tempo. Os valores dinamicos
// sao FILHOS da imagem, ancorados por fracao, entao ficam sempre no lugar
// certo por cima do desenho, em qualquer tamanho de tela.
{
    var canvas = GetComponentInParent<Canvas>();
    var hud = UIFabrica.CriarImagem(canvas.transform, "HudSuperior",
        Color.white, Vector2.zero, new Vector2(0, 258), Resources.Load<Sprite>("ui/hud"));
    hud.raycastTarget = false;
    var hr = hud.rectTransform;
    hr.anchorMin = new Vector2(0f, 1f); hr.anchorMax = new Vector2(1f, 1f);
    hr.pivot = new Vector2(0.5f, 1f); hr.anchoredPosition = Vector2.zero;
    Canvas.ForceUpdateCanvases();
    float wRef = hr.rect.width;
    if (wRef < 1f) { var cr = canvas.GetComponent<RectTransform>(); wRef = cr != null ? cr.rect.width :
1920f; }

    hr.sizeDelta = new Vector2(0f, wRef * 0.1345f); // ja no tamanho certo, sem atraso
    painelStats = hud.gameObject; chipScore = painelStats; hudRect = hr;

    textoScore = OverlayText(hud.transform, "Pontos", "0", 46f, Color.white,
        ancoraPontos, new Vector2(0f, 0.5f), TextAlignmentOptions.Left);
    containerVidas = OverlayText(hud.transform, "Vidas", "", 10f, Color.white,
        ancoraVidas, new Vector2(0f, 0.5f), TextAlignmentOptions.Left).gameObject;
    textoTempo = OverlayText(hud.transform, "Tempo", "0", 68f, COR_TEMPO_OK,
        ancoraTempo, new Vector2(0.5f, 0.5f), TextAlignmentOptions.Center);

    for (int i = 0; i < 5; i++)
    {
        var coracao = UIFabrica.CriarImagem(containerVidas.transform, "Coracao" + i,
            COR_VIDA, new Vector2(i * espacoCoracao, 0),
            new Vector2(tamanhoCoracao, tamanhoCoracao), UIFabrica.Coracao());
        coracao.raycastTarget = false;
        coracoes.Add(coracao.gameObject);
    }

    // MENU e SOM ja estao desenhados na imagem; botoes invisiveis por cima
    menuPrincipal = FindObjectOfType<MenuPrincipal>();
    HitInvisivel(hud.transform, "HitMenu", new Vector2(0.878f, 0.55f), new Vector2(0.985f, 0.95f),
        () => { if (menuPrincipal != null) menuPrincipal.AbrirMenuComSom(); });
    var hitSom = HitInvisivel(hud.transform, "HitSom", new Vector2(0.905f, 0.05f), new Vector2(0.99f,
0.48f),

        AlternarSomHud);
    riscoSomHud = UIFabrica.CriarImagem(hitSom, "Risco",
        new Color(0.9f, 0.2f, 0.2f, 0.95f), Vector2.zero, new Vector2(70, 8),
        UIFabrica.Arredondado(), true);
    riscoSomHud.rectTransform.localEulerAngles = new Vector3(0, 0, 20f);
    riscoSomHud.raycastTarget = false;
    riscoSomHud.gameObject.SetActive(!GerenciadorDeAudio.MusicaLigada);

    painelStats.SetActive(false);
}

// Barra de progresso do sinal: mostra "estou quase aceitando a letra X"
// - o jogador vê o sistema trabalhando em vez de achar que travou
var fundoBarra = UIFabrica.CriarImagem(transform.parent, "BarraSinal",
    new Color(0f, 0f, 0f, 0.15f), new Vector2(20, 22), new Vector2(340, 16),
    UIFabrica.Arredondado(), true);
UIFabrica.Ancorar(fundoBarra, new Vector2(0.5f, 0f), new Vector2(0.5f, 0f));

preenchimentoSinal = UIFabrica.CriarImagem(fundoBarra.transform, "Preenchimento",
    COR_BARRA_INICIO, Vector2.zero, new Vector2(340, 16),
    UIFabrica.Arredondado(), true);
preenchimentoSinal.type = Image.Type.Filled;
preenchimentoSinal.fillMethod = Image.FillMethod.Horizontal;
preenchimentoSinal.fillAmount = 0f;

letraSinal = UIFabrica.CriarTexto(fundoBarra.transform, "Letra", "",
    34f, new Color(0.25f, 0.25f, 0.3f, 1f), new Vector2(-200, 2), new Vector2(60, 44));

barraSinal = fundoBarra.gameObject;
barraSinal.SetActive(false);

raizCanvas = GetComponentInParent<Canvas>().transform;

// Flash de tela: feedback VISUAL de acerto (verde) e erro (vermelho).
// Essencial para o público surdo - os sons têm gêmeos visuais!
flashTela = UIFabrica.CriarImagem(raizCanvas, "FlashTela",
    new Color(0f, 0f, 0f, 0f), Vector2.zero, Vector2.zero);
flashTela.rectTransform.anchorMin = Vector2.zero;
flashTela.rectTransform.anchorMax = Vector2.one;
flashTela.rectTransform.sizeDelta = Vector2.zero;
flashTela.raycastTarget = false;
}

// Botao invisivel (so area de clique) sobre um trecho da imagem do HUD.
Transform HitInvisivel(Transform pai, string nome, Vector2 aMin, Vector2 aMax, UnityAction acao)
{

```

```

        var img = UIFabrica.CriarImagem(pai, nome, new Color(1f, 1f, 1f, 0f), Vector2.zero, Vector2.zero);
        var rt = img.rectTransform;
        rt.anchorMin = aMin; rt.anchorMax = aMax; rt.offsetMin = Vector2.zero; rt.offsetMax = Vector2.zero;
        img.raycastTarget = true;
        img.gameObject.AddComponent<Button>().onClick.AddListener(acao);
        return img.transform;
    }

    void AlternarSomHud()
    {
        GerenciadorDeAudio.TocarClique();
        GerenciadorDeAudio.AlternarMusica();
        if (riscoSomHud != null) riscoSomHud.gameObject.SetActive(!GerenciadorDeAudio.MusicaLigada);
    }

    // Mantem a faixa do HUD na proporcao certa (altura = largura x aspecto)
    void LateUpdate()
    {
        if (hudRect == null) return;
        float alvo = hudRect.rect.width * 0.1345f;
        if (Mathf.Abs(hudRect.sizeDelta.y - alvo) > 1f)
            hudRect.sizeDelta = new Vector2(0f, alvo);
    }

    // Cria um texto ancorado por FRACAO (0..1) dentro de um pai, para ficar
    // por cima de um ponto especifico da imagem do HUD.
    TextMeshProUGUI OverlayTexto(Transform pai, string nome, string txt, float tam,
        Color cor, Vector2 anc, Vector2 pivo, TextAlignmentOptions alinhamento)
    {
        var t = UIFabrica.CriarTexto(pai, nome, txt, tam, cor, Vector2.zero, new Vector2(260, 80));
        t.alignment = alinhamento;
        var rt = t.rectTransform;
        rt.anchorMin = rt.anchorMax = anc;
        rt.pivot = pivo;
        rt.anchoredPosition = Vector2.zero;
        return t;
    }

    // OnEnable/OnDisable: o painel é ligado/desligado pelo MenuPrincipal,
    // então a inscrição nos eventos precisa acompanhar isso
    void OnEnable() { Inscrever(); }
    void Start() { Inscrever(); }

    void OnDisable()
    {
        celebrando = false; // corrotina morre junto com o objeto desativado

        // Painel desativado (menu aberto): esconde os chips também
        if (chipScore != null) chipScore.SetActive(false);
        if (chipVidas != null) chipVidas.SetActive(false);
        if (chipTempo != null) chipTempo.SetActive(false);

        if (!inscrito || gerenciador == null) return;
        gerenciador.OnPalavraCompleta -= IniciarCelebracao;
        gerenciador.OnPontuacaoAtualizada -= AtualizarScore;
        gerenciador.OnVidasAtualizadas -= AtualizarVidas;
        gerenciador.OnNovaFase -= MostrarFase;
        gerenciador.OnPontosGastos -= MostrarPontosGastos;
        gerenciador.OnLetraCorreta -= FlashVerde;
        gerenciador.OnVidaPerdida -= FlashVermelho;
        gerenciador.OnVidaGanha -= AnimarVidaExtra;
        gerenciador.OnSemSaldo -= AvisarSemSaldo;
        tremendoSaldo = false;
        inscrito = false;
    }

    void Inscrever()
    {
        if (inscrito || gerenciador == null) return;
        gerenciador.OnPalavraCompleta += IniciarCelebracao;
        gerenciador.OnPontuacaoAtualizada += AtualizarScore;
        gerenciador.OnVidasAtualizadas += AtualizarVidas;
        gerenciador.OnNovaFase += MostrarFase;
        gerenciador.OnPontosGastos += MostrarPontosGastos;
        gerenciador.OnLetraCorreta += FlashVerde;
        gerenciador.OnVidaPerdida += FlashVermelho;
        gerenciador.OnVidaGanha += AnimarVidaExtra;
        gerenciador.OnSemSaldo += AvisarSemSaldo;
        inscrito = true;
    }

    // Sem pontos para pular: o chip de pontos TREME e pisca vermelho
    // (feedback visual - o som de erro sozinho não serve ao público surdo)
    private bool tremendoSaldo = false;

    void AvisarSemSaldo()
    {
        if (!tremendoSaldo) StartCoroutine(RotinaSemSaldo());
    }

    IEnumerator RotinaSemSaldo()
    {

```

```

        if (chipScore == null) yield break;
        tremendoSaldo = true;

        var imagem      = chipScore.GetComponent<Image>();
        var rt          = (RectTransform)chipScore.transform;
        Color corOriginal = imagem.color;
        Vector2 posOriginal = rt.anchoredPosition;
        Color vermelho    = new Color(0.85f, 0.15f, 0.15f, 0.9f);

        float duracao = 0.5f;
        for (float t = 0f; t < duracao; t += Time.deltaTime)
        {
            float p = t / duracao;
            imagem.color = Color.Lerp(vermelho, corOriginal, p);
            // tremida horizontal que vai se acalmando
            rt.anchoredPosition = posOriginal +
                new Vector2(Mathf.Sin(p * 45f) * 12f * (1f - p), 0f);
            yield return null;
        }

        imagem.color      = corOriginal;
        rt.anchoredPosition = posOriginal;
        tremendoSaldo     = false;
    }

    // "+1 coracao": um coração com "+1" sobe perto do chip de vidas, e o coração
    // novo do chip aparece dando um "pulo" (escala 0 -> 1 com exagero)
    void AnimarVidaExtra()
    {
        StartCoroutine(RotinaVidaExtra());
    }

    IEnumerator RotinaVidaExtra()
    {
        // Coração flutuante com "+1" ao lado do chip de vidas
        var coracao = UIFabrica.CriarImagem(raizCanvas, "VidaExtra", COR_VIDA,
            new Vector2(380, -175), new Vector2(56, 56), UIFabrica.Coracao());
        UIFabrica.Ancorar(coracao, new Vector2(0f, 1f), new Vector2(0f, 1f));
        coracao.raycastTarget = false;
        var mais = UIFabrica.CriarTexto(coracao.transform, "Mais", "+1",
            36f, Color.white, new Vector2(52, 0), new Vector2(80, 56));

        // O coração novo do chip "pula" ao aparecer
        int indiceNovo = gerenciador.Vidas - 1;
        Transform coracaoDoChip = (indiceNovo >= 0 && indiceNovo < coracoes.Count)
            ? coracoes[indiceNovo].transform : null;

        var rt = coracao.rectTransform;
        Vector2 inicio = rt.anchoredPosition;
        float duracao = 1.3f;

        for (float t = 0f; t < duracao; t += Time.deltaTime)
        {
            float p = t / duracao;

            rt.anchoredPosition = inicio + new Vector2(0f, 100f * p);
            var c = coracao.color; c.a = 1f - p; coracao.color = c;
            var cm = mais.color; cm.a = 1f - p; mais.color = cm;

            // "pulo" do coração do chip nos primeiros 40% da animação
            if (coracaoDoChip != null && p < 0.4f)
            {
                float pulo = Mathf.Sin((p / 0.4f) * Mathf.PI); // 0 -> 1 -> 0
                coracaoDoChip.localScale = Vector3.one * (1f + 0.6f * pulo);
            }
            yield return null;
        }

        if (coracaoDoChip != null) coracaoDoChip.localScale = Vector3.one;
        Destroy(coracao.gameObject);
    }

    void FlashVerde() { StartCoroutine(RotinaDeFlash(new Color(0.2f, 0.9f, 0.3f, 0.30f))); }
    void FlashVermelho() { StartCoroutine(RotinaDeFlash(new Color(0.95f, 0.2f, 0.2f, 0.40f))); }

    IEnumerator RotinaDeFlash(Color cor)
    {
        if (flashTela == null) yield break;
        float duracao = 0.45f;
        for (float t = 0f; t < duracao; t += Time.deltaTime)
        {
            var c = cor;
            c.a = Mathf.Lerp(cor.a, 0f, t / duracao);
            flashTela.color = c;
            yield return null;
        }
        flashTela.color = new Color(0, 0, 0, 0);
    }

    // Animação "-5"/"-10" em vermelho subindo perto da pontuação
    void MostrarPontosGastos(int valor)
    {

```

```

        StartCoroutine(RotinaPontosGastos(valor));
    }

IEnumerator RotinaPontosGastos(int valor)
{
    var texto = UIFabrica.CriarTexto(raizCanvas, "PontosGastos", "-" + valor,
        52f, new Color(0.95f, 0.2f, 0.2f, 1f), new Vector2(200, -70), new Vector2(200, 80));
    UIFabrica.Ancorar(texto, new Vector2(0f, 1f), new Vector2(0f, 1f));

    var rt = (RectTransform)texto.transform;
    Vector2 inicio = rt.anchoredPosition;
    float duracao = 1.1f;

    for (float t = 0f; t < duracao; t += Time.deltaTime)
    {
        float p = t / duracao;
        rt.anchoredPosition = inicio + new Vector2(0f, 90f * p); // sobe
        var c = texto.color;
        c.a = 1f - p; // some
        texto.color = c;
        yield return null;
    }
    Destroy(texto.gameObject);
}

void AtualizarVidas(int vidas)
{
    for (int i = 0; i < coracoes.Count; i++)
        coracoes[i].SetActive(i < vidas);
}

void MostrarFase(string texto)
{
    StartCoroutine(RotinaDeFase(texto));
}

IEnumerator RotinaDeFase(string texto)
{
    celebrando = true;
    rotulo.gameObject.SetActive(false);
    if (barraSinal != null) barraSinal.SetActive(false);

    tmp.color = new Color(0.15f, 0.4f, 0.85f, 1f); // azul de fase
    tmp.text = texto;
    yield return new WaitForSeconds(2.2f);

    tmp.color = COR_NORMAL;
    tmp.text = "";
    palavraAnterior = null;
    indiceAnterior = -1;
    celebrando = false;
}

void AtualizarScore(int score)
{
    if (textoScore != null)
        textoScore.text = score.ToString();
}

void Update()
{
    if (gerenciador == null || celebrando) return;

    // Chips (pontos, vidas, tempo) só aparecem com o jogo rodando.
    // No modo sem pressão, tempo e vidas nem existem na tela.
    bool rodando = gerenciador.JogoIniciado;
    bool jogando = rodando && !gerenciador.JogoTerminado;
    bool comPressao = !gerenciador.modoSemPressao;
    bool verVidas = rodando && comPressao;
    bool verTempo = jogando && comPressao;
    if (chipScore != null && chipScore.activeSelf != rodando) chipScore.SetActive(rodando);
    if (chipVidas != null && chipVidas.activeSelf != verVidas) chipVidas.SetActive(verVidas);
    if (chipTempo != null && chipTempo.activeSelf != verTempo) chipTempo.SetActive(verTempo);

    // Relógio da palavra: fica vermelho nos últimos 5 segundos
    if (verTempo && textoTempo != null)
    {
        int segundos = Mathf.CeilToInt(gerenciador.TempoRestante);
        textoTempo.text = segundos.ToString();
        textoTempo.color = (segundos <= 5) ? COR_TEMPO_FIM : COR_TEMPO_OK;
    }

    // No menu / treinamento não há palavra para mostrar
    if (!gerenciador.JogoIniciado)
    {
        if (palavraAnterior != null)
        {
            tmp.text = "";
            palavraAnterior = null;
            indiceAnterior = -1;
        }
        return;
    }
}

```

```

    }

    if (gerenciador.JogoTerminado)
    {
        if (palavraAnterior != "FIM")
        {
            rotulo.gameObject.SetActive(false);
            barraSinal.SetActive(false);
            if (gerenciador.Venceu)
            {
                tmp.color = COR_Celebracao;
                tmp.text = "PARABÉNS!\nVOCÊ VENCEU!";
                ChuvaDeConfetes.Lancar(raizCanvas);
            }
            else
            {
                tmp.color = COR_Parabens;
                tmp.text = "FIM DE JOGO";
            }
            palavraAnterior = "FIM";
        }
        return;
    }

    AtualizarBarraSinal();

    string palavraAtual = gerenciador.PalavraAtual;
    int indiceAtual = gerenciador.IndiceLetraAtual;

    if (palavraAtual == palavraAnterior && indiceAtual == indiceAnterior) return;

    palavraAnterior = palavraAtual;
    indiceAnterior = indiceAtual;

    ExibirPalavra(palavraAtual, indiceAtual);
}

void IniciarCelebracao(string palavraCompleta)
{
    StartCoroutine(RotinaDeCelebracao(palavraCompleta));
}

// Abre espaço à ESQUERDA do cartão para o objeto 3D ficar ao lado da
// palavra (as letras, o rótulo e a barrinha deslocam para a direita).
// Chamado pelo MenuPrincipal quando a orientação da tela muda.
public void DefinirEspacoDoObjeto(float margemEsquerda, float deslocamentoX)
{
    if (tmp != null)
    {
        var margem = tmp.margin;
        margem.x = margemEsquerda;
        tmp.margin = margem;
    }
    if (rotulo != null)
    {
        var rt = rotulo.rectTransform;
        rt.anchoredPosition = new Vector2(deslocamentoX, rt.anchoredPosition.y);
        rotuloAuto = rt.anchoredPosition;
    }
    if (barraSinal != null)
    {
        var rt = (RectTransform)barraSinal.transform;
        rt.anchoredPosition = new Vector2(deslocamentoX + 20f, rt.anchoredPosition.y);
    }
}

// Move os textos do jogo pelo ajuste do Inspector (delta do automatico).
public void AjustarTextos(bool horizontal)
{
    if (!basesPegas) return;
    Vector2 off = horizontal ? offTextoPC : offTextoTablet;
    if (rotulo != null) rotulo.rectTransform.anchoredPosition = rotuloAuto + off;
    if (tmp != null) tmp.rectTransform.anchoredPosition = tmpBase + off;
}

// Mostra/atualiza a barrinha "reconhecendo o sinal X..."
// Quando a letra esperada é DINÂMICA, vira uma dica pulsante de movimento.
void AtualizarBarraSinal()
{
    if (barraSinal == null || controlador == null) return;

    // Letra com movimento: barra pulsa como convite para mexer a mão
    if (controlador.EsperandoMovimento)
    {
        if (!barraSinal.activeSelf) barraSinal.SetActive(true);
        preenchimentoSinal.fillAmount = Mathf.PingPong(Time.time * 0.7f, 1f);
        preenchimentoSinal.color = COR_BARRA_INICIO;
        letraSinal.text = "MOV";
        return;
    }

    string candidata = controlador.LetraCandidata;

```

```

        bool mostrar = !string.IsNullOrEmpty(candidata);

        if (barraSinal.isActiveSelf != mostrar) barraSinal.SetActive(mostrar);
        if (!mostrar) return;

        float progresso = controlador.ProgressoCandidata;
        preenchimentoSinal.fillAmount = progresso;
        preenchimentoSinal.color = Color.Lerp(COR_BARRA_INICIO, COR_BARRA_FIM, progresso);
        letraSinal.text = candidata;
    }

    IEnumerator RotinaDeCelebracao(string palavra)
    {
        celebrando = true;
        if (barraSinal != null) barraSinal.SetActive(false);

        // Mostra palavra completa em verde
        tmp.color = COR_CELEBRACAO;
        MostrarPalavraCompleta(palavra);
        yield return new WaitForSeconds(1.2f);

        // Mensagem de parabéns
        rotulo.gameObject.SetActive(false);
        tmp.text = "MUITO BEM!";
        yield return new WaitForSeconds(1.0f);

        tmp.color      = COR_NORMAL;
        tmp.text       = "";
        palavraAnterior = null;
        indiceAnterior = -1;
        celebrando     = false;
    }

    void MostrarPalavraCompleta(string palavra)
    {
        var sb = new StringBuilder();
        for (int i = 0; i < palavra.Length; i++)
        {
            if (i > 0) sb.Append(" ");
            sb.Append(palavra[i]);
        }
        tmp.text = sb.ToString();
    }

    void ExibirPalavra(string palavra, int preenchidas)
    {
        if (string.IsNullOrEmpty(palavra)) { tmp.text = ""; return; }

        rotulo.gameObject.SetActive(true);
        // Mostra o NOME do objeto (não é trapaça: soletrar em LIBRAS é o desafio,
        // e ler a palavra escrita é justamente a parte de alfabetização!)
        rotulo.text = "PALAVRA: " + palavra;
        tmp.color = COR_NORMAL;

        var sb = new StringBuilder();
        for (int i = 0; i < palavra.Length; i++)
        {
            if (i > 0) sb.Append(" ");
            sb.Append(i < preenchidas ? palavra[i] : '_');
        }
        tmp.text = sb.ToString();
    }
}

```

Assets/UIFabrica.cs (348 linhas)

```
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.Events;
using TMPro;

// UIFabrica: cria elementos de interface por código (botões, textos, painéis)
// e gera sprites simples (cantos arredondados, círculo, gradiente) sem precisar
// de arquivos de imagem no projeto.
public static class UIFabrica
{
    static Sprite spriteArredondado;
    static Sprite spriteCirculo;
    static Sprite spriteCoracao;
    static Sprite spriteAltoFalante;
    static Sprite spriteEngrenagem;
    static Sprite spriteSeta;
    static Sprite spriteEstrela;
    static Sprite spriteXis;

    // — Sprites gerados por código —————

    // Retângulo de cantos arredondados com borda 9-slice
    // (pode esticar em qualquer tamanho sem deformar os cantos)
    public static Sprite Arredondado()
    {
        if (spriteArredondado != null) return spriteArredondado;

        int t = 64; float raio = 20f;
        var tex = new Texture2D(t, t, TextureFormat.ARGB32, false);
        for (int y = 0; y < t; y++)
            for (int x = 0; x < t; x++)
            {
                float dx = Mathf.Max(0f, Mathf.Max(raio - x, x - (t - 1 - raio)));
                float dy = Mathf.Max(0f, Mathf.Max(raio - y, y - (t - 1 - raio)));
                float d = Mathf.Sqrt(dx * dx + dy * dy);
                float a = Mathf.Clamp01(raio - d + 0.5f); // borda suavizada
                tex.SetPixel(x, y, new Color(1f, 1f, 1f, a));
            }
        tex.Apply();

        spriteArredondado = Sprite.Create(tex, new Rect(0, 0, t, t),
            new Vector2(0.5f, 0.5f), 100f, 0, SpriteMeshType.FullRect,
            new Vector4(raio + 2, raio + 2, raio + 2, raio + 2));
        return spriteArredondado;
    }

    // Círculo branco com borda suavizada
    public static Sprite Circulo()
    {
        if (spriteCirculo != null) return spriteCirculo;

        int t = 128; float c = (t - 1) / 2f; float raio = c - 1f;
        var tex = new Texture2D(t, t, TextureFormat.ARGB32, false);
        for (int y = 0; y < t; y++)
            for (int x = 0; x < t; x++)
            {
                float d = Mathf.Sqrt((x - c) * (x - c) + (y - c) * (y - c));
                float a = Mathf.Clamp01(raio - d + 0.5f);
                tex.SetPixel(x, y, new Color(1f, 1f, 1f, a));
            }
        tex.Apply();

        spriteCirculo = Sprite.Create(tex, new Rect(0, 0, t, t), new Vector2(0.5f, 0.5f), 100f);
        return spriteCirculo;
    }

    // Coração (usado para as vidas) - curva implícita clássica do coração
    public static Sprite Coracao()
    {
        if (spriteCoracao != null) return spriteCoracao;

        int t = 64;
        var tex = new Texture2D(t, t, TextureFormat.ARGB32, false);
        for (int py = 0; py < t; py++)
            for (int px = 0; px < t; px++)
            {
                // Converte o pixel para coordenadas -1.4..1.4 (coração cabe nisso)
                float x = (px - t / 2f) / (t * 0.36f);
                float y = (py - t / 2f) / (t * 0.36f) + 0.25f;
                // Equação do coração: (x^2 + y^2 - 1)^3 - x^2·y^3 < 0 -> dentro
                float f = Mathf.Pow(x * x + y * y - 1f, 3f) - x * x * y * y * y;
                tex.SetPixel(px, py, new Color(1, 1, 1, f < 0f ? 1f : 0f));
            }
        tex.Apply();
        spriteCoracao = Sprite.Create(tex, new Rect(0, 0, t, t), new Vector2(0.5f, 0.5f), 100f);
        return spriteCoracao;
    }
}
```



```

// Alto-falante com ondas de som (ícone do botão de música)
public static Sprite AltoFalante()
{
    if (spriteAltoFalante != null) return spriteAltoFalante;

    int t = 64;
    var tex = new Texture2D(t, t, TextureFormat.ARGB32, false);
    for (int y = 0; y < t; y++)
        for (int x = 0; x < t; x++)
        {
            bool dentro = false;

            // Caixinha do alto-falante
            if (x >= 8 && x <= 22 && y >= 24 && y <= 40) dentro = true;

            // Cone (triângulo abrindo para a direita)
            if (x > 22 && x <= 36)
            {
                float meiaAltura = 8f + (x - 22f) * 0.9f;
                if (Mathf.Abs(y - 32f) <= meiaAltura) dentro = true;
            }

            // Duas ondas de som (arcos à direita do cone)
            float dist = Mathf.Sqrt((x - 34f) * (x - 34f) + (y - 32f) * (y - 32f));
            if (x >= 42 && (Mathf.Abs(dist - 12f) <= 1.8f || Mathf.Abs(dist - 18f) <= 1.8f))
                dentro = true;

            tex.SetPixel(x, y, new Color(1, 1, 1, dentro ? 1f : 0f));
        }
    tex.Apply();
    spriteAltoFalante = Sprite.Create(tex, new Rect(0, 0, t, t), new Vector2(0.5f, 0.5f), 100f);
    return spriteAltoFalante;
}

// Engrenagem (icone do botao de opcoes): anel + 8 dentes + furo central
public static Sprite Engrenagem()
{
    if (spriteEngrenagem != null) return spriteEngrenagem;

    int t = 64; float c = (t - 1) / 2f;
    var tex = new Texture2D(t, t, TextureFormat.ARGB32, false);
    for (int y = 0; y < t; y++)
        for (int x = 0; x < t; x++)
        {
            float dx = x - c, dy = y - c;
            float dist = Mathf.Sqrt(dx * dx + dy * dy);
            float angulo = Mathf.Atan2(dy, dx) * Mathf.Rad2Deg + 180f; // 0..360

            bool dentro = false;

            // Corpo da engrenagem (anel cheio)
            if (dist >= 9f && dist <= 21f) dentro = true;

            // 8 dentes: fatias de 20 graus a cada 45 graus
            if (dist > 21f && dist <= 29f)
            {
                float resto = angulo % 45f;
                if (resto < 11f || resto > 34f) dentro = true;
            }

            tex.SetPixel(x, y, new Color(1f, 1f, 1f, dentro ? 1f : 0f));
        }
    tex.Apply();
    spriteEngrenagem = Sprite.Create(tex, new Rect(0, 0, t, t), new Vector2(0.5f, 0.5f), 100f);
    return spriteEngrenagem;
}

// Seta triangular apontando para a DIREITA (gire 180 graus para a esquerda)
public static Sprite Seta()
{
    if (spriteSeta != null) return spriteSeta;

    int t = 64;
    var tex = new Texture2D(t, t, TextureFormat.ARGB32, false);
    for (int y = 0; y < t; y++)
        for (int x = 0; x < t; x++)
        {
            // Triangulo com base em x=12 e ponta em x=54
            float meiaAltura = (54f - x) * (24f / 42f);
            bool dentro = x >= 12 && x <= 54 && Mathf.Abs(y - 32f) <= meiaAltura;
            tex.SetPixel(x, y, new Color(1f, 1f, 1f, dentro ? 1f : 0f));
        }
    tex.Apply();
    spriteSeta = Sprite.Create(tex, new Rect(0, 0, t, t), new Vector2(0.5f, 0.5f), 100f);
    return spriteSeta;
}

// Estrela de 5 pontas (icone do botao de treinamento). Monta os 10 vertices
// (ponta e vale alternados) e pinta o que estiver dentro do poligono.
public static Sprite Estrela()
{
    if (spriteEstrela != null) return spriteEstrela;

```

```

int t = 64; float c = (t - 1) / 2f;
float rExt = c - 2f, rInt = rExt * 0.40f;

var v = new Vector2[10];
for (int i = 0; i < 10; i++)
{
    float ang = Mathf.Deg2Rad * (90f + i * 36f); // começa no topo
    float r = (i % 2 == 0) ? rExt : rInt; // ponta / vale
    v[i] = new Vector2(c + r * Mathf.Cos(ang), c + r * Mathf.Sin(ang));
}

var tex = new Texture2D(t, t, TextureFormat.ARGB32, false);
for (int y = 0; y < t; y++)
for (int x = 0; x < t; x++)
    tex.SetPixel(x, y, new Color(1f, 1f, 1f,
        PontoNoPoligono(x + 0.5f, y + 0.5f, v) ? 1f : 0f));
tex.Apply();
spriteEstrela = Sprite.Create(tex, new Rect(0, 0, t, t), new Vector2(0.5f, 0.5f), 100f);
return spriteEstrela;
}

// "X" de duas barras cruzadas (icone do botao de sair)
public static Sprite Xis()
{
    if (spriteXis != null) return spriteXis;

    int t = 64; float meia = 5.5f; // meia-espessura das barras
    float ini = 14f, fim = 50f; // as barras vao de 14 a 50
    var tex = new Texture2D(t, t, TextureFormat.ARGB32, false);
    for (int y = 0; y < t; y++)
    for (int x = 0; x < t; x++)
    {
        bool naDiag1 = Mathf.Abs(x - y) <= meia; // "\"
        bool naDiag2 = Mathf.Abs(x + y - (t - 1)) <= meia; // "/"
        bool naFaixa = x >= ini - meia && x <= fim + meia &&
            y >= ini - meia && y <= fim + meia;
        tex.SetPixel(x, y, new Color(1f, 1f, 1f,
            (naFaixa && (naDiag1 || naDiag2)) ? 1f : 0f));
    }
    tex.Apply();
    spriteXis = Sprite.Create(tex, new Rect(0, 0, t, t), new Vector2(0.5f, 0.5f), 100f);
    return spriteXis;
}

// Ponto dentro de um poligono, pela regra do numero de cruzamentos
static bool PontoNoPoligono(float px, float py, Vector2[] poly)
{
    bool dentro = false;
    for (int i = 0, j = poly.Length - 1; i < poly.Length; j = i++)
    {
        if ((poly[i].y > py) != (poly[j].y > py) &&
            px < (poly[j].x - poly[i].x) * (py - poly[i].y) /
                (poly[j].y - poly[i].y) + poly[i].x)
            dentro = !dentro;
    }
    return dentro;
}

// Gradiente vertical (usado como fundo do menu)
public static Sprite Gradiente(Color topo, Color baixo)
{
    var tex = new Texture2D(1, 256, TextureFormat.ARGB32, false);
    for (int y = 0; y < 256; y++)
        tex.SetPixel(0, y, Color.Lerp(baixo, topo, y / 255f));
    tex.Apply();
    tex.wrapMode = TextureWrapMode.Clamp;
    return Sprite.Create(tex, new Rect(0, 0, 1, 256), new Vector2(0.5f, 0.5f), 100f);
}

// — Criadores de elementos —————

public static Image CriarImagem(Transform pai, string nome, Color cor,
    Vector2 pos, Vector2 tamanho,
    Sprite sprite = null, bool fatiado = false)
{
    var go = new GameObject(nome, typeof(RectTransform), typeof(CanvasRenderer), typeof(Image));
    go.layer = 5; // camada UI
    var rt = go.GetComponent<RectTransform>();
    rt.SetParent(pai, false);
    rt.anchoredPosition = pos;
    rt.sizeDelta = tamanho;

    var img = go.GetComponent<Image>();
    img.color = cor;
    if (sprite != null)
    {
        img.sprite = sprite;
        if (fatiado) img.type = Image.Type.Sliced;
    }
    return img;
}

```

```

public static TextMeshProUGUI CriarTexto(Transform pai, string nome, string texto,
                                         float tamanhoFonte, Color cor,
                                         Vector2 pos, Vector2 tamanho, bool negrito = true)
{
    var go = new GameObject(nome, typeof(RectTransform));
    go.layer = 5;
    var rt = go.GetComponent<RectTransform>();
    rt.SetParent(pai, false);
    rt.anchoredPosition = pos;
    rt.sizeDelta = tamanho;

    var tmp = go.AddComponent<TextMeshProUGUI>();
    tmp.text = texto;
    tmp.fontSize = tamanhoFonte;
    tmp.color = cor;
    tmp.alignment = TextAlignmentOptions.Center;
    tmp.fontStyle = negrito ? FontStyles.Bold : FontStyles.Normal;
    tmp.raycastTarget = false; // texto não bloqueia cliques
    return tmp;
}

// Botão completo: fundo arredondado + rótulo + círculo de progresso da mão.
// Funciona com toque/mouse (Button) E com a mão parada 3s (HoverButton).
public static Button CriarBotao(Transform pai, string nome, string rotulo, Color corFundo,
                                Vector2 pos, Vector2 tamanho, float tamanhoFonte,
                                ControladorCamera controlador, UnityAction acao)
{
    var img = CriarImagem(pai, nome, corFundo, pos, tamanho, Arredondado(), true);
    var go = img.gameObject;

    var botao = go.AddComponent<Button>();
    botao.targetGraphic = img;
    if (acao != null) botao.onClick.AddListener(acao);

    // O rótulo ESTICA junto com o botão (se o botão for redimensionado
    // depois, o texto acompanha) e a fonte encolhe sozinha até caber
    var rotuloTmp = CriarTexto(go.transform, "Rotulo", rotulo, tamanhoFonte,
                               Color.white, Vector2.zero, tamanho);
    var rotuloRt = rotuloTmp.rectTransform;
    rotuloRt.anchorMin = Vector2.zero;
    rotuloRt.anchorMax = Vector2.one;
    rotuloRt.anchoredPosition = Vector2.zero;
    rotuloRt.sizeDelta = new Vector2(-28, -12); // folga nas bordas
    rotuloTmp.enableAutoSizing = true;
    rotuloTmp.fontSizeMax = tamanhoFonte;
    rotuloTmp.fontSizeMin = 16f;

    // Círculo que enche enquanto a mão fica sobre o botão
    float diametro = Mathf.Min(tamanho.x, tamanho.y) * 0.85f;
    var circulo = CriarImagem(go.transform, "CirculoHover",
                              new Color(0.2f, 0.8f, 1f, 0.75f), Vector2.zero,
                              new Vector2(diametro, diametro), Circulo());
    circulo.type = Image.Type.Filled;
    circulo.fillMethod = Image.FillMethod.Radial360;
    circulo.fillOrigin = (int)Image.Origin360.Top;
    circulo.fillAmount = 0f;
    circulo.raycastTarget = false;

    var hover = go.AddComponent<HoverButton>();
    hover.controlador = controlador;
    hover.imagemCirculo = circulo;

    return botao;
}

// Prende o elemento em um canto/borda da tela (âncora e pivô juntos)
public static void Ancorar(Component alvo, Vector2 ancora, Vector2 pivo)
{
    var rt = alvo.transform as RectTransform;
    rt.anchorMin = ancora;
    rt.anchorMax = ancora;
    rt.pivot = pivo;
}
}

```

Assets/VisualizadorMaoUI.cs (163 linhas)

```
using UnityEngine;
using UnityEngine.UI;
using System.Collections.Generic;

// VisualizadorMaoUI: desenha o esqueleto da mão por cima do feed da câmera.
//
// Ele se configura SOZINHO: no início vira uma camada de tela cheia dentro do
// Canvas (logo acima da imagem da câmera) e usa a conversão de coordenadas do
// ControladorCamera - que já desconta o recorte e o espelho da imagem.
// Não é preciso ajustar rotação/escala manualmente no Inspector.
public class VisualizadorMaoUI : MonoBehaviour
{
    [Header("Fonte dos dados da mao")]
    public ControladorCamera controlador;

    [Header("Visual")]
    public Color corLinha = new Color(0f, 0.85f, 0.85f, 0.9f); // ciano
    public Color corPonto = new Color(1f, 1f, 1f, 0.95f); // branco
    public Color corPulso = new Color(1f, 0.8f, 0.1f, 1f); // amarelo - ponto 0

    [Range(1f, 8f)] public float espessuraLinha = 4f;
    [Range(6f, 24f)] public float tamanhoPonto = 14f;

    [Header("Suavizacao (maior = responde mais rapido; menor = mais macio)")]
    [Range(4f, 30f)] public float suavizacao = 14f;

    // Conexões do esqueleto (pares de índices dos 21 keypoints do MediaPipe)
    private static readonly int[,] OSSOS =
    {
        // Polegar
        {0,1},{1,2},{2,3},{3,4},
        // Indicador
        {0,5},{5,6},{6,7},{7,8},
        // Médio
        {0,9},{9,10},{10,11},{11,12},
        // Anelar
        {0,13},{13,14},{14,15},{15,16},
        // Mínimo
        {0,17},{17,18},{18,19},{19,20},
        // Palma (nós entre os dedos)
        {5,9},{9,13},{13,17}
    };

    private RectTransform meuRect;
    private Camera cameraUI;
    private List<RectTransform> linhas = new List<RectTransform>();
    private List<RectTransform> pontos = new List<RectTransform>();

    private Vector2[] suavizados = new Vector2[21];
    private bool temPosicaoAnterior = false;

    void Awake()
    {
        var canvas = GetComponentInParent<Canvas>();
        meuRect = (RectTransform)transform;

        // Vira uma camada de tela cheia direto no Canvas.
        // Isso desfaz qualquer rotação/escala antiga que atrapalhava o alinhamento.
        meuRect.SetParent(canvas.transform, false);
        meuRect.SetAsLastSibling(); // por cima de TUDO: funciona como um "cursor"
        meuRect.localRotation = Quaternion.identity;
        meuRect.localScale = Vector3.one;
        meuRect.anchorMin = Vector2.zero;
        meuRect.anchorMax = Vector2.one;
        meuRect.pivot = new Vector2(0.5f, 0.5f);
        meuRect.anchoredPosition = Vector2.zero;
        meuRect.sizeDelta = Vector2.zero;

        // Câmera usada pelo Canvas (necessária para converter tela -> local)
        cameraUI = (canvas.renderMode != RenderMode.ScreenSpaceOverlay)
            ? canvas.worldCamera : null;

        int numOssos = OSSOS.GetLength(0);
        for (int i = 0; i < numOssos; i++) linhas.Add(CriarLinha());
        for (int i = 0; i < 21; i++) pontos.Add(CriarPonto(i));
    }

    RectTransform CriarLinha()
    {
        var go = new GameObject("_Linha", typeof(Image));
        go.layer = 5;
        go.transform.SetParent(transform, false);
        var img = go.GetComponent<Image>();
        img.color = corLinha;
        img.raycastTarget = false; // IMPORTANTE: não pode roubar cliques dos botões!
        var rt = go.GetComponent<RectTransform>();
        rt.sizeDelta = new Vector2(0, espessuraLinha);
        rt.pivot = new Vector2(0.5f, 0.5f);
    }
}
```

```

    return rt;
}

RectTransform CriarPonto(int indice)
{
    var go = new GameObject("_Ponto" + indice, typeof(Image));
    go.layer = 5;
    go.transform.SetParent(transform, false);
    var img = go.GetComponent<Image>();
    img.color = (indice == 0) ? corPulso : corPonto;
    img.sprite = UIFabrica.Circulo(); // bolinha redonda em vez de quadrado
    img.raycastTarget = false;
    var rt = go.GetComponent<RectTransform>();
    rt.sizeDelta = new Vector2(tamanhoPonto, tamanhoPonto);
    rt.pivot = new Vector2(0.5f, 0.5f);
    return rt;
}

void Update()
{
    bool visivel = controlador != null && controlador.MaoDetectada
        && controlador.PontosDaMaoAtuais != null;

    for (int i = 0; i < linhas.Count; i++) linhas[i].gameObject.SetActive(visivel);
    for (int i = 0; i < pontos.Count; i++) pontos[i].gameObject.SetActive(visivel);

    if (!visivel)
    {
        temPosicaoAnterior = false; // quando a mão voltar, "teleporta" em vez de deslizar
        return;
    }

    // Mantém o esqueleto acima de tudo (o menu usa SetAsLastSibling também,
    // então reconquistamos o topo sempre que a mão está visível)
    if (meuRect.GetSiblingIndex() != meuRect.parent.childCount - 1)
        meuRect.SetAsLastSibling();

    Vector3[] pts = controlador.PontosDaMaoAtuais;

    // Suavização exponencial: cada ponto desliza em direção à posição nova.
    // Independente do FPS graças ao Time.deltaTime.
    float fator = 1f - Mathf.Exp(-suavizacao * Time.deltaTime);

    for (int i = 0; i < 21; i++)
    {
        Vector2 posTela = controlador.PontoParaTela(pts[i]);
        Vector2 posLocal;
        RectTransformUtility.ScreenPointToLocalPointInRectangle(
            meuRect, posTela, cameraUI, out posLocal);

        suavizados[i] = temPosicaoAnterior
            ? Vector2.Lerp(suavizados[i], posLocal, fator)
            : posLocal;
    }
    temPosicaoAnterior = true;

    for (int i = 0; i < 21 && i < pontos.Count; i++)
        pontos[i].anchoredPosition = suavizados[i];

    for (int i = 0; i < OSSOS.GetLength(0); i++)
        PosicionarLinha(linhas[i], suavizados[OSSOS[i, 0]], suavizados[OSSOS[i, 1]]);
}

void PosicionarLinha(RectTransform rt, Vector2 a, Vector2 b)
{
    Vector2 dir = b - a;
    float len = dir.magnitude;
    float ang = Mathf.Atan2(dir.y, dir.x) * Mathf.Rad2Deg;

    rt.anchoredPosition = (a + b) * 0.5f;
    rt.sizeDelta = new Vector2(len, espessuraLinha);
    rt.localEulerAngles = new Vector3(0, 0, ang);
}
}

```

RastreadorPython/gerar_figuras.py (149 linhas)

```
# -*- coding: utf-8 -*-
"""
Gera as imagens comparativas (antes/depois) do relatorio de Visao Computacional.

Produz, a partir de UM quadro da webcam (ou de uma foto), um painel com as
etapas do processamento lado a lado, exatamente o que o professor recomendou:
    1) quadro original capturado
    2) quadro reduzido para 640 px de largura (entrada da inferencia)
    3) recorte da regio da mao (ROI)
    4) esqueleto de 21 pontos sobreposto (saida do rastreamento)

USO:
    py gerar_figuras.py                -> captura da webcam (aperte ESPACO)
    py gerar_figuras.py foto.jpg       -> usa uma foto existente

Saidas (nesta pasta): fig_1_original.png, fig_2_reduzido.png,
fig_3_recorte.png, fig_4_esqueleto.png e fig_painel.png (as quatro juntas).
"""

import os
import sys
import cv2
import numpy as np
import mediapipe as mp
from mediapipe.tasks import python as mp_tasks
from mediapipe.tasks.python import vision

PASTA = os.path.dirname(os.path.abspath(__file__))
MODELO = os.path.join(PASTA, "hand_landmarker.task")

# Conexoes do esqueleto (mesmos ossos usados no jogo)
OSSOS = [
    (0,1),(1,2),(2,3),(3,4), (0,5),(5,6),(6,7),(7,8),
    (0,9),(9,10),(10,11),(11,12), (0,13),(13,14),(14,15),(15,16),
    (0,17),(17,18),(18,19),(19,20), (5,9),(9,13),(13,17),
]

def capturar_da_webcam():
    cam = cv2.VideoCapture(0, cv2.CAP_DSHOW)
    cam.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
    cam.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)
    print("Enquadre a mao e aperte ESPACO para capturar (ou ESC para sair).")
    quadro = None
    while True:
        ok, q = cam.read()
        if not ok:
            continue
        cv2.imshow("Captura (ESPACO = tirar foto)", q)
        tecla = cv2.waitKey(1) & 0xFF
        if tecla == 32: # espaco
            quadro = q.copy()
            break
        if tecla == 27: # esc
            break
    cam.release()
    cv2.destroyAllWindows()
    return quadro

def rotular(img, texto):
    """Escreve um titulo com faixa preta no topo da imagem."""
    img = img.copy()
    cv2.rectangle(img, (0, 0), (img.shape[1], 34), (0, 0, 0), -1)
    cv2.putText(img, texto, (10, 24), cv2.FONT_HERSHEY_SIMPLEX, 0.7,
                (255, 255, 255), 2, cv2.LINE_AA)
    return img

def principal():
    if not os.path.exists(MODELO):
        print("ERRO: modelo hand_landmarker.task nao encontrado.")
        return

    if len(sys.argv) > 1:
        original = cv2.imread(sys.argv[1])
        if original is None:
            print("ERRO: nao consegui abrir a imagem", sys.argv[1])
            return
    else:
        original = capturar_da_webcam()
        if original is None:
            print("Cancelado.")
            return

    # — Etapa 1: original —————
    h0, w0 = original.shape[:2]
```

```

# — Etapa 2: reduzido para 640 de largura —————
escala = 640.0 / w0
reduzido = cv2.resize(original, (640, int(h0 * escala)))

# — Deteccao dos pontos —————
detector = vision.HandLandmarker.create_from_options(
    vision.HandLandmarkerOptions(
        base_options=mp_tasks.BaseOptions(model_asset_path=MODELO),
        running_mode=vision.RunningMode.IMAGE, num_hands=1))
rgb = cv2.cvtColor(reduzido, cv2.COLOR_BGR2RGB)
resultado = detector.detect(mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb))

esqueleto = reduzido.copy()
recorte = reduzido.copy()
if resultado.hand_landmarks:
    m = resultado.hand_landmarks[0]
    hh, ww = reduzido.shape[:2]
    pts = [(int(p.x * ww), int(p.y * hh)) for p in m]

    # — Etapa 4: esqueleto sobreposto —————
    for (a, b) in OSSOS:
        cv2.line(esqueleto, pts[a], pts[b], (230, 220, 0), 2, cv2.LINE_AA)
    for i, (x, y) in enumerate(pts):
        cor = (0, 200, 255) if i == 0 else (255, 255, 255)
        cv2.circle(esqueleto, (x, y), 4, cor, -1, cv2.LINE_AA)

    # — Etapa 3: recorte da ROI (caixa da mao com margem) —————
    xs = [p[0] for p in pts]; ys = [p[1] for p in pts]
    margem = 30
    x1 = max(0, min(xs) - margem); y1 = max(0, min(ys) - margem)
    x2 = min(ww, max(xs) + margem); y2 = min(hh, max(ys) + margem)
    recorte = reduzido[y1:y2, x1:x2].copy()
else:
    print("AVISO: nenhuma mao detectada. Repita com a mao bem enquadrada.")

# — Salva as quatro imagens —————
f1 = rotular(original, "1) Original " + str(w0) + "x" + str(h0))
f2 = rotular(reduzido, "2) Reduzido 640 (entrada da IA)")
f3 = rotular(recorte, "3) Recorte da mao (ROI)")
f4 = rotular(esqueleto, "4) 21 pontos (saida)")
cv2.imwrite(os.path.join(PASTA, "fig_1_original.png"), f1)
cv2.imwrite(os.path.join(PASTA, "fig_2_reduzido.png"), f2)
cv2.imwrite(os.path.join(PASTA, "fig_3_recorte.png"), f3)
cv2.imwrite(os.path.join(PASTA, "fig_4_esqueleto.png"), f4)

# — Painel: original e esqueleto lado a lado (mesma altura) —————
alvo_h = 480
def redim(img):
    r = alvo_h / img.shape[0]
    return cv2.resize(img, (int(img.shape[1] * r), alvo_h))
painel = np.hstack([redim(f2), redim(f4)])
cv2.imwrite(os.path.join(PASTA, "fig_painel.png"), painel)

print("Imagens geradas nesta pasta:")
print(" fig_1_original.png, fig_2_reduzido.png, fig_3_recorte.png,")
print(" fig_4_esqueleto.png e fig_painel.png (entrada + saida lado a lado).")

if __name__ == "__main__":
    principal()

```

RastreadorPython/importar_dataset.py (179 linhas)

```
# -*- coding: utf-8 -*-
"""
Importador de dataset de alfabeto em LIBRAS para o DitadoEstrelado.

Pega uma pasta de FOTOS organizadas por letra, passa cada foto pelo
MediaPipe (offline), extrai os 21 pontos da mao e adiciona as amostras
direto no banco do jogo (Assets/MeuAlfabeto.asset).

Resultado: o jogo passa a reconhecer maos DIFERENTES da sua - crucial
para a demo (a banca pode testar com a propria mao!).

USO:
1) Baixe um dataset de alfabeto em LIBRAS (ex: Kaggle "libras alphabet")
   e descompacte. A pasta deve ter uma SUBPASTA POR LETRA:
       dataset/
         A/ foto1.jpg foto2.jpg ...
         B/ ...
2) FECHE o Unity (ou nao deixe o MeuAlfabeto aberto no Inspector)
3) Rode: py importar_dataset.py caminho/da/pasta/dataset
   Opcoes: --max 15 (maximo de amostras novas por letra)
           --espelhar (tambem adiciona a versao espelhada = outra mao)
4) Abra o Unity - ele recarrega o banco sozinho

Um backup do MeuAlfabeto.asset e criado automaticamente antes de mexer.
"""

import argparse
import os
import shutil
import sys
import time

import mediapipe as mp
from mediapipe.tasks import python as mp_tasks
from mediapipe.tasks.python import vision

PASTA = os.path.dirname(os.path.abspath(__file__))
MODELO = os.path.join(PASTA, "hand_landmarker.task")
ASSET = os.path.normpath(os.path.join(PASTA, "..", "Assets", "MeuAlfabeto.asset"))

# Letras com MOVIMENTO nao podem vir de fotos (foto nao tem movimento)
LETRAS_DINAMICAS = {"H", "J", "K", "W", "X", "Z", "Ç", "C_CEDILHA"}

EXTENSOES = {".jpg", ".jpeg", ".png", ".bmp", ".webp"}

def extrair_pontos(detector, caminho_imagem):
    """Retorna 21 pontos relativos ao pulso (convencao do jogo) ou None."""
    try:
        imagem = mp.Image.create_from_file(caminho_imagem)
    except Exception:
        return None

    resultado = detector.detect(imagem)
    if not resultado.hand_landmarks:
        return None

    marcos = resultado.hand_landmarks[0]
    # Converte para a convencao do jogo: Y para CIMA, relativo ao pulso
    absolutos = [(m.x, 1.0 - m.y, m.z) for m in marcos]
    px, py, pz = absolutos[0]
    return [(x - px, y - py, z - pz) for (x, y, z) in absolutos]

def formatar_amostra(letra, pontos):
    """Gera o bloco YAML de uma amostra no formato do MeuAlfabeto.asset."""
    linhas = [f"  nome: {letra}", "  pontosNormalizados:"]
    for (x, y, z) in pontos:
        linhas.append(f"    - {{x: {x:.8f}, y: {y:.8f}, z: {z:.8f}}}")
    return "\n".join(linhas)

def principal():
    parser = argparse.ArgumentParser(description="Importa fotos de LIBRAS para o banco do jogo")
    parser.add_argument("pasta", help="pasta do dataset (uma subpasta por letra)")
    parser.add_argument("--max", type=int, default=15, help="maximo de amostras novas por letra")
    parser.add_argument("--espelhar", action="store_true",
                        help="adiciona tambem a versao espelhada (cobre a outra mao)")
    args = parser.parse_args()

    if not os.path.isdir(args.pasta):
        print("ERRO: pasta nao encontrada:", args.pasta)
        return
    if not os.path.exists(ASSET):
        print("ERRO: MeuAlfabeto.asset nao encontrado em:", ASSET)
        return

    # Detector no modo IMAGEM (uma foto por vez)
```



```

opcoes = vision.HandLandmarkerOptions(
    base_options=mp_tasks.BaseOptions(model_asset_path=MODELO),
    running_mode=vision.RunningMode.IMAGE,
    num_hands=1,
    min_hand_detection_confidence=0.5,
)
detector = vision.HandLandmarker.create_from_options(opcoes)
print("[OK] Detector MediaPipe carregado")

# Percorre as subpastas de letras
blocos = []
resumo = {}
for nome_sub in sorted(os.listdir(args.pasta)):
    sub = os.path.join(args.pasta, nome_sub)
    if not os.path.isdir(sub):
        continue
    letra = nome_sub.strip().upper()
    if len(letra) != 1 or not letra.isalpha():
        print(f" (pulando pasta '{nome_sub}': nome nao e uma letra)")
        continue
    if letra in LETRAS_DINAMICAS:
        print(f" (pulando '{letra}': letra com movimento nao vem de foto)")
        continue

    adicionadas = 0
    for arquivo in sorted(os.listdir(sub)):
        if adicionadas >= args.max:
            break
        if os.path.splitext(arquivo)[1].lower() not in EXTENSOES:
            continue

        pontos = extrair_pontos(detector, os.path.join(sub, arquivo))
        if pontos is None:
            continue # sem mao detectada nessa foto

        blocos.append(formatar_amostra(letra, pontos))
        adicionadas += 1

        if args.espelhar and adicionadas < args.max:
            espelhado = [(-x, y, z) for (x, y, z) in pontos]
            blocos.append(formatar_amostra(letra, espelhado))
            adicionadas += 1

    if adicionadas > 0:
        resumo[letra] = adicionadas
        print(f" [{letra}] {adicionadas} amostras extraidas")

if not blocos:
    print("Nenhuma amostra extraida - confira a organizacao da pasta.")
    return

# Backup e insercao no asset
backup = ASSET + ".backup-" + time.strftime("%Y%m%d-%H%M%S")
shutil.copy2(ASSET, backup)
print("[OK] Backup criado:", os.path.basename(backup))

with open(ASSET, "r", encoding="utf-8") as f:
    conteudo = f.read()

novo_texto = "\n".join(blocos) + "\n"

if " letrasGravadas: []" in conteudo:
    # Banco vazio: abre a lista
    conteudo = conteudo.replace(" letrasGravadas: []",
                                " letrasGravadas:\n" + novo_texto.rstrip("\n"))
elif "\n sinaisDinamicos:" in conteudo:
    # Insere as amostras novas no FIM da lista de letras estaticas
    conteudo = conteudo.replace("\n sinaisDinamicos:",
                                "\n" + novo_texto + " sinaisDinamicos:")
else:
    # Formato antigo sem sinais dinamicos: anexa no final
    conteudo = conteudo.rstrip("\n") + "\n" + novo_texto

with open(ASSET, "w", encoding="utf-8", newline="\n") as f:
    f.write(conteudo)

total = sum(resumo.values())
print()
print(f"[SUCESSO] {total} amostras adicionadas ao MeuAlfabeto.asset")
print("Letras:", " ".join(f"{l}: {n}" for l, n in sorted(resumo.items())))
print()
print("Agora abra o Unity - o banco novo carrega sozinho.")
print("Se algo der errado, restaure o backup:", os.path.basename(backup))

if __name__ == "__main__":
    try:
        principal()
    except KeyboardInterrupt:
        print("\nCancelado.")

```

RastreadorPython/rastreador_maos.py (146 linhas)

```
# -*- coding: utf-8 -*-
"""
Rastreador de maos com MediaPipe (Google) para o DitadoEstrelado.

Roda 100% OFFLINE: captura a webcam, detecta os 21 pontos da mao com o
pipeline completo do MediaPipe (deteccao de palma + landmarks + rastreamento
temporal) e envia tudo para o Unity por UDP em 127.0.0.1 (comunicacao interna
do proprio PC - nao usa internet).

Como usar: execute EXECUTAR_RASTREADOR.bat (na pasta do projeto) ANTES de
dar Play no Unity. Se este script nao estiver rodando, o jogo usa o
rastreador interno antigo automaticamente.

Pacotes: py -m pip install mediapipe opencv-python
Modelo: hand_landmarker.task (nesta pasta, ja baixado)
"""

import os
import socket
import struct
import time

import cv2
import mediapipe as mp
from mediapipe.tasks import python as mp_tasks
from mediapipe.tasks.python import vision

# — Configuracao —
IP_UNITY = "127.0.0.1" # localhost: mesmo PC, sem internet
PORTA_LANDMARKS = 5052 # pontos da mao
PORTA_VIDEO = 5053 # imagem da camera (JPEG)
LARGURA_CAPTURA = 640 # resolucao pedida a webcam
ALTURA_CAPTURA = 480
LARGURA_ENVIO = 640 # video reduzido p/ caber em 1 pacote UDP

PASTA = os.path.dirname(os.path.abspath(__file__))
MODELO = os.path.join(PASTA, "hand_landmarker.task")

def principal():
    print("=" * 60)
    print(" RASTREADOR DE MAOS - DitadoEstrelado (MediaPipe/Google)")
    print("=" * 60)

    if not os.path.exists(MODELO):
        print("ERRO: modelo 'hand_landmarker.task' nao encontrado em:", PASTA)
        return

    # Detector no modo VIDEO: usa o resultado do frame anterior para rastrear
    # (mais estavel e rapido do que detectar do zero a cada frame)
    opcoes = vision.HandLandmarkerOptions(
        base_options=mp_tasks.BaseOptions(model_asset_path=MODELO),
        running_mode=vision.RunningMode.VIDEO,
        num_hands=1,
        min_hand_detection_confidence=0.5,
        min_hand_presence_confidence=0.5,
        min_tracking_confidence=0.5,
    )
    detector = vision.HandLandmarker.create_from_options(opcoes)
    print("[OK] Detector MediaPipe carregado (modelo local, offline)")

    # CAP_DSHOW abre a camera mais rapido no Windows
    camera = cv2.VideoCapture(0, cv2.CAP_DSHOW)
    # MJPG: formato comprimido - sem ele, muitas webcams travam em 10fps no 720p
    camera.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc("MJPG"))
    camera.set(cv2.CAP_PROP_FRAME_WIDTH, LARGURA_CAPTURA)
    camera.set(cv2.CAP_PROP_FRAME_HEIGHT, ALTURA_CAPTURA)
    camera.set(cv2.CAP_PROP_FPS, 30)
    if not camera.isOpened():
        print("ERRO: nao consegui abrir a webcam.")
        print("Dica: feche o Unity (se estiver em Play) e outros apps de camera.")
        return
    largura = int(camera.get(cv2.CAP_PROP_FRAME_WIDTH))
    altura = int(camera.get(cv2.CAP_PROP_FRAME_HEIGHT))
    print(f"[OK] Webcam aberta: {largura}x{altura}")

    soquete = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    print(f"[OK] Enviando para o Unity em {IP_UNITY}:{PORTA_LANDMARKS} (pontos) e :{PORTA_VIDEO} (video)")
    print()
    print(">>> Deixe esta janela ABERTA e de Play no Unity. (Ctrl+C encerra) <<<")
    print()

    inicio = time.time()
    quadros = 0
    detecoos = 0
    ultimo_relatorio = time.time()

    while True:
```

```

ok, quadro = camera.read()
if not ok:
    time.sleep(0.01)
    continue

# Reduz o quadro ANTES de tudo: o MediaPipe redimensiona internamente
# de qualquer forma, entao detectar no quadro menor nao perde precisao
# e dobra a velocidade (os pontos sao normalizados 0-1, independem
# da resolucao)
altura_envio = int(LARGURA_ENVIO * altura / largura)
reduzido = cv2.resize(quadro, (LARGURA_ENVIO, altura_envio))

# — Deteccao —————
rgb = cv2.cvtColor(reduzido, cv2.COLOR_BGR2RGB)
imagem = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb)
carimbo_ms = int((time.time() - inicio) * 1000)
resultado = detector.detect_for_video(imagem, carimbo_ms)

confianca = 0.0
pontos = [0.0] * 63 # 21 pontos x (x, y, z)
if resultado.hand_landmarks:
    marcos = resultado.hand_landmarks[0]
    confianca = (resultado.handedness[0][0].score
                 if resultado.handedness else 1.0)
    for i, marco in enumerate(marcos):
        pontos[i * 3 + 0] = marco.x
        pontos[i * 3 + 1] = 1.0 - marco.y # Unity usa Y para CIMA
        pontos[i * 3 + 2] = marco.z      # profundidade relativa!
    detecoos += 1

# Pacote binario: byte magico 'M' + confianca + 63 floats
pacote = struct.pack("<Bf63f", 0x4D, confianca, *pontos)
soquete.sendto(pacote, (IP_UNITY, PORTA_LANDMARKS))

# — Video para o Unity (JPEG reduzido) —————
ok_jpg, jpg = cv2.imencode(".jpg", reduzido,
                           [cv2.IMWRITE_JPEG_QUALITY, 70])
if ok_jpg and len(jpg) < 65000: # limite de 1 datagrama UDP
    soquete.sendto(jpg.tobytes(), (IP_UNITY, PORTA_VIDEO))

# — Relatorio no console a cada 5s —————
quadros += 1
agora = time.time()
if agora - ultimo_relatorio >= 5.0:
    fps = quadros / (agora - ultimo_relatorio)
    taxa = 100.0 * detecoos / max(1, quadros)
    print(f" {fps:5.1f} fps | mao detectada em {taxa:4.1f}% dos quadros")
    quadros = 0
    detecoos = 0
    ultimo_relatorio = agora

if __name__ == "__main__":
    try:
        principal()
    except KeyboardInterrupt:
        print("\nEncerrado pelo usuario. Ate mais!")

```